



Operating System

(Principles of Operating Systems)

Đỗ Quốc Huy

huydq@soict.hust.edu.vn

Department of Computer Science

School of Information and Communication Technology

ONE LOVE. ONE FUTURE.

① Process's definition

Process (review)

- A **running program**
 - Provided resources (CPU, memory, I/O devices. . .) to complete its task
 - Resources are provided at:
 - **process creating time**
 - While **running**
- system includes many processes running at the same time
 - **OS's** : Perform system instruction code
 - **User's**: Perform user's code
- May contain ≥ 1 **threads**

- OS's roles:
 - Guarantee process and thread activities
 - Create/delete process (user, system)
 - Process scheduling
 - Provide synchronization mechanism, communication and prevent deadlock among processes

- ① Process
- ② Thread
- ③ CPU scheduling
- ④ Critical resource and process synchronization
- ⑤ Deadlock and solutions

- Concept of process
- Process Scheduling
- Operations on process
- Process cooperation
- Inter-process communication

- **System's state**
 - Processor: Registers' values
 - Memory: Content of memory block
 - Peripheral devices: Devices' status
- Program's executing $\Rightarrow$ system's state changing
 - Change discretely, follow each executed instruction



- Process: a sequence of system's state changing
 - Begin with start state
 - Change from state to state is done according to requirement in user's program

Process is the execution of a program



Process >< program

```
#include <stdio.h>
int main() {
    printf("hello world!");
    return 0;
}
```

\$gcc hello.c

Executable
(a.out)

\$/a.out

Process

Disk

Ram

- **Program:** passive object (content of a file on disk)
 - Program's **Code:** *machine instruction* (CD2190EA...)
 - **Data:**
 - Variable stored and used in memory
 - Global variable
 - Dynamic allocation variable (malloc, new,...)
 - Stack variable (function's parameter, local variable)
 - Dynamic linked library (DLL)
 - Not compiled and lined with program

Process >< program

```
#include <stdio.h>
int main() {
    printf("hello world!");
    return 0;
}
```

\$gcc hello.c

Executable
(a.out)

\$/a.out

Process

Disk

Ram

When program is running,
minimum resource
requirement

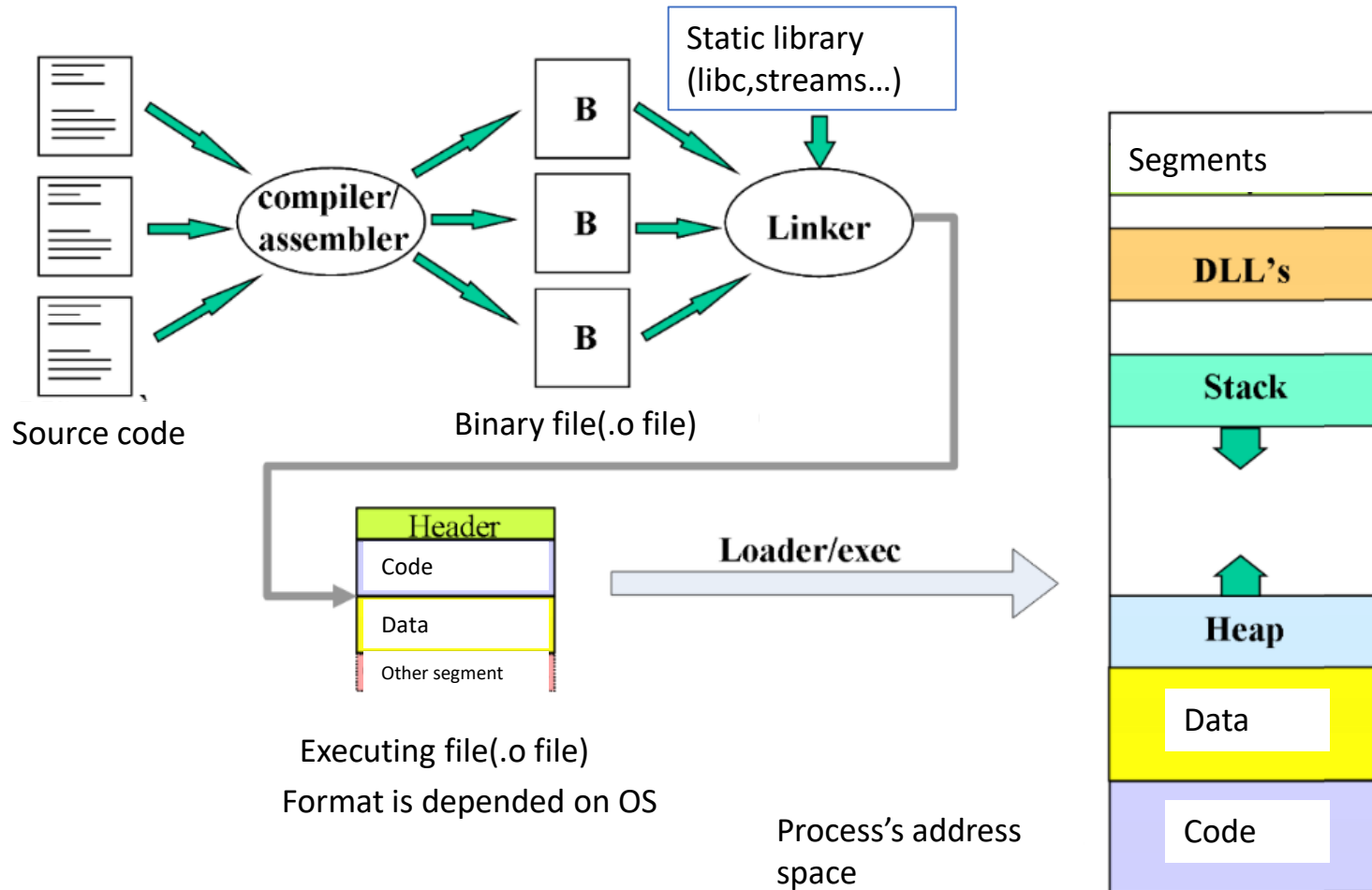
- **Memory** for program's **code** and **data**
- Processor's **registers** used for program **execution**

Process >< program

(Cont.)

- **Process:** active object (instruction pointer, set of resources)
- A program can be
- Part of process's **state**
 - One program, many process (different data set)
 - Example: `gcc hello.c || gcc baitap.c`
- Call to many process

Compile and run a program



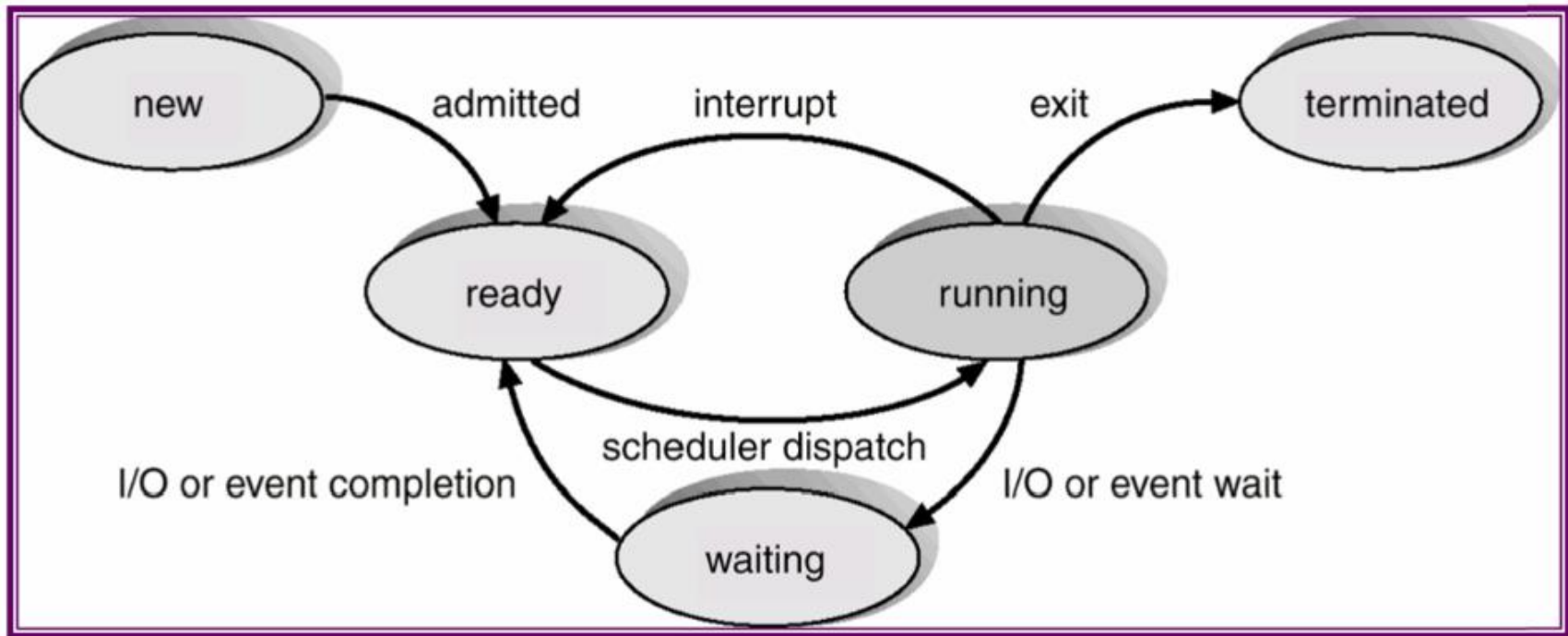
Compile and run a program

- The OS creates a process and allocate a memory area for it
- System program loader/exec
 - Read and interprets the executive file (file's header)
 - Set up address space for process to store code and data from executive file
 - Put command's paramenters, environment variable (argc, argv, envp) into stack
 - Set proper values for processor's register and call "_start()" (OS's fucntion)
- Program begins to run at "_start()". This function call to main() (program's functions)
 - ⇒ "Process" is running, not mention "program" anymore
- When main() function end, OS call to "_exit()" to cancel the process and take back resources



Process's state

Process's state is part of process's current activity

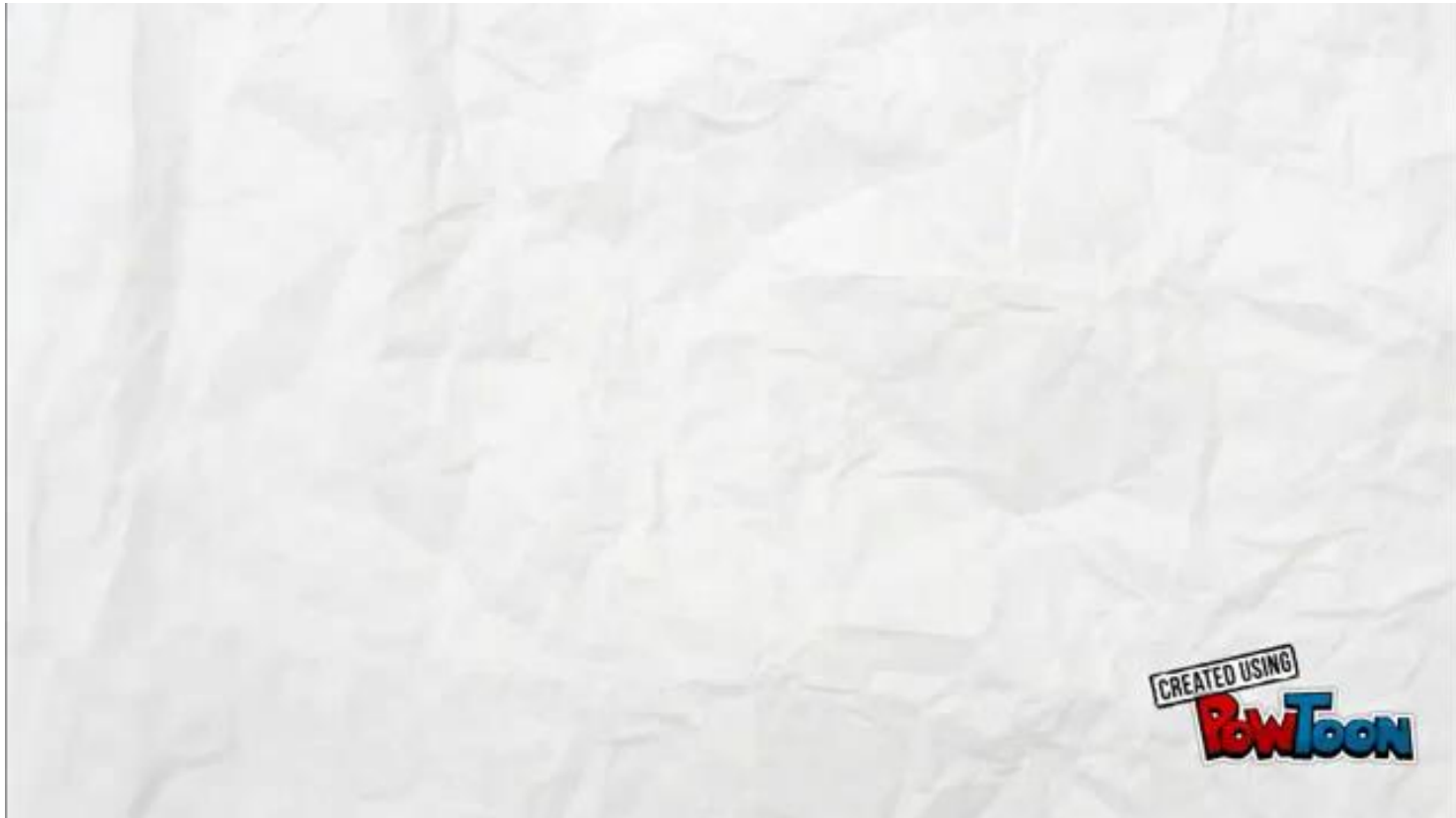


Process's states changing flowchart [Silberschatz]

System that has only 1 processor

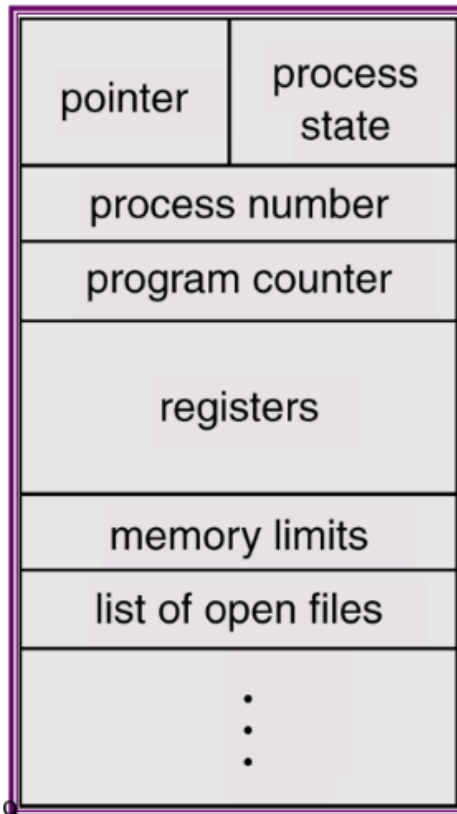
- Only 1 process in **running** state
- Many processes in ready or **waiting** state

Process's state



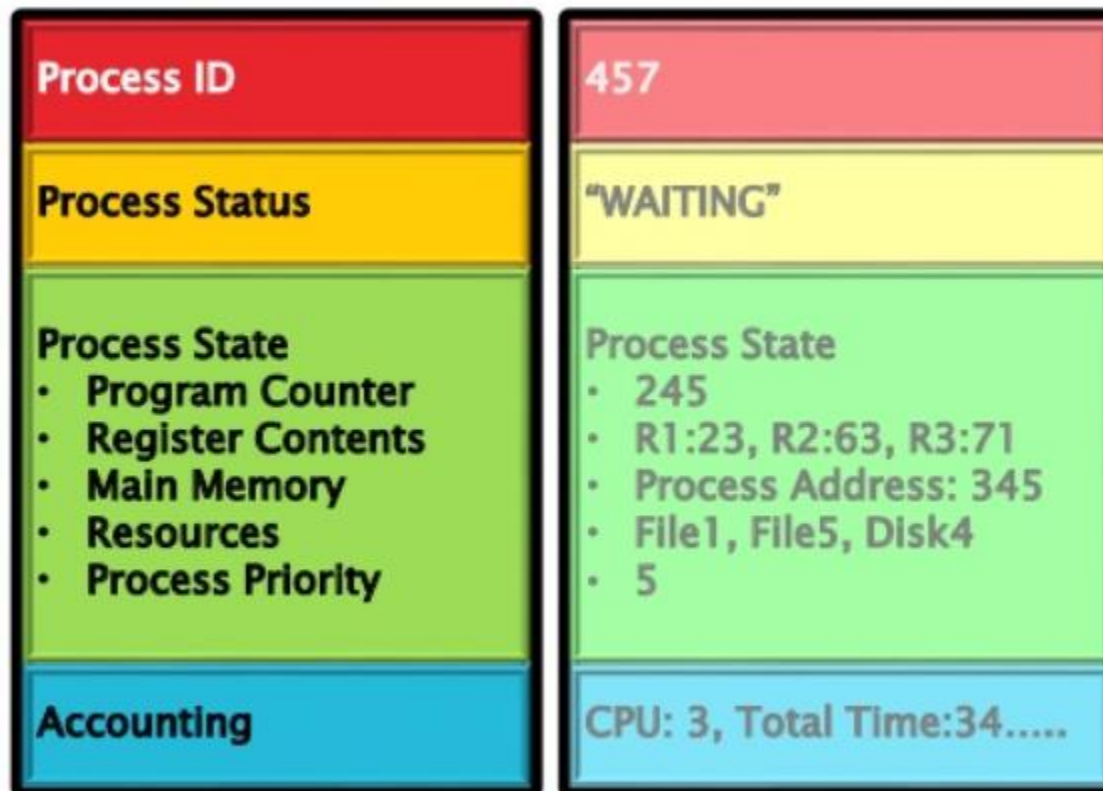
Process Control Block (PCB)

- Each process is presented in the system by a **process control block**
- PCB: an **information structure** allows **identify only 1** process



- Process state
- Process counter
- CPU's registers
- Information for process scheduling
- Information for memory management
- Information of usable resources
- Statistic information
- Pointer to another PCB
- ...

Process Control Block (PCB)

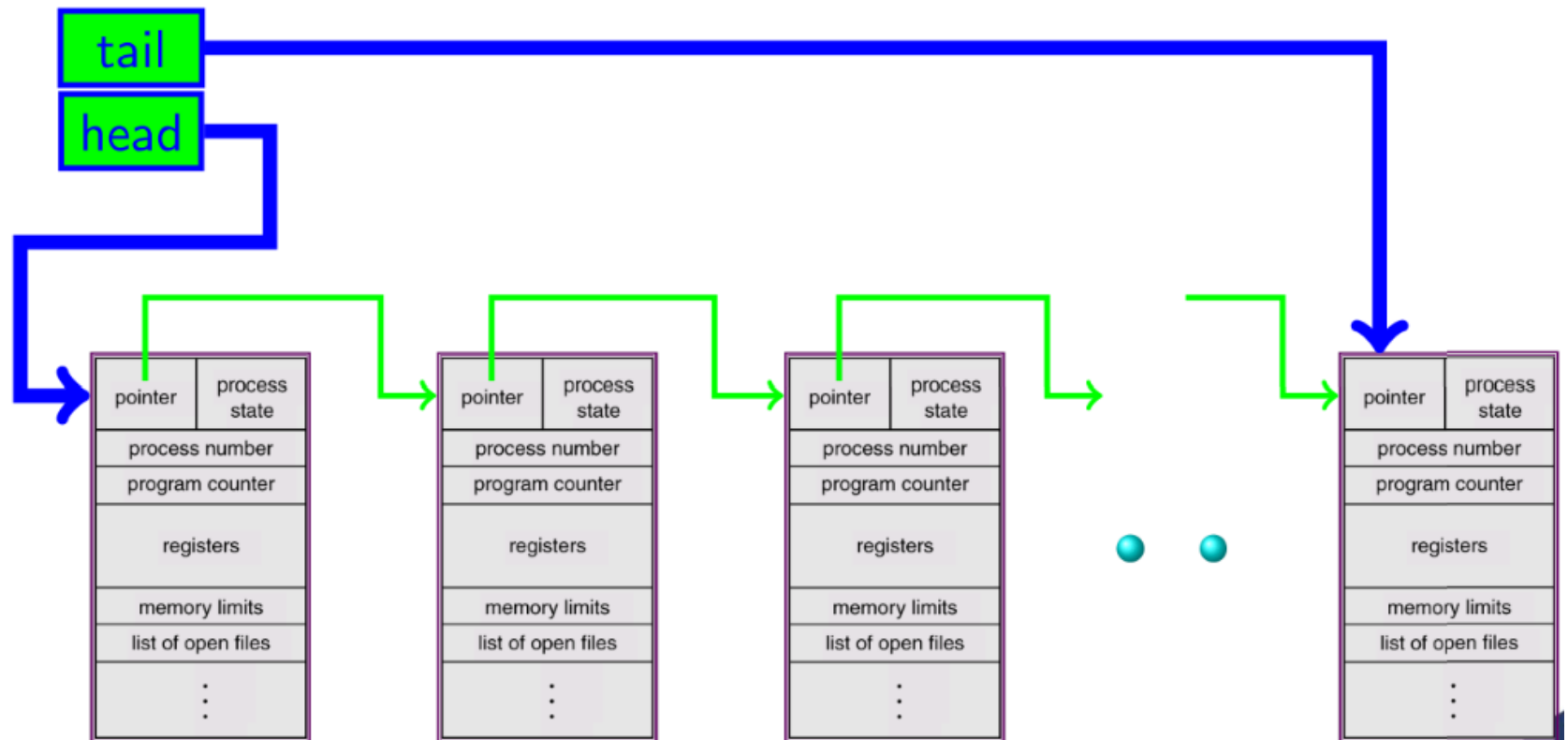


Chapter 2 Process Management

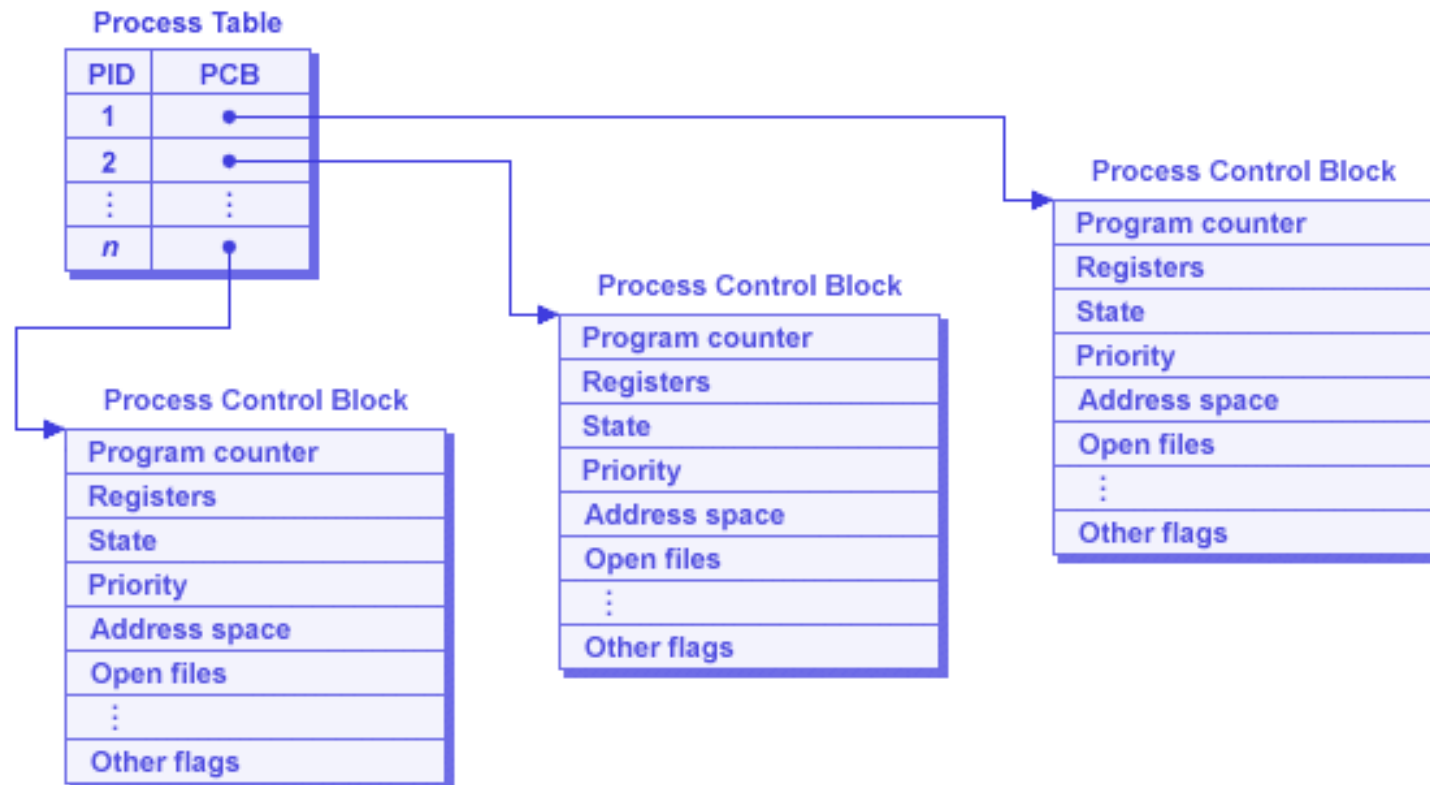
1. Process

2.1.1 Concept of process

Process List



Process Table (Linux)



Single-thread process and Multi-thread process

- **Single-thread process** : A **running process** with **only 1** executed **thread**
Have 1 thread of executive instruction
⇒ Allow executing **only 1 task** at a moment
- **Multi-thread process** : Process with **more than 1** executed thread
⇒ Allow more than 1 task to be done at a moment

Chapter 2 Process Management

1. Process

2.2.1 Process Scheduling

- Concept of process
- **Process Scheduling**
- Operations on process
- Process cooperation
- Inter-process communication

Introduction

Objective Maximize CPU's usage time
⇒ Need many processes in the system

Problem CPU switching between processes
⇒ Need a queue for processes

Single processor system

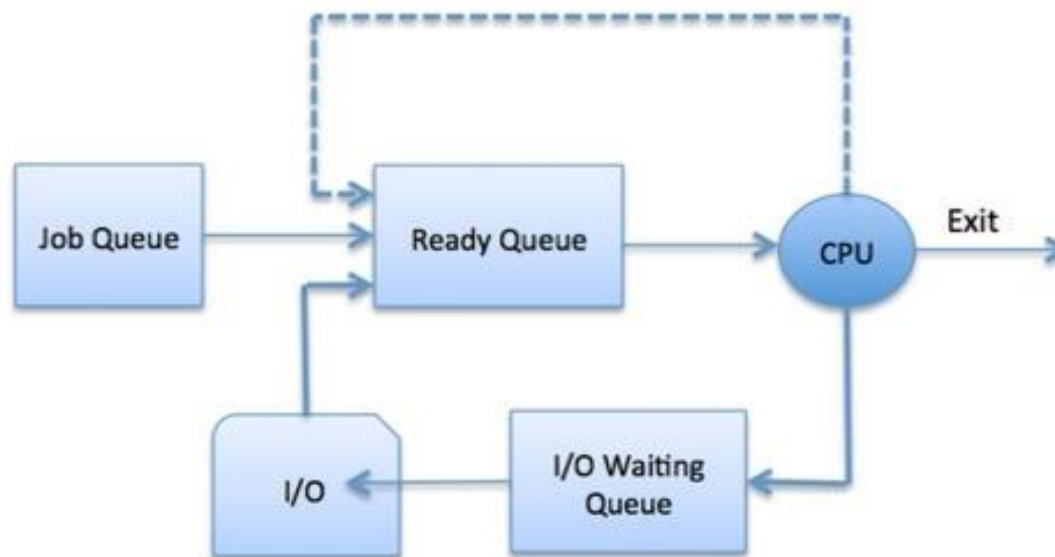
- ⇒ 1 process running
- ⇒ Other processes must wait until processor is free

Process queue I

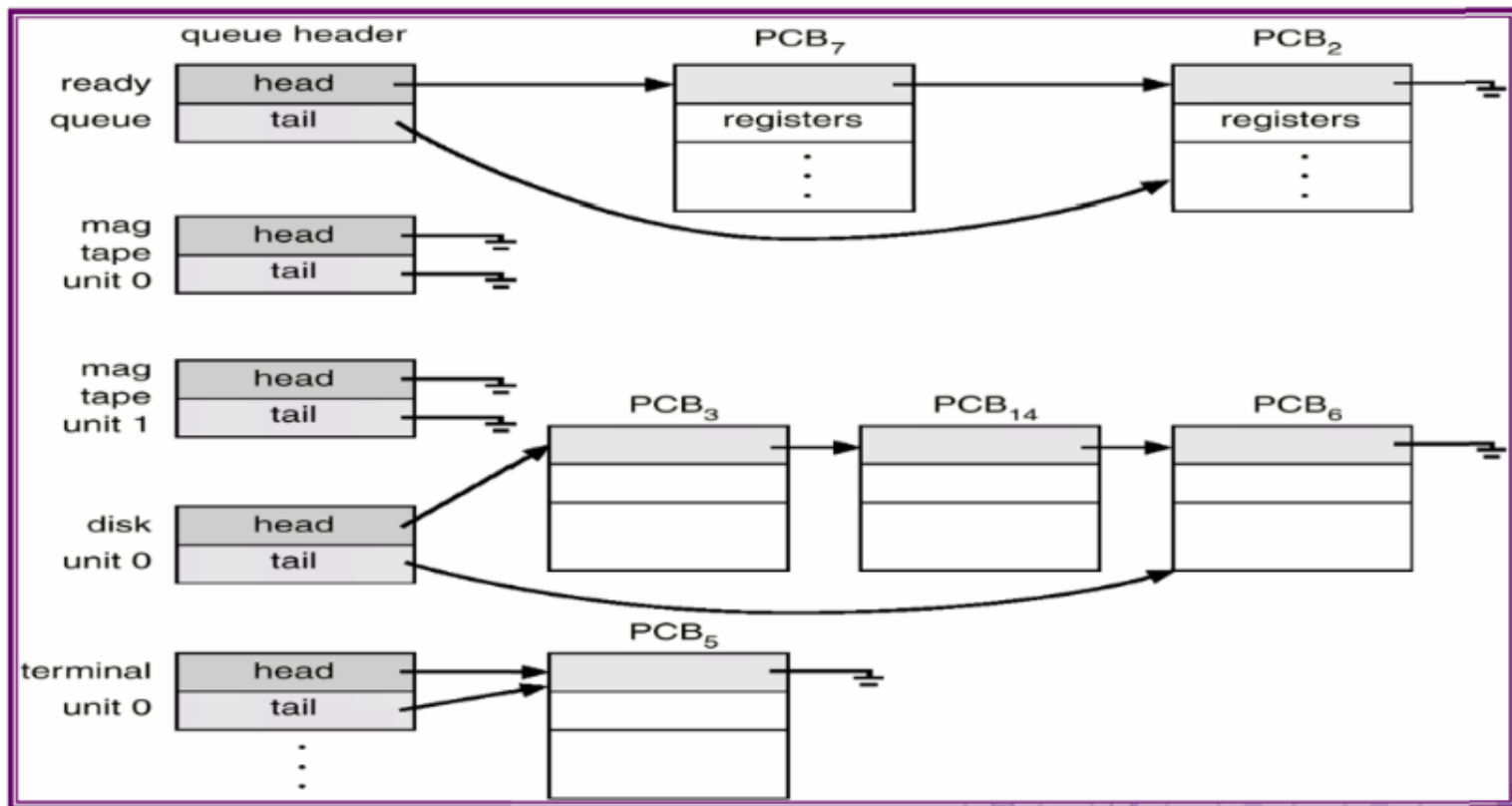
The system have **many queues** for processes

- **Job-queue** Set of processes in the system
- **Ready-Queue** Set of processes exist in the memory, ready to run
- **Device queues** Set of processes waiting for an I/O device.

Queues for each device are different

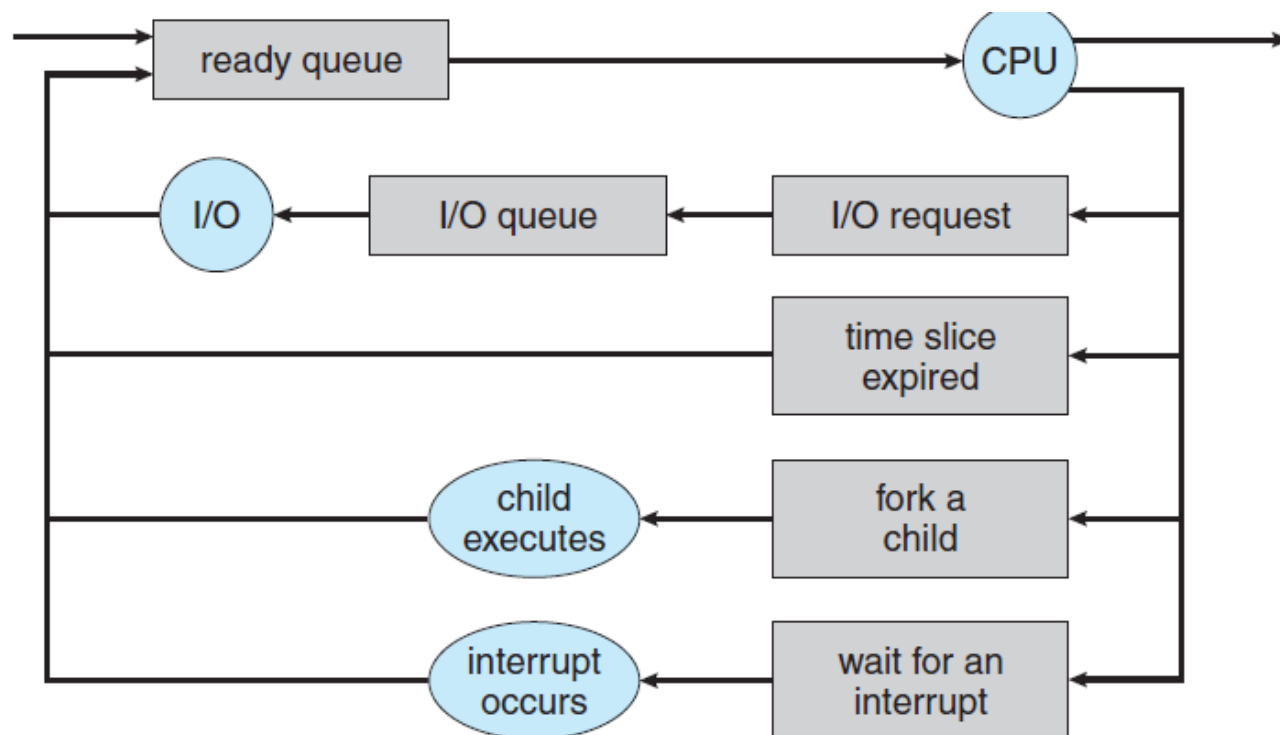


Process queue I



Process queue II

- * Process moves between different queues

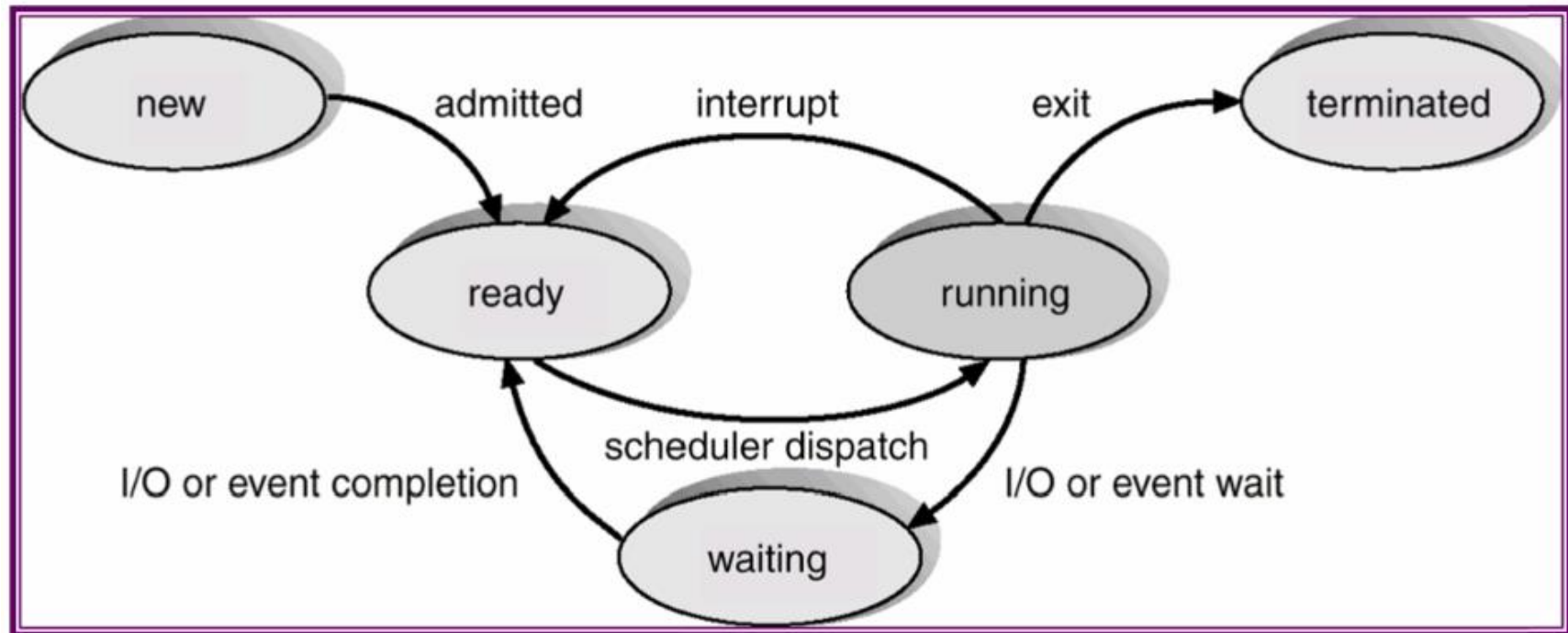


- * Newly created process is putted in **ready queue** and wait until it's selected to execute

Process queue III

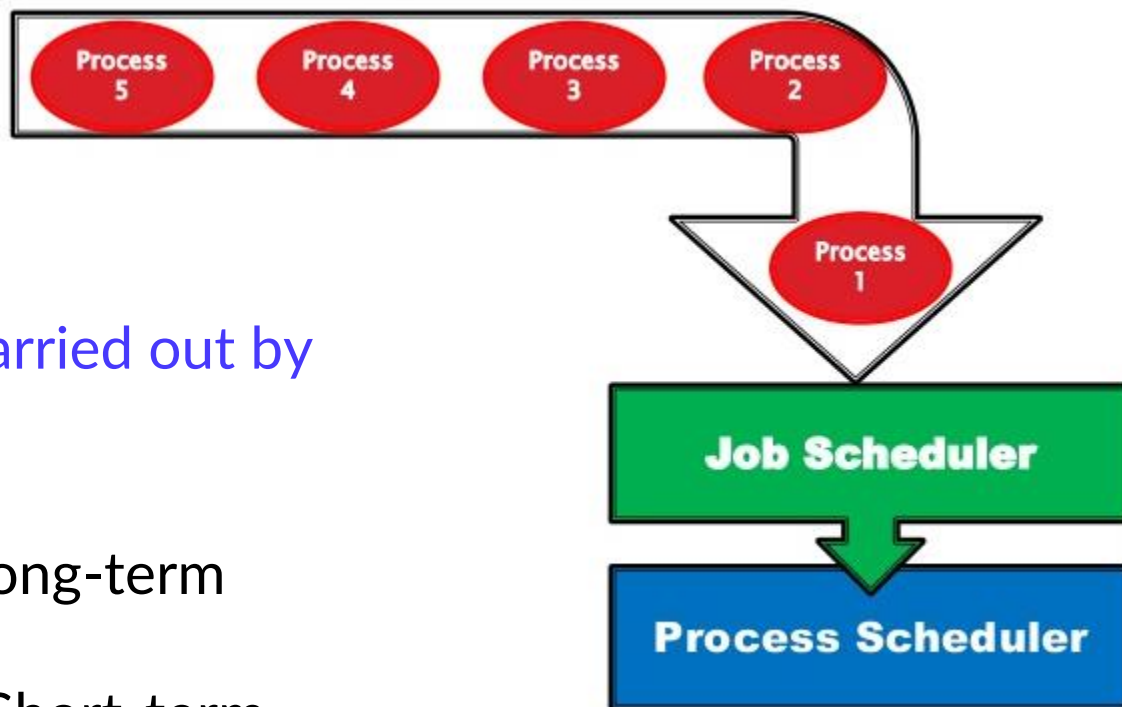
- Process **selected** and **running**
 - ① process issue an **I / O request**, and then be **placed** in an **I / O queue**
 - ② process **create** a **new sub-process** and **wait** for its termination
 - ③ process **removed forcibly** from the CPU, as a result of an **interrupt**, and be put back in the ready queue
- In case (1&2) after the waiting event is finished,
 - Process eventually **switches** from **waiting** → **ready** state
 - Process is then **put back** to **ready queue**
- Process **continues** this **cycle** (*ready, running, waiting*) **until** it **terminates**.
 - It's **removed** from all queues
 - its PCB and resources deallocated

Scheduler



- *Process selection is carried out by appropriate scheduler
 - * Job scheduler; Long-term scheduler
 - * CPU scheduler; Short-term scheduler

Scheduler



*process selection is carried out by appropriate scheduler

- Job scheduler; Long-term scheduler
- CPU scheduler; Short-term scheduler

Job Scheduler

- Select processes from program queue stored in disk and put into memory to execute
- Not frequently (seconds/minutes)
- Control the degree of the multi-programming (number of process in memory)
- When the degree of multi-programming is stable, the scheduler may need to be invoked when a process leaves the system

Job Scheduler

- Job selection's problem
 - Consider the following program

```
PROGRAM PrintValue:
```

```
BEGIN
```

```
Input A;
```

```
Input B;
```

IO

```
C = A + B;
```

```
D = A - B;
```

CPU

```
Print "The sum of inputs is: ", C;
```

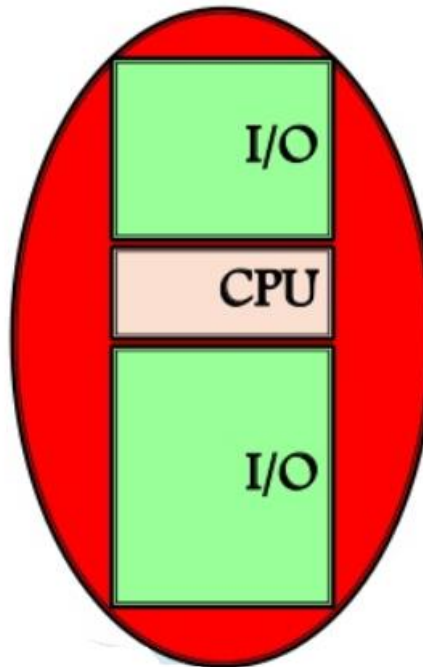
```
Print "The Difference of inputs is: ", D;
```

IO

```
END.
```

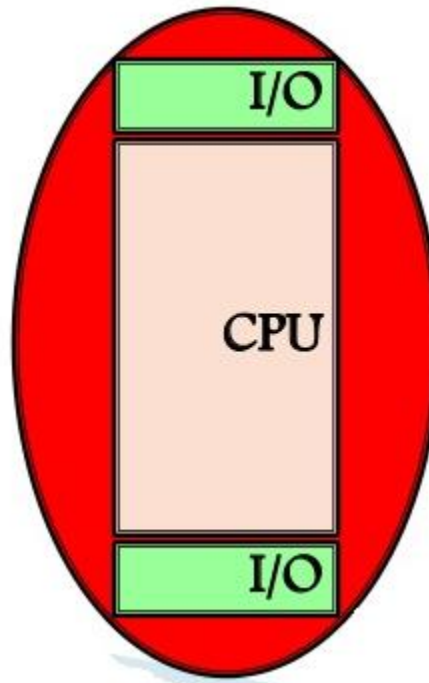
Job Scheduler

- I/O bound process: utilizes less CPU time
- Ex: Graphic programs



Job Scheduler

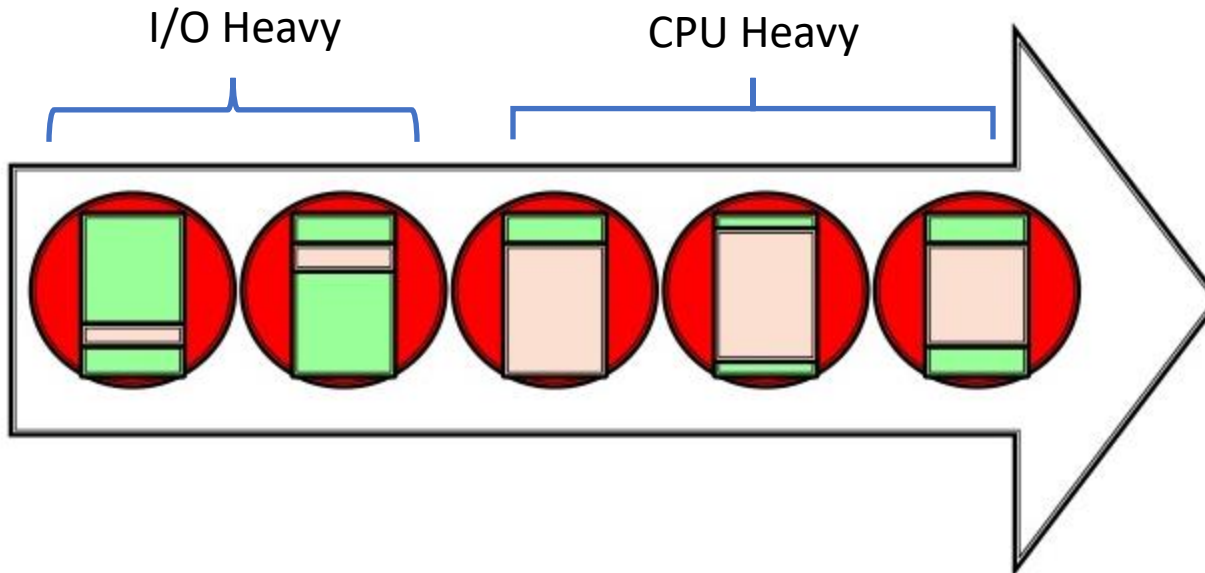
- CPU bound process: uses more CPU time
- Ex: math solving programs...



Job Scheduler

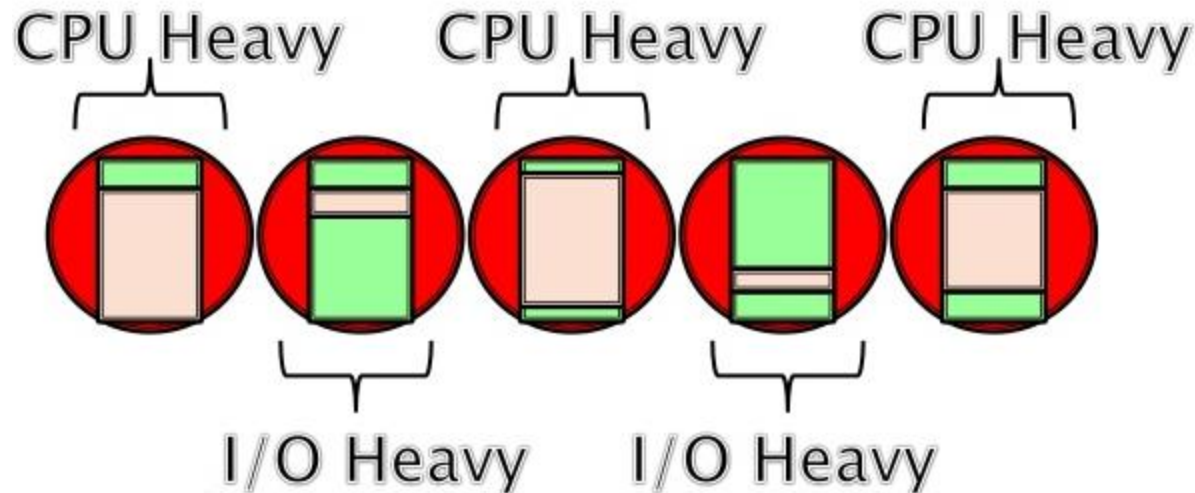
- Job selection's problem
 - Should select good process mix of both
 - ⇒ I/O bound : ready queue is empty, waste CPU
 - ⇒ CPU bound: device queue is empty, device is wasted

If input are the following jobs



Job Scheduler

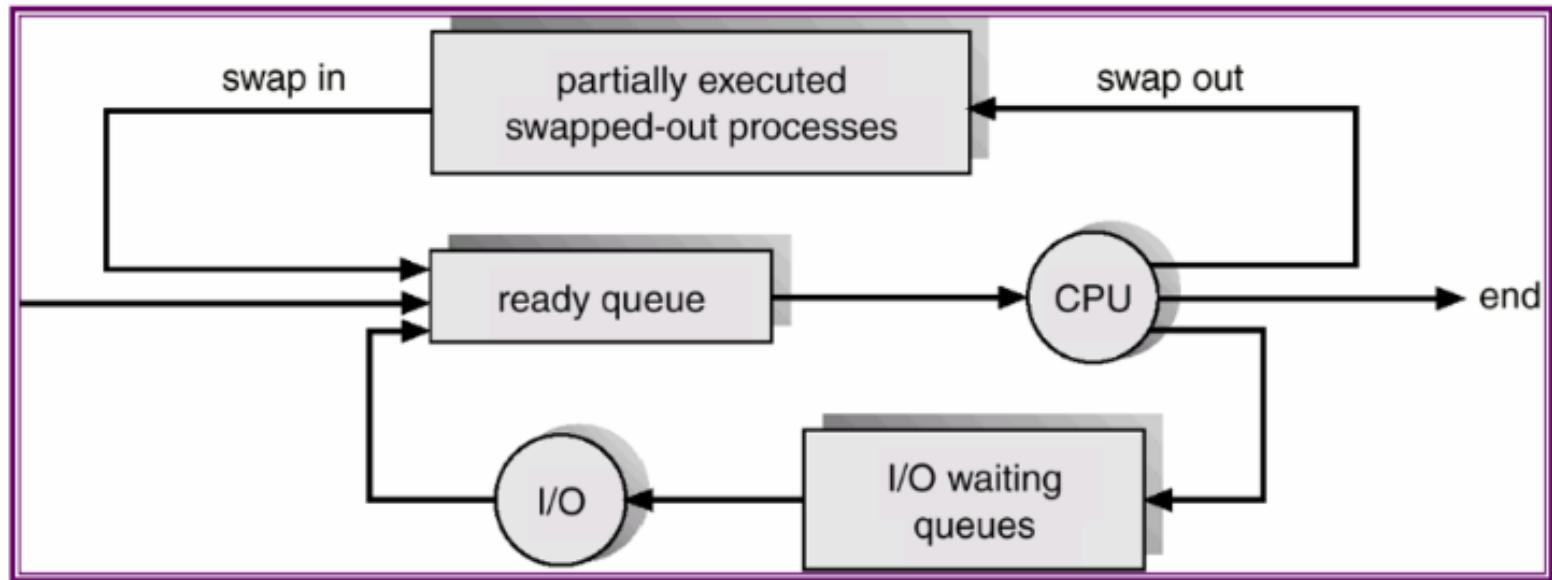
- Job scheduler will arrange the Jobs then send them to **Process Scheduler**



CPU Scheduler

- **Selects** from processes that are **ready** to execute, and allocates the CPU to 1 of them
- **Frequently** work (example: at least 100ms/once)
 - A process may execute for only a few milliseconds before waiting for an I/O request
 - Select new process that is ready
- the short-term **scheduler** must be **fast**
 - 10ms to decide $\Rightarrow 10/(110) = 9\%$ CPU time is wasted
- Process selection problem (CPU scheduler)

Process swapping (Medium-term scheduler)



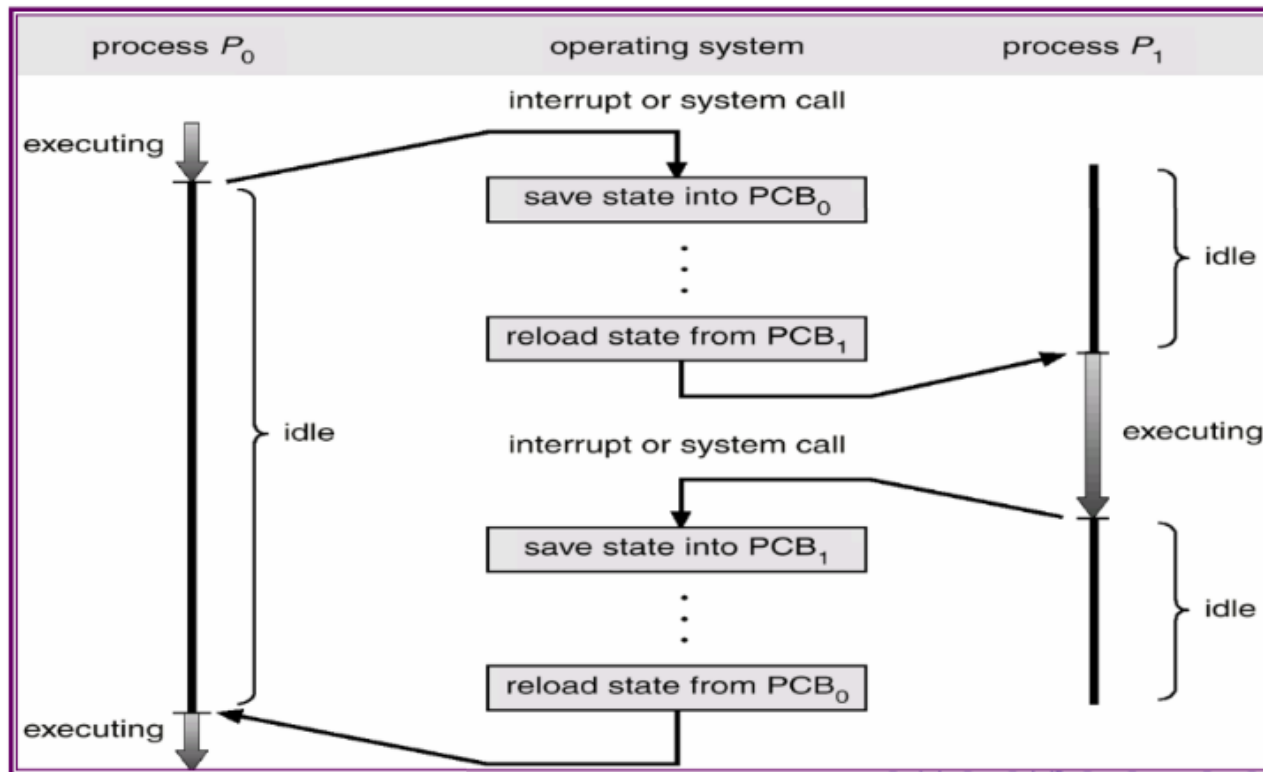
- Task
 - Removes processes from memory, reduces the degree of multiprogramming
 - Process can be brought back into memory and its execution can be continued where it left off
- Objective: Free memory, make a wider unused memory area

Context switch

- **Switch CPU from 1 process to another process**
 - **Save** the **state** of the **old** process and **load** the **saved state** for the **new** process
 - includes the value of the CPU's **registers**, the process state and **memory-management information**
 - **Take time** and **CPU cannot** do anything while switching
 - Depend on the **support** of the **hardware**
 - More **complex** the **OS**, the **more works** must be done during a **context switch**

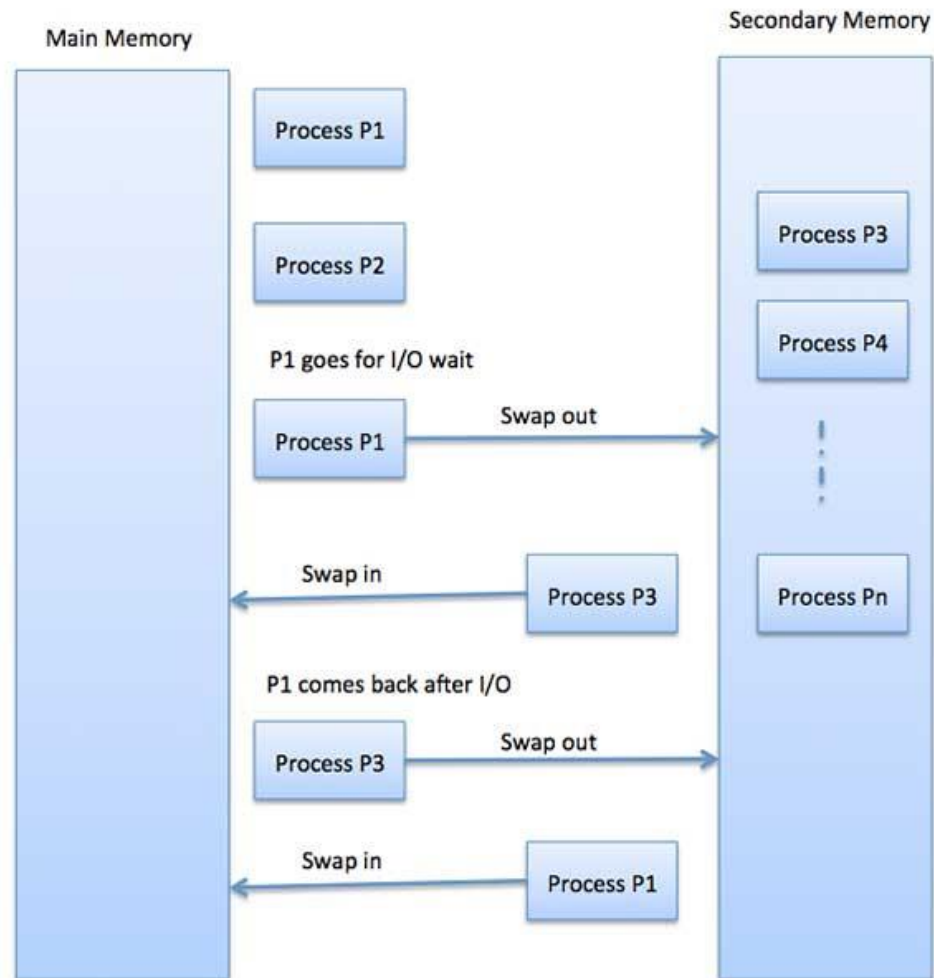
Context switch

- Occurs when an interrupt signal appear(timely interrupt) or when process issue a system call (to perform I/O)
- CPU switching diagram(Silberschatz 2002)

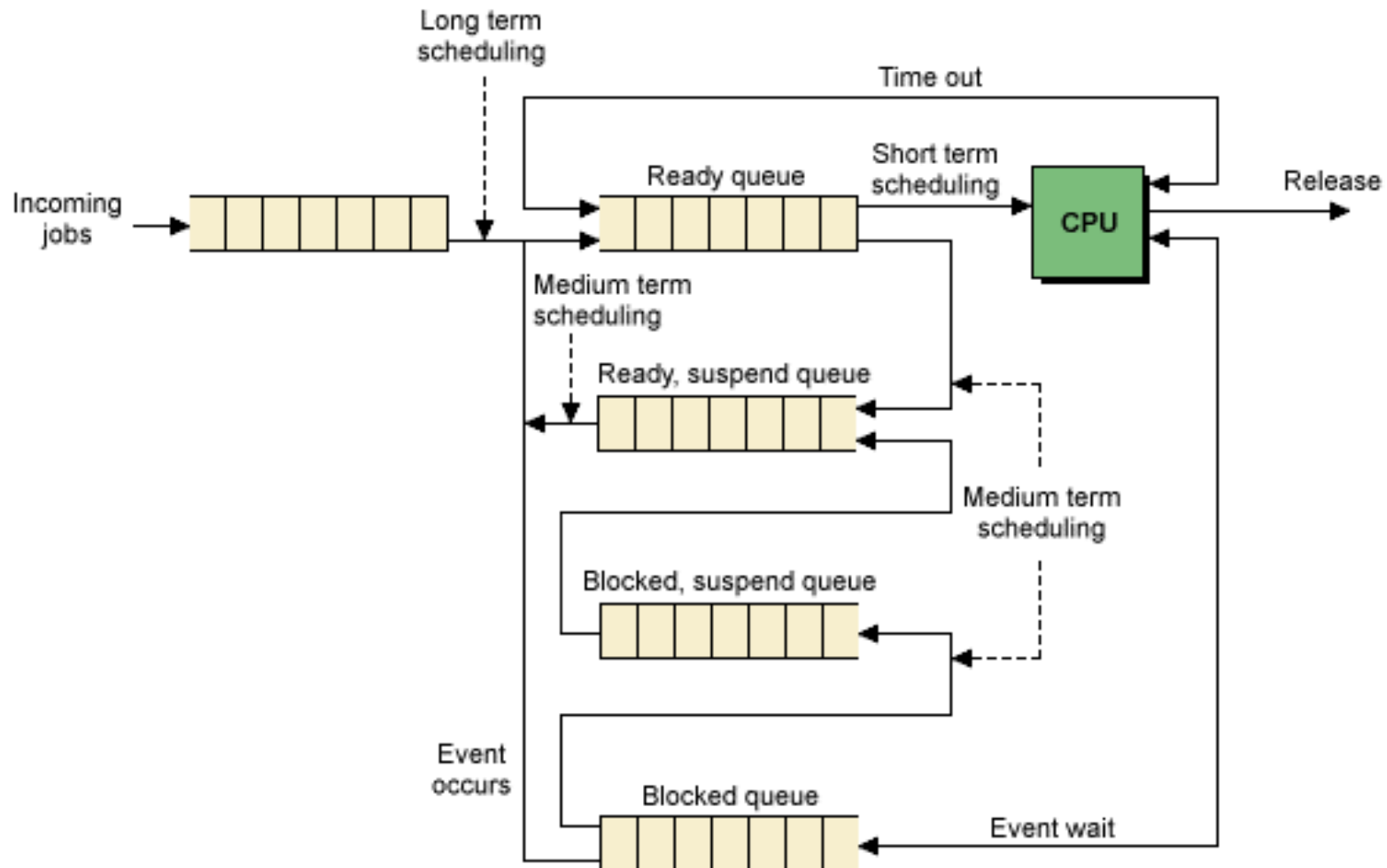


- CPU switch from this process to another process (switch the running process)

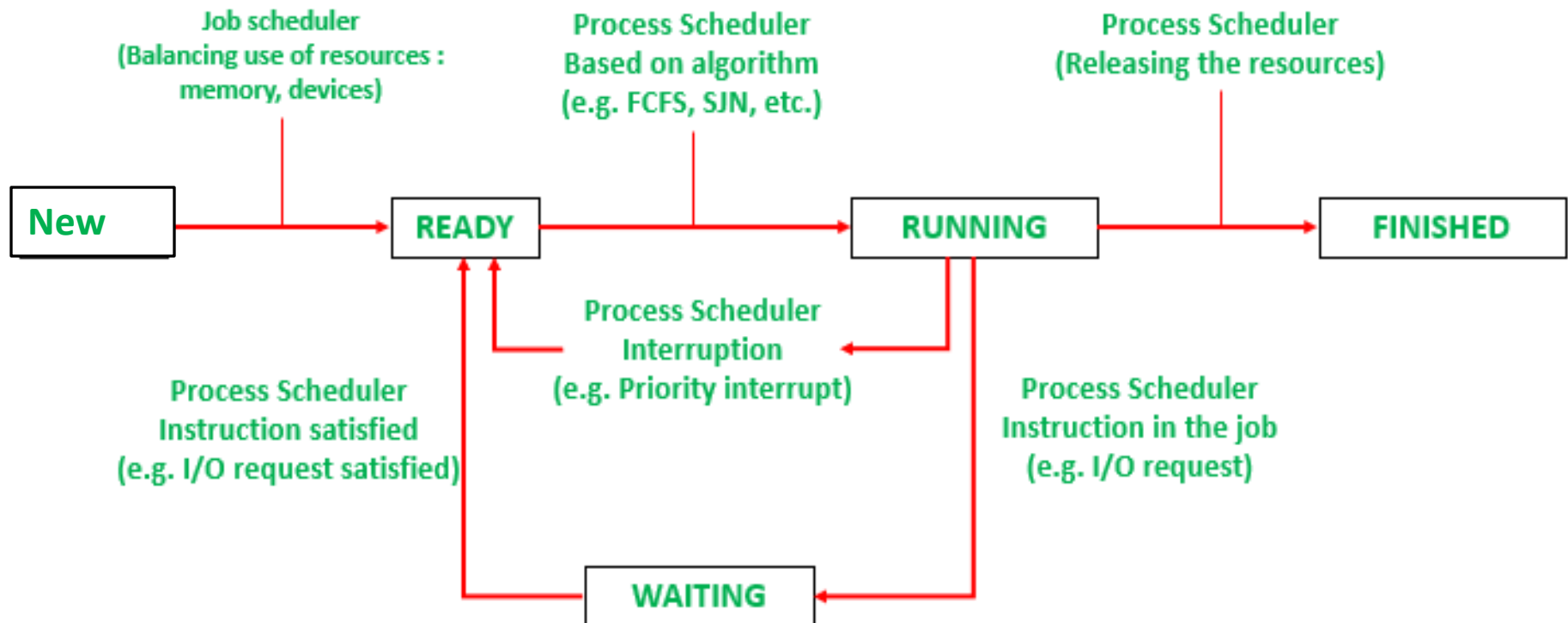
Process swapping (Medium-term scheduler)



Medium-term scheduler



Schedulers and process's states



Chapter 2 Process Management

1. Process

2.1 Operations on process

- Concept of process
- Process Scheduling
- Operations on process
- Process cooperation
- Inter-process communication

Operations on process

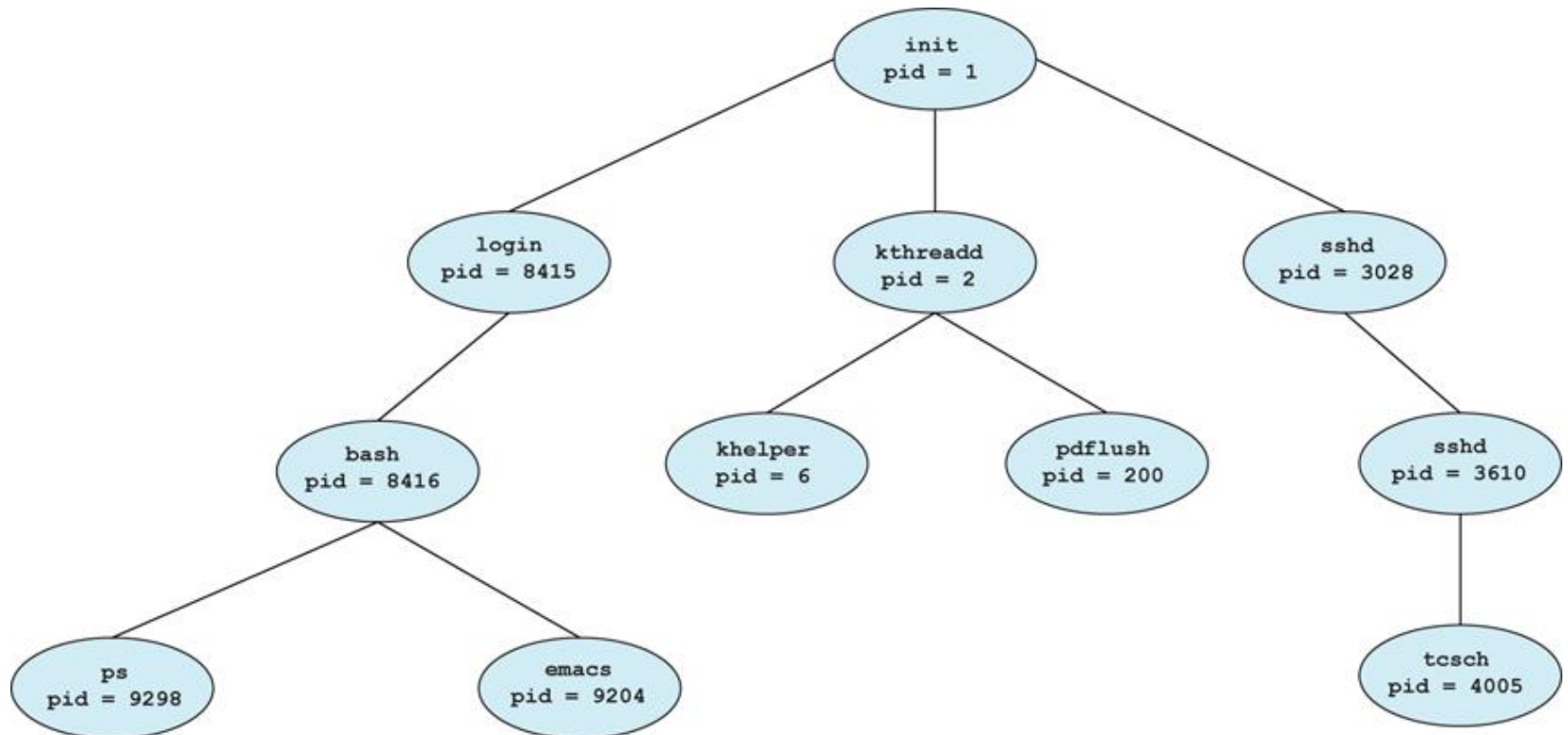
- Process Creation
- Process Termination

Process Creation

- Process may create several new processes (CreateProcess(), fork())
 - The creating process: **parent-process**
 - The new process: **child-process**
- **Child-process** may **create new process** ⇒ **Tree of process**

Process Creation

- Example: A process tree in Solaris OS



Example Process Creation in Linux

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    /* fork a child process */
    pid = fork();
    if (pid < 0) { /* error occurred
    */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child
    process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the
        child to complete */
        wait(NULL);
        printf("Child Complete");
    }
    return 0;
}
```



Process Creation

- Resource allocation
 - Child receives resource from
 - OS
 - parent-process
 - All of the resource
 - a subset of the resources of the parent process (to prevent a process from creating too many children)
- When a process creates a new process, two possibilities exist in terms of execution:
 - The parent continues to execute concurrently with its children
 - The parent waits until some or all of its children have terminated

Process Termination

- Finishes executing its final statement and asks the OS to delete (exit)
 - Return data to parent process
 - Allocated resources are returned to the system
- Parent process may terminate the execution of child process
 - Parent must know child process's id $\Rightarrow$ child process has to send its id to parent process when created
 - Use the system call (abort)
- Parent process terminate child process when
 - The child has exceeded its usage of some of the resources
 - The task assigned to the child is no longer required
 - The parent is exiting, and the OS does not allow a child to continue
 - $\Rightarrow$ Cascading termination. Example: *turn off computer*

Some functions with process in WIN32 API

- `CreateProcess(...)`
 - `LPCTSTR` Name of the execution program
 - `LPTSTR` Command line parameter
 - `LPSECURITY_ATTRIBUTES` A pointer to process **security attribute** structure
 - `LPSECURITY_ATTRIBUTES` A pointer to thread **security attribute** structure
 - `BOOL` allow process inherit devices (TRUE/FALSE)
 - `DWORD` process creation flag (Example: `CREATE_NEW_CONSOLE`)
 - `LPVOID` Pointer to environment block
 - `LPCTSTR` full path to program
 - `LPSTARTUPINFO` info structure for new process
 - `LPPROCESS_INFORMATION` Information of new process
- `TerminateProcess(HANDLE hProcess, UINT uExitCode)`
 - *`hProcess`* A handle to the process to be terminated
 - *`uExitCode`* process termination code
- `WaitForSingleObject(HANDLE hHandle, DWORD dwMs)`
 - *`hHandle`* Object handle
 - *`dwMs`* Waiting time (`INFINITE`)



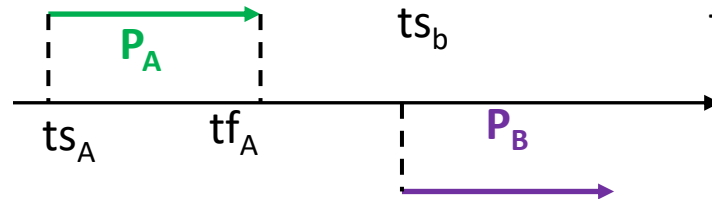
Some functions with process in WIN32 API(Example)

```
#include <stdio.h>
#include <windows.h>
int main(VOID){
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    if (!CreateProcess(NULL, /"C:\\\\WINDOWS\\system32\\mspaint.exe", NULL,
        NULL, FALSE, 0, NULL, NULL, &si,&pi)) {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

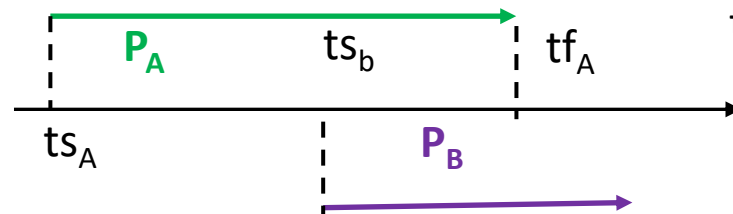
- Concept of process
- Process Scheduling
- Operations on process
- **Process cooperation**
- Inter-process communication

Processes' relation classification

- Sequential processes



- Parallel processes



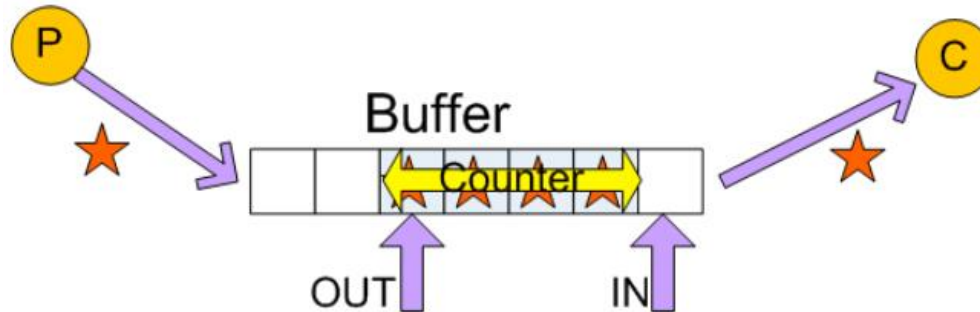
- **Independence:** Not affect or affected by other running process in the system
- **Cooperation:** affect or affected by other running process in the system

Processes' relation classification

- Cooperation in order to
 - Share Information
 - Speedup Computation
 - Provide Modularity
 - Increase the convenience
- Process collaborate requires mechanism that allow process to
 - Communicate with each other
 - Synchronize their actions

Producer - Consumer Problem

I



- The system includes 2 processes
 - Producer creates product
 - Consumer consumes created product
- Application
 - Printing program (producer) creates characters that are consumed by printer driver (consumer)
 - Compiler (producer) creates assembly code, Assembler (consumer/producer) consumes assembly code and generate object module which is consumed by the exec/loader (consumer)

Producer - Consumer Problem

II

- *Producer* and *Consumer* work simultaneously
Use a sharing **Buffer** which store product filled in by Producer and taken out by Consumer
 - IN Next empty place in buffer
 - OUT First filled place in the buffer.
 - Counter Number of products in the buffer
- **Producer** and **Consumer** must be synchronized
 - Consumer does not try to consume a product that was not created
- Unlimited size buffer
 - Buffer is empty -> Consumer need to wait
 - Producer can put product into buffer without waiting
- Limited size buffer
 - Buffer is empty -> *Consumer* need to wait
 - *Producer* need to wait if the Buffer is full

Producer - Consumer Problem

II

Producer

```
while(1) {  
    /*produce an item in nextProduced*/  
    while (Counter == BUFFER_SIZE) ; /*do nothing*/  
    Buffer[IN] = nextProduced;  
    IN = (IN + 1) % BUFFER_SIZE;  
    Counter++;  
}
```

Consumer

```
while(1){  
    while(Counter == 0) ; /*do nothing*/  
    nextConsumed = Buffer[OUT];  
    OUT =(OUT + 1) % BUFFER_SIZE;  
    Counter--; /*consume the item in nextConsumed*/  
}
```

2. Inter-process communication

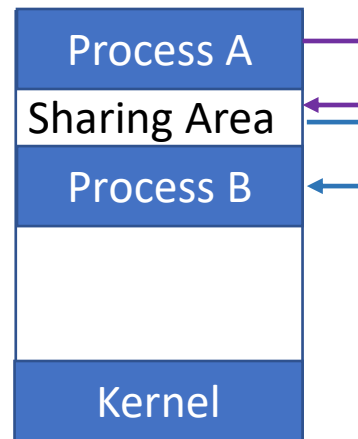
- Concept of process
- Process Scheduling
- Operations on process
- Process cooperation
- **Inter-process communication**

Inter-Process Communication



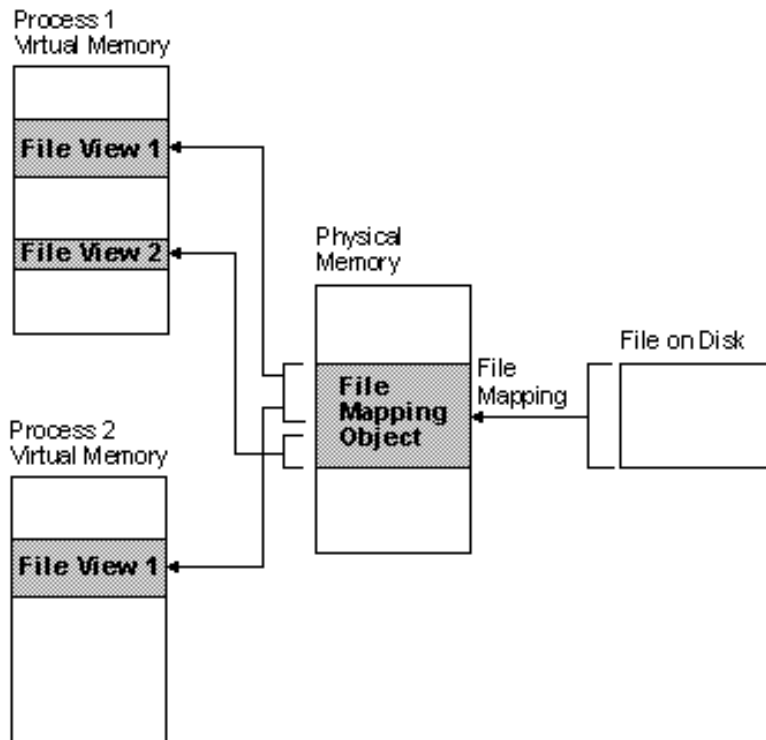
Communicate between process

- Memory sharing model
 - Process share a **common memory area**
 - Implementation codes are written explicitly by application **programmer**
 - Example: Producer-Consumer problem



Memory sharing model

Example file mapping in Windows



Chapter 2 Process Management

1. Process

Ex: File mapping- Process 1

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
#include <tchar.h>
#define BUF_SIZE 256
TCHAR szName[]=TEXT("Global\\MyFileMappingObject");
TCHAR szMsg[]=TEXT("Message from first process.");
int _tmain()
{
    HANDLE hMapFile;
    LPCTSTR pBuf;
    hMapFile = CreateFileMapping(
        INVALID_HANDLE_VALUE, // use paging file
        NULL,                 // default security
        PAGE_READWRITE,       // read/write access
        0,                     // maximum object size (high-order
DWORD)
        BUF_SIZE,             // maximum object size (low-order
DWORD)
        szName);              // name of mapping object
    if (hMapFile == NULL)
    {
        _tprintf(TEXT("Could not create file mapping object (%d).\n"),
            GetLastError());
        return 1;
    }

    pBuf = (LPTSTR) MapViewOfFile(hMapFile,
        // handle to map object
        FILE_MAP_ALL_ACCESS,
        // read/write permission
        0,
        0,
        BUF_SIZE);

    if (pBuf == NULL)
    {
        _tprintf(TEXT("Could not map view of file
(%d).\n"),
            GetLastError());

        CloseHandle(hMapFile);

        return 1;
    }

    CopyMemory((PVOID)pBuf, szMsg,
        (_tcslen(szMsg) * sizeof(TCHAR)));
    _getch();

    UnmapViewOfFile(pBuf);

    CloseHandle(hMapFile);

    return 0;
}
```

Chapter 2 Process Management

1. Process

Ex: File mapping- Process 2

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
#include <tchar.h>
#pragma comment(lib, "user32.lib")

#define BUF_SIZE 256
TCHAR szName[]=TEXT("Global\\MyFileMappingObject");

int _tmain()
{
    HANDLE hMapFile;
    LPCTSTR pBuf;

    hMapFile = OpenFileMapping(
        FILE_MAP_ALL_ACCESS, // read/write access
        FALSE,                // do not inherit the name
        szName);              // name of mapping object

    if (hMapFile == NULL)
    {
        tprintf(TEXT("Could not open file mapping object
(%d).\\n"),
            GetLastError());
        return 1;
    }
}
```

```
pBuf = (LPTSTR) MapViewOfFile(hMapFile,
    // handle to map object
    FILE_MAP_ALL_ACCESS,
    // read/write permission
    0,
    0,
    BUF_SIZE);

if (pBuf == NULL)
{
    tprintf(TEXT("Could not map view of
file (%d).\\n"),
        GetLastError());

    CloseHandle(hMapFile);

    return 1;
}

MessageBox(NULL, pBuf,
TEXT("Process2"), MB_OK);

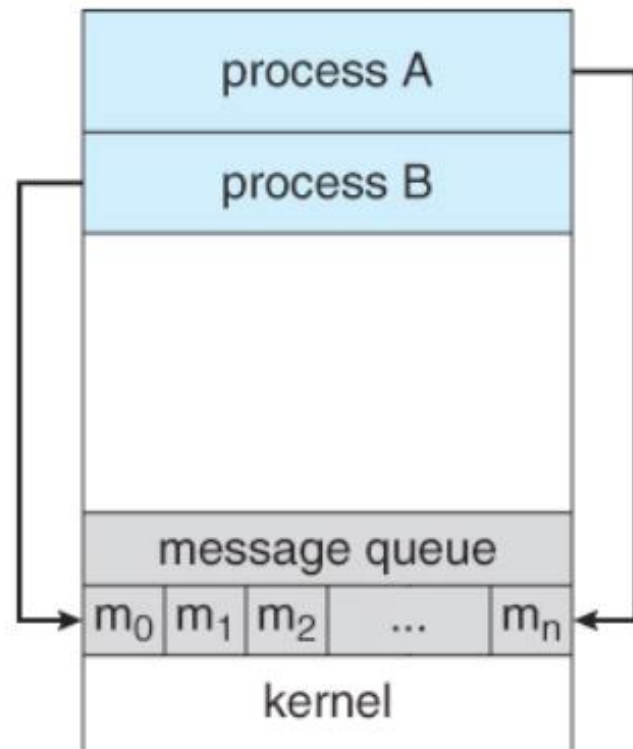
UnmapViewOfFile(pBuf);

CloseHandle(hMapFile);

return 0;
}
```

Communicate between process

- Inter-process communication model
 - A mechanism allow processes to **communicate** and **synchronize**
 - Often used in distributed system where **processes** lie on **different computers** (chat)
 - Guaranteed by **message passing system**



Message passing system

- Allow processes to communicate without sharing variable
- Require 2 basic operations
 - Send (msg) msg has **fixed** or **variable** size
 - Receive (msg)
- If 2 processes **P** and **Q** want to **communicate**, they need to
 - Establish a **communication link** (physical/logical) between them
 - Exchange messages via operations: send/receive

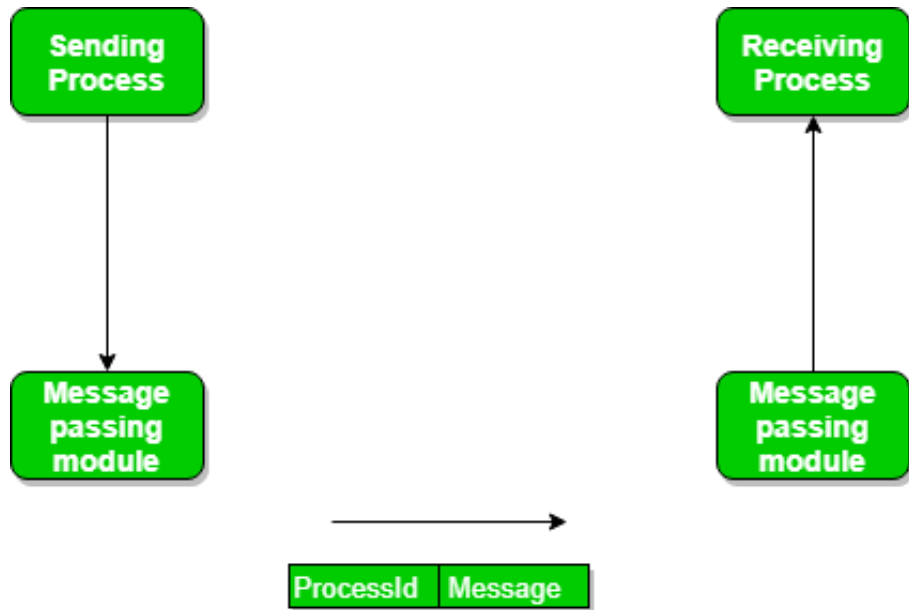
Direct Communication

- Each process that wants to communicate must **explicitly name** the **recipient** or **sender** of the communication
 - send (P, message) – Send a message to process P
 - receive(Q, message) – Receive a message from process Q

```
void producer(void)
{
    while (TRUE) {
        ...
        produce item;
        ...
        send( consumer, item );
    }
}
```

```
void consumer(void)
{
    while (TRUE) {
        recv( producer, item );
        ...
        consume item;
        ...
    }
}
```

Direct Communication



- Properties of a communication link
 - A link is established automatically
 - A link is associated with exactly 2 processes
 - Exactly 1 link exists between each pair of processes
 - Link can be 1 direction but often 2 directions

Indirect Communication

- Messages are sent to and received from **mailboxes**, or **ports**
- Each mailbox has a **unique identification**
 - 2 processes can communicate only if they share a mailbox
- Communication link's properties
 - Established **only if** both **members** of the pair have a **shared mailbox**
 - A link may be associated with **more than 2** processes
 - **Each pair** of communicating processes may have **many links**
 - **Each link** corresponding to **1 mailbox**
 - A link can be either 1 or 2 directions



Indirect Communication

- Operations:
 - Create a new mailbox
 - Send and receive messages through the mailbox
 - Send(A, msg): send a msg to mailbox A
 - Receive(A, msg): receive a msg from mailbox A
 - Delete a mailbox

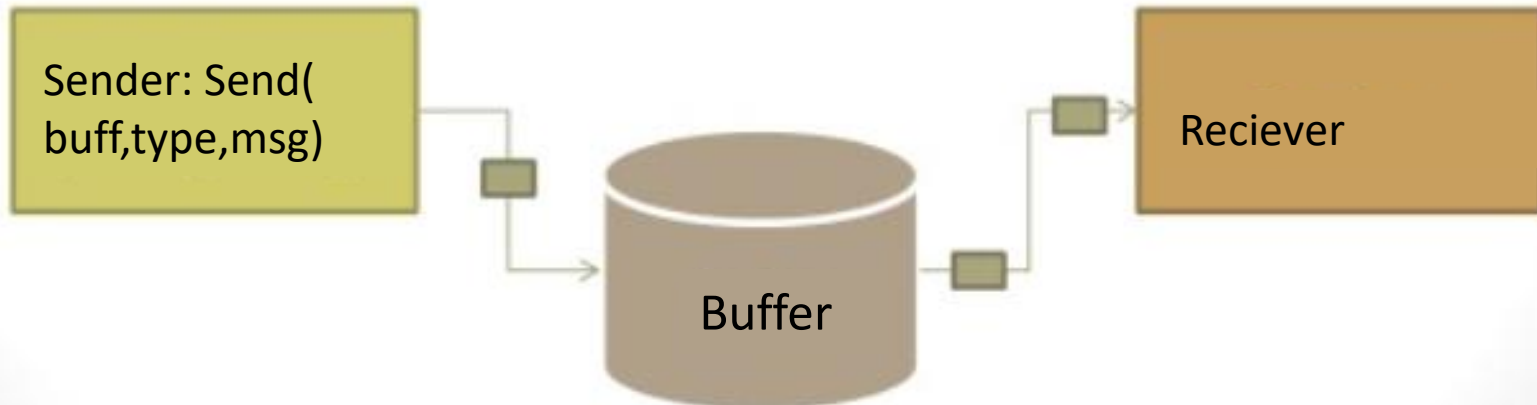


Synchronization

- Message passing may be either blocking or nonblocking
 - Blocking synchronous communication
 - Non-blocking asynchronous communication
- send() and receive() can be blocking or non-blocking
 - Blocking send: sending process is blocked until the message is received by the receiving process or by the mailbox
 - Non-blocking send: The sending process sends the message and resumes operation
 - Blocking receive: receiver blocks until a message is available
 - Non-blocking receive: receiver retrieves either a valid message or a null

Message Buffering

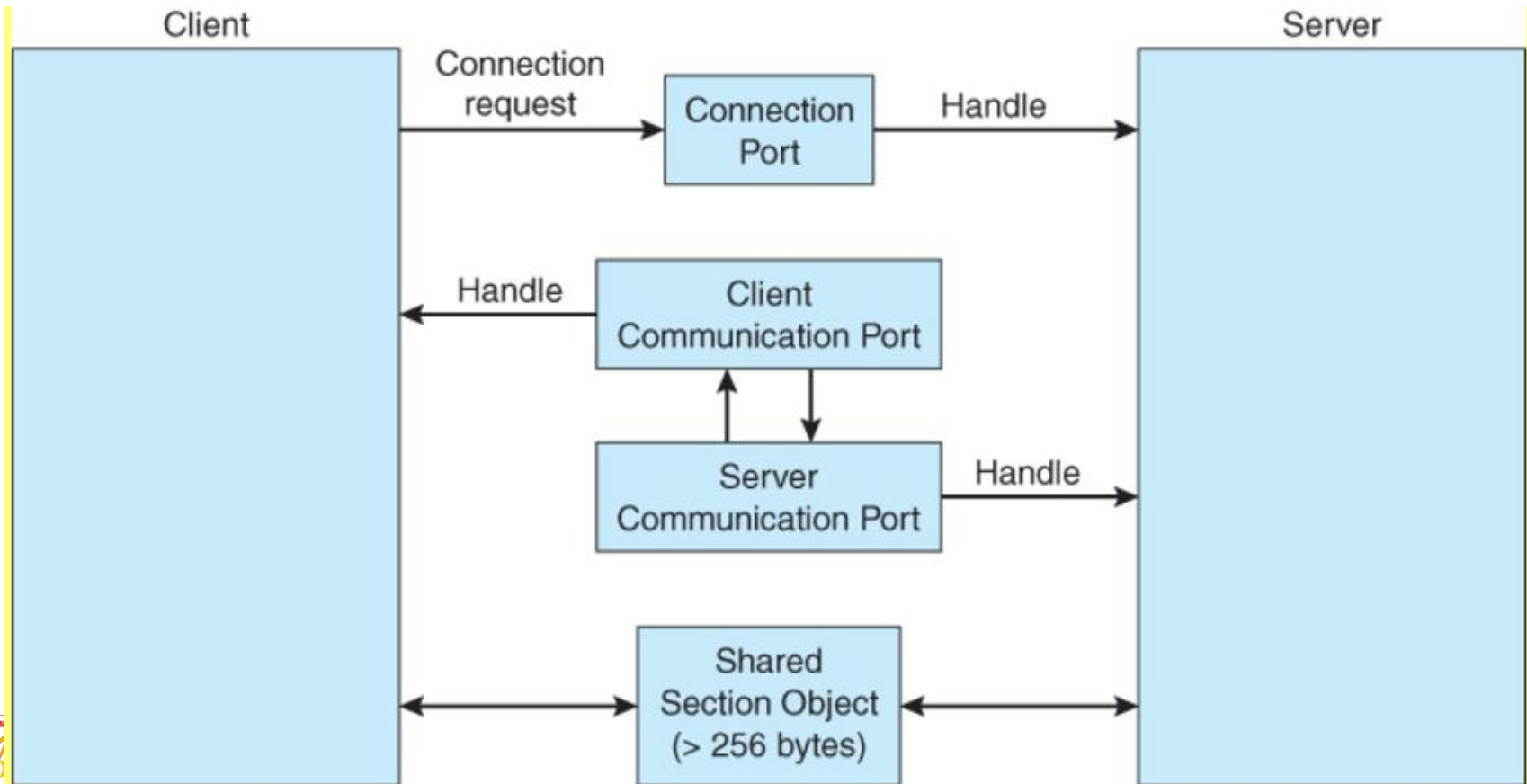
- Messages exchanged by communicating processes reside in a temporary **queue**



Buffering

- A queue can be implemented in 3 ways
 - Zero capacity:
 - maximum length 0 $\Rightarrow$ link cannot have any messages waiting in it
 - sender must block until the recipient receives the message
 - Bounded capacity
 - Queue has finite length $n \Rightarrow$ store at most n messages
 - If the queue is not full, message is placed in the queue and the sender can continue execution without waiting
 - If link is full, sender must block until space is available in the queue
 - Unbound capacity
 - The sender never blocks

Windows XP Message Passing



Communication in Client - Server Systems with Sockets

- Socket is defined as an endpoint for communication,
 - Each process has 1 socket
- Made up of an IP address and a port number.
- E.g.: 161.25.19.8:1625
 - IP Address: Computer address in the network
 - Port: identifies a specific process
- Types of sockets
 - Stream Socket: Based on TCP/IP protocol → Reliable data transfer
 - Datagram Socket: based on UDP/IP protocol → Unreliable data transfer
- Win32 API: Winsock
 - Windows Sockets Application Programming Interface

Winsock API 32 functions

`socket()` Create socket for data communication

`bind()` Assign an identifier for created socket (assign with a port)

`listen()` Listen to one connection

`accept()` Accept connection

`connect()` Connect to the server.

`send()` Send data with stream socket.

`sendto()` Send data with datagram socket.

`receive()` Receive data with stream socket.

`recvfrom()` Receive data with datagram socket.

`closesocket()` End an existed socket.

.....



Homework

Use Winsock to make a Client-Server program

Chat program.

.....

- ① Process
- ② Thread
- ③ CPU scheduling
- ④ Critical resource and process synchronization
- ⑤ Deadlock and solutions

- Introduction
- Multithreading model
- Thread implementation with Windows
- Multithreading problem

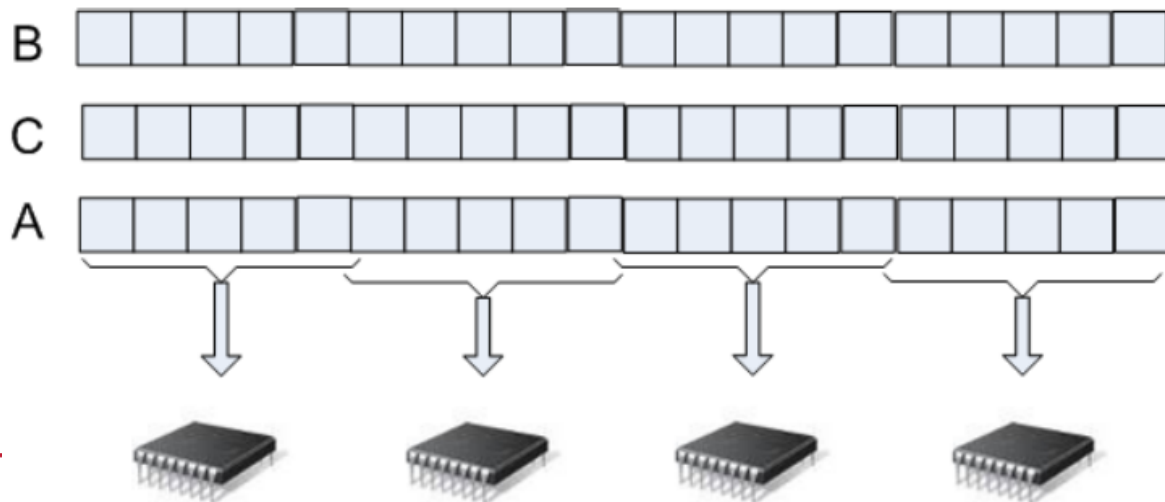
- Large size vector computing

Example: Vector computation

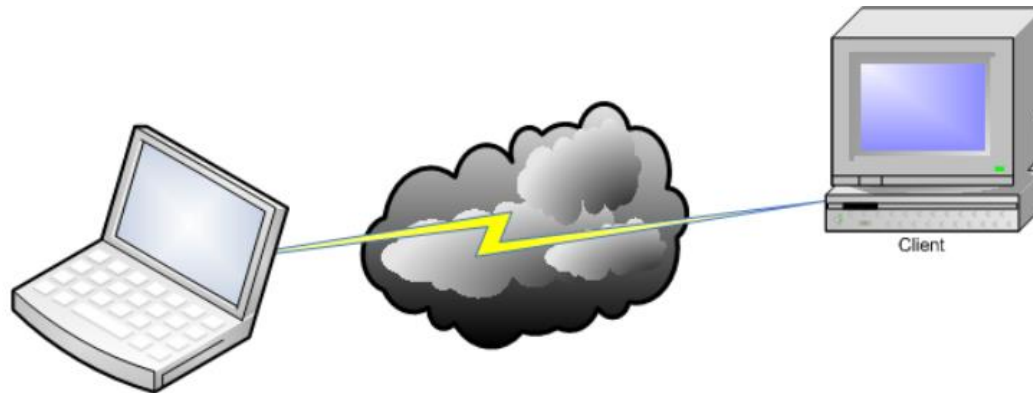
$a[k] = b[k] \cdot c[k];$

}

With multi processors system



Example: Chat



Process P

```
While (1) {  
    ReadLine(Msg);  
    Send(Q,Msg);  
    Receive(Q,Msg);  
    PrintLine(Msg);  
}
```

Msg receiving problem

- Blocking Recieve
- Non-blocking Receive

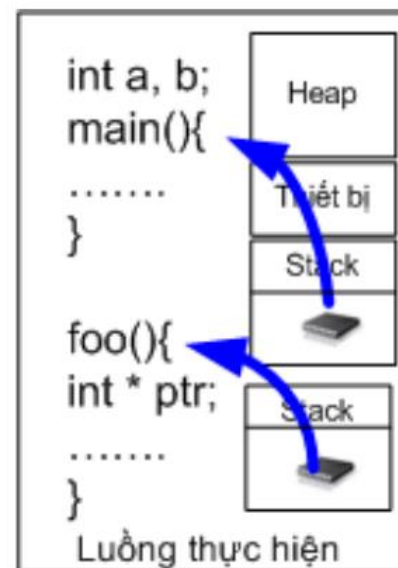
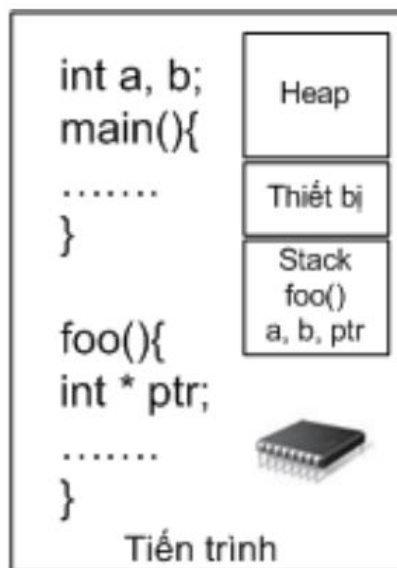
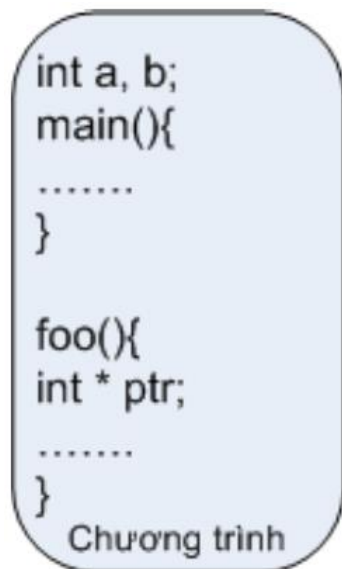
Solution

Concurrently perform
Receive & Send

Process Q

```
While (1) {  
    Receive(P,Msg);  
    PrintLine(Msg);  
    ReadLine(Msg);  
    Send(P,Msg);  
}
```

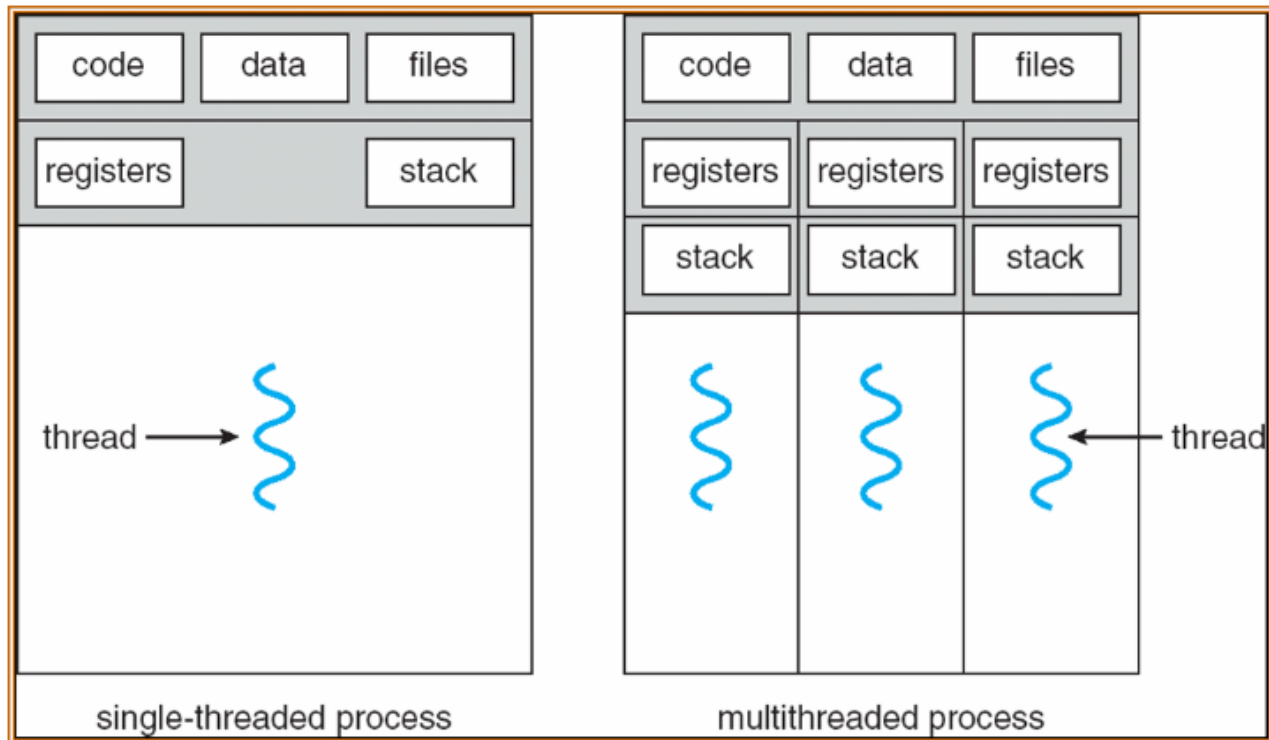
Program - Process - Thread



- Program: Sequence of instructions, variables,...
- Process: Running program: Stack, devices, processor,...
- Thread: A running program in process context
 - Multi-processor → Multi threads, each thread runs on 1 processor
 - Different in term of registers' values, stack's content

Single-threaded and multi-threaded process

- **Traditional** OS (MS-DOS, UNIX)
 - Process has 1 controlling thread (heavyweight process)
- **Modern** OS (Windows, Linux)
 - Process may have many threads
 - Perform many tasks at a single time

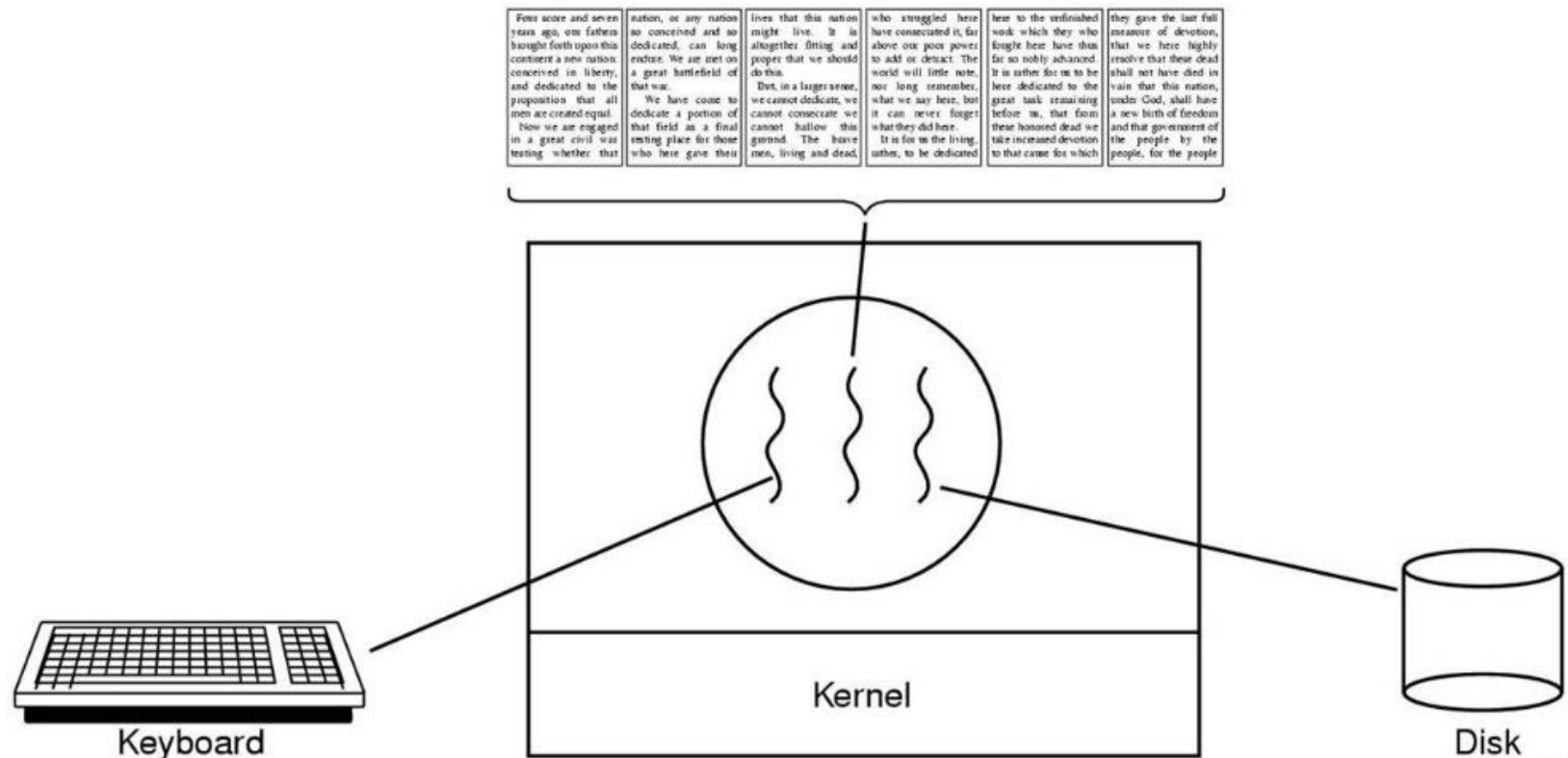


Chapter 2 Process Management

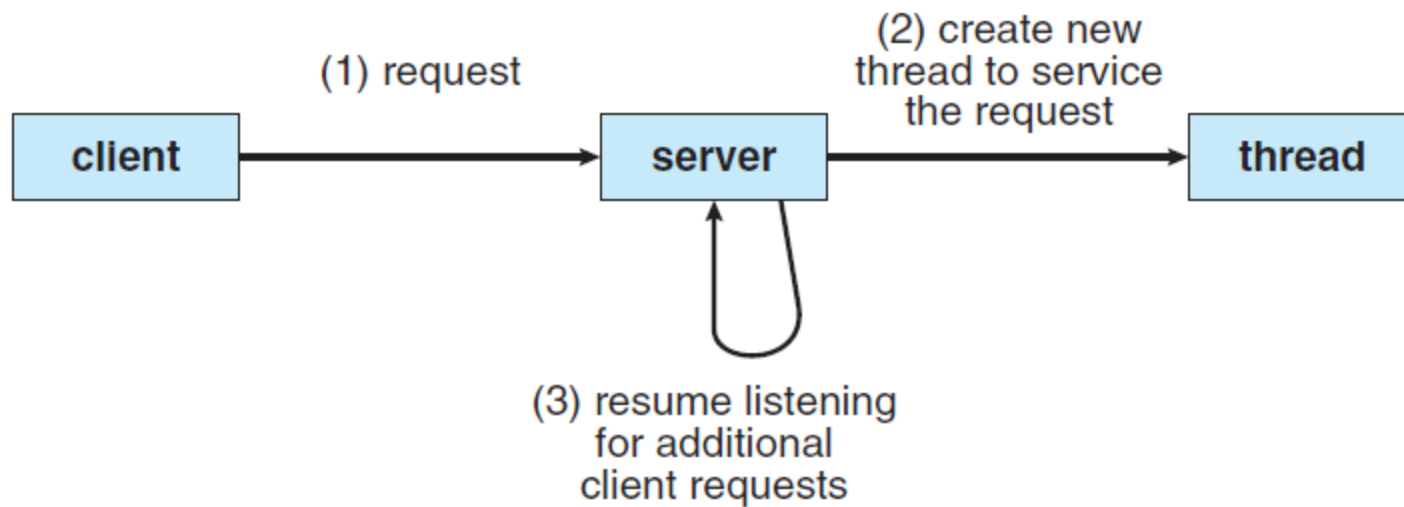
2. Thread

EVERYTHING IS A THREAD

Example: Word processor (Tanenbaum 2001)

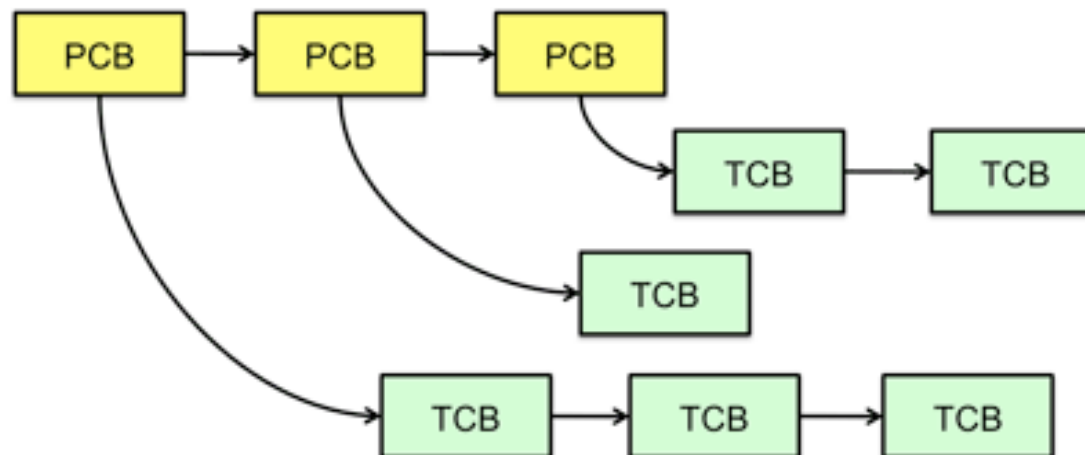


Example: Multithreaded server (Silberschartz)



Concept of thread

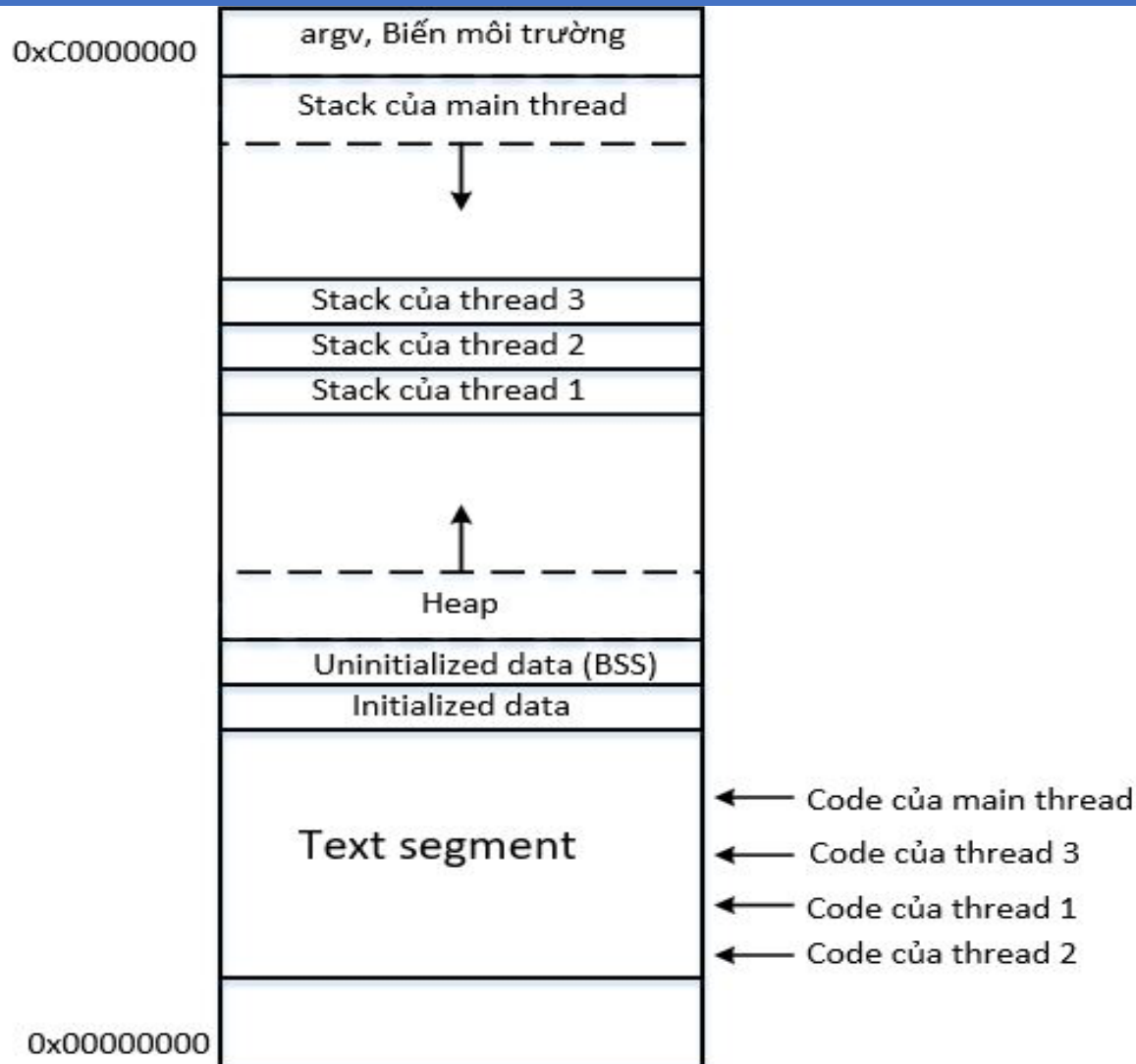
- Basic CPU using unit, consists of
 - Thread ID
 - Program Counter
 - Registers
 - Stack space



Concept of thread

- Threads in the same process share
 - Code segment
 - Data segment (global objects)
 - Other OS's resources (opening file...)
- Thread can run same code segment with different context
(*Register set, program counter, stack*)
- LWP: Lightweight Process
- A process has at least one thread

Process's memory organization with 4 threads (Linux / x86-32)



Distinguishing between process and thread

Process	Thread
Has code/data/heap and other segments	No separating code or heap segment
Has at least 1 thread	Not stand alone but must be inside a process
Threads in the same process share code/data/heap, devices but have separated stack and registers	Many threads can exist at the same time in each process. First thread is the main thread and own process's stack
Create and switch process operations are expansive	Create and switch thread operations are inexpensive
Good protection due to own address space	Common address space, need to protect
When process terminated, resources are returned and threads have to terminated	Thread terminated, its stack is returned

Benefits of multithreaded programming

- Responsiveness
- Resource sharing
- Economy
- Utilization of multiprocessor architectures

Benefits of multithreaded programming

- Responsiveness
 - allow a program to continue running even if part of it is blocked or is performing a long operation
 - Example: A multi-threaded Web browser
 - 1 thread interactives with user
 - 1 thread downloads data
- Resource sharing
 - threads share the memory and the resources of the process
 - Good for parallel algorithms (share data structures)
 - Threads communicate via sharing memory
 - Allow application to have many threads act in the same address space

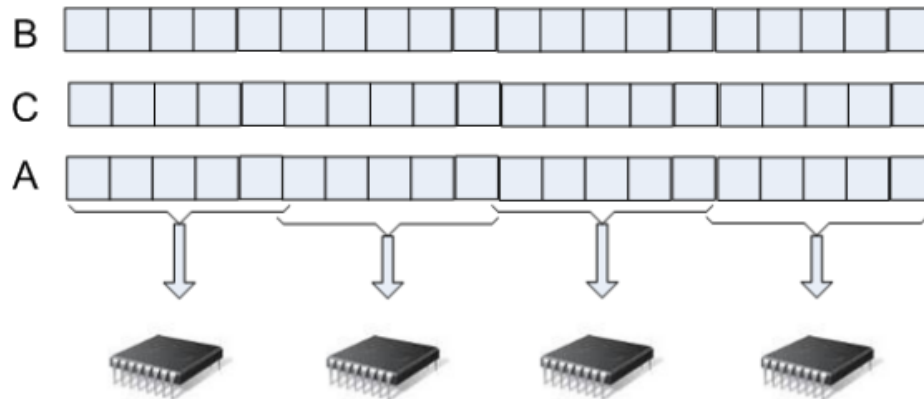
Benefits of multithreaded programming

- Economy
 - Create, switch, terminate threads is less expensive than process
- Utilization of multiprocessor architectures
 - each thread may be running in parallel on a different processor.

Benefit of multithreading-> example

Vector computing

```
for (k = 0; k < n; k++) {  
    a[k] = b[k]*c[k];  
}
```



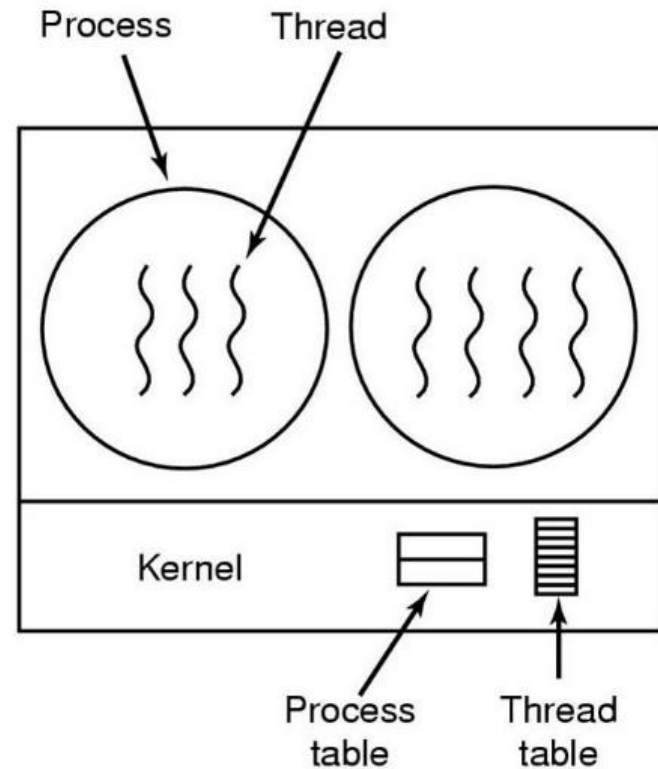
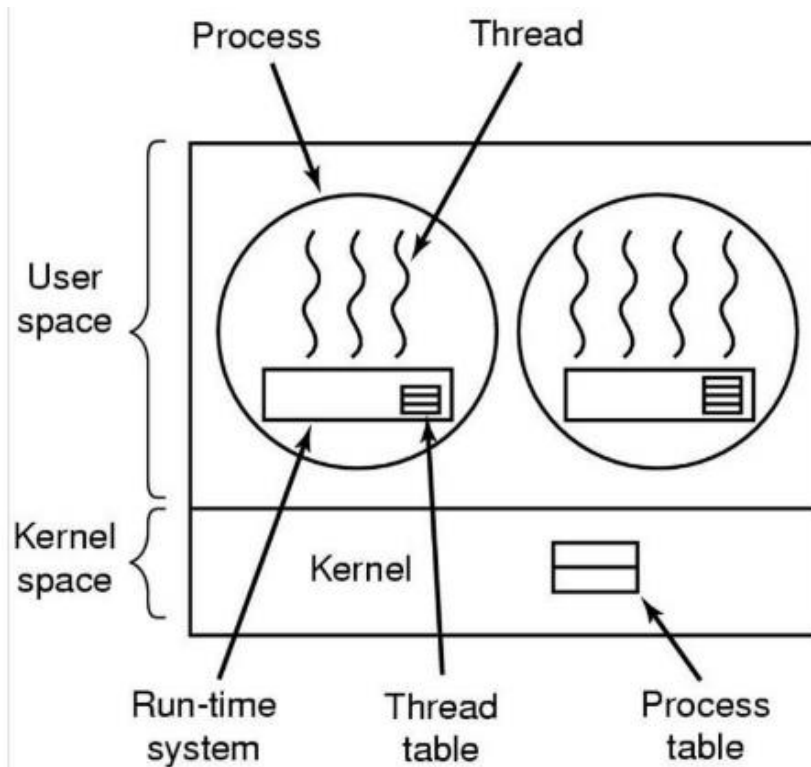
Multi-threading model

```
void fn(a,b)  
for(k = a; k < b; k ++){  
    a[k] = b[k] * c[k];  
}  
  
void main(){  
    CreateThread(fn(0, n/4));  
    CreateThread(fn(n/4, n/2));  
    CreateThread(fn(n/2, 3n/4));  
    CreateThread(fn(3n/4, n));  
}
```

Thread implementation

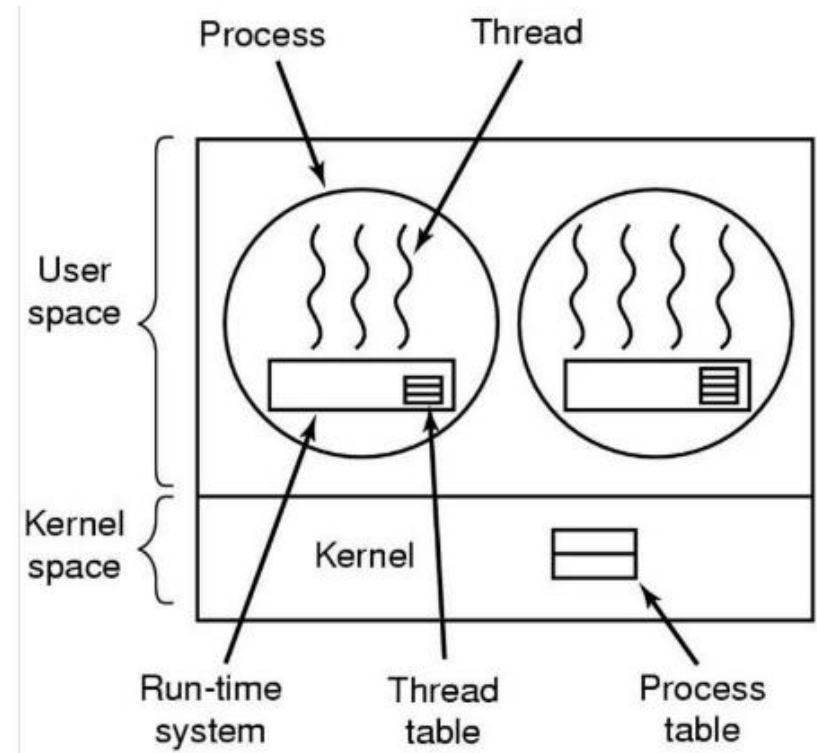
User space

Kernel space



User -Level Threads

- Thread management is done by application
- Kernel **does not know** about thread existence
 - Process scheduled like a single unit
 - Each process is assigned with a single state
 - Ready, waiting, running,...
- User threads are supported above the kernel and are implemented by a **thread library**
 - Library support creation, scheduling and management..

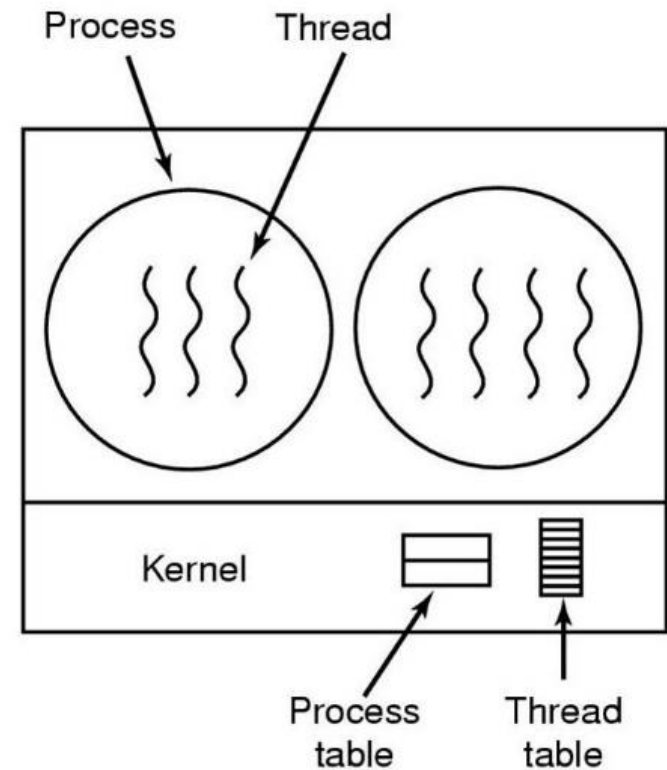


User -Level Threads

- Advantage
 - Fast to create and manage
- Disadvantage
 - if a thread perform blocking system call , the entire process will be blocked $\Rightarrow$ Cannot make use of multi-thread.

Kernel - Level threads

- Kernel keeps information of process and threads
- Threads management is performed by kernel
 - No thread management code inside application
 - Thread scheduling is done by kernel
- OSs support kernel thread: Windows NT/2000/XP, Linux, OS/2,...



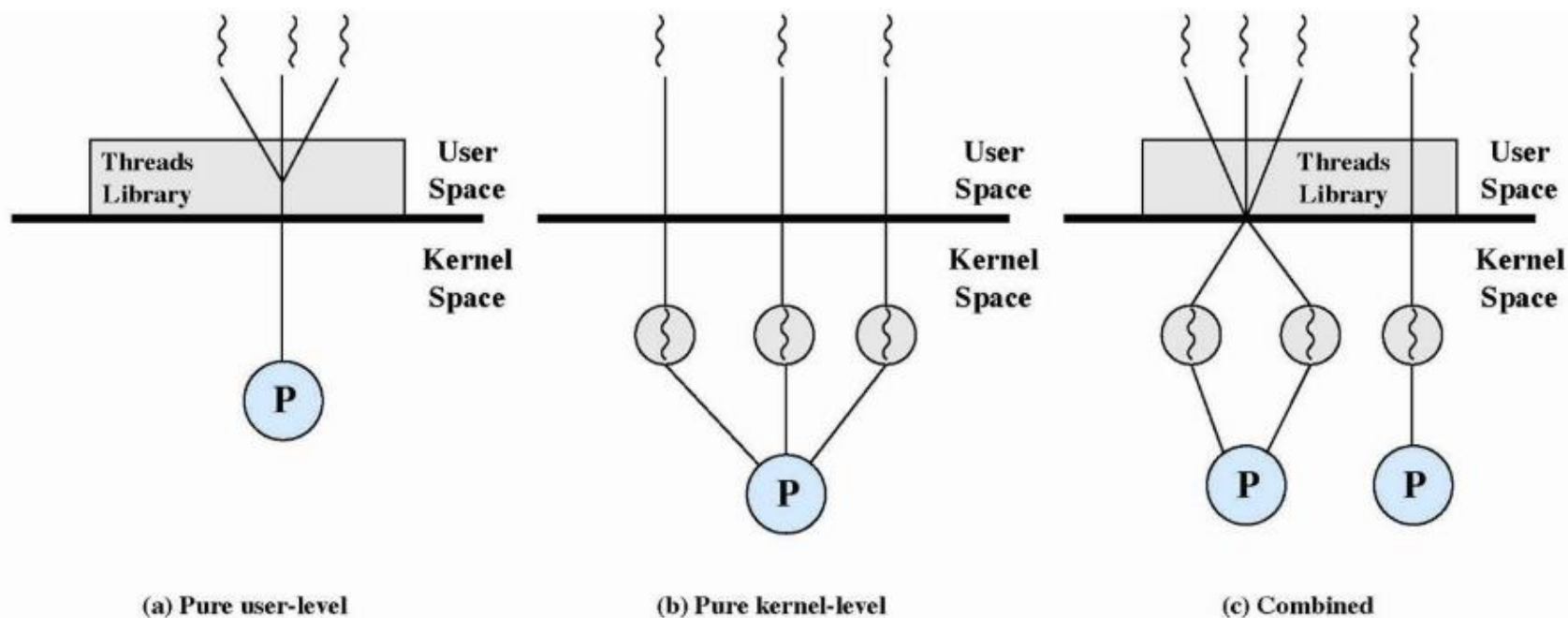
Kernel - Level threads

- Advantage:
 - 1 thread perform system call (e.g. I/O request), other threads are not affected
 - In a multiprocessor environment, the kernel can schedule threads on different processors
- Disadvantage:
 - Slow in thread creation and management

- Introduction
- **Multithreading model**
- Thread implementation with Windows
- Multithreading problem

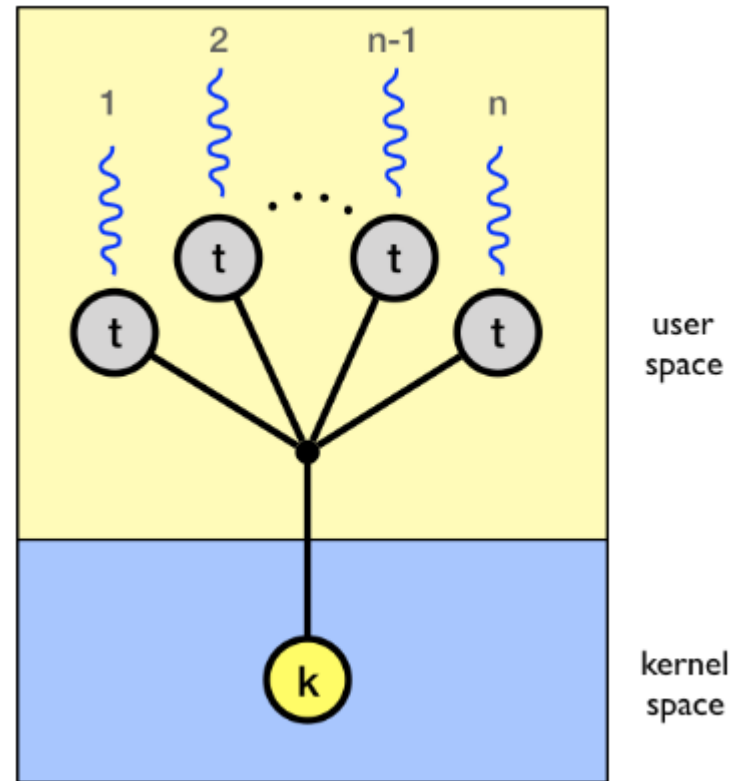
Introduction

- Many systems provide support for both user and kernel threads -
> different multithreading models

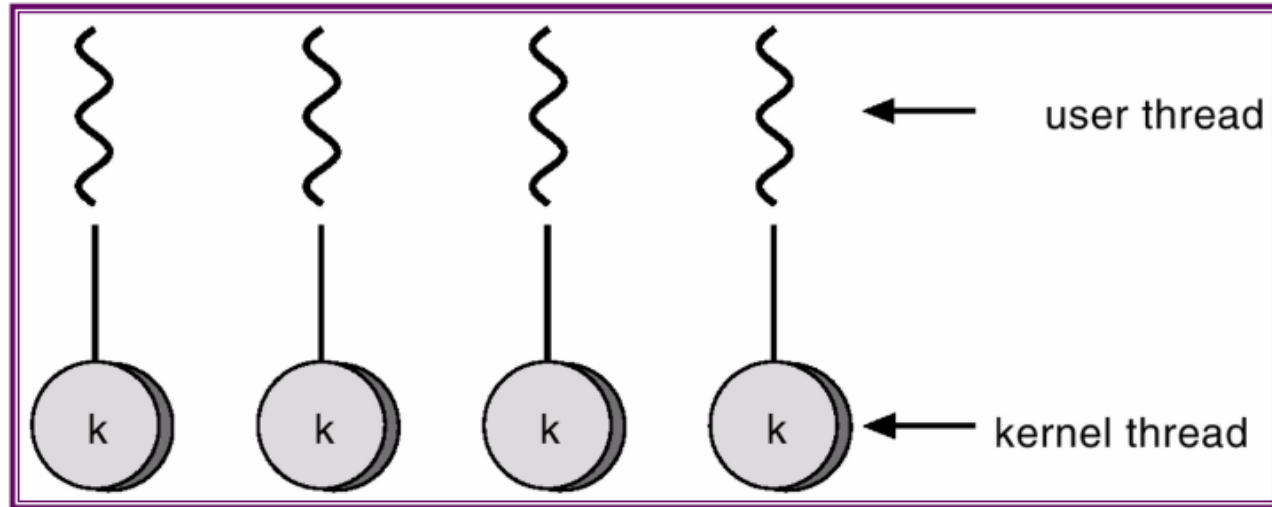


Many-to-One Model

- Maps many user-level threads to 1 kernel thread.
- Thread management is done in user space
 - Efficient
 - the entire process will block if a thread makes a blocking system call
 - multiple threads are unable to run in parallel on multi processors
- implemented on OSs that do not support kernel threads use the many-to-one model



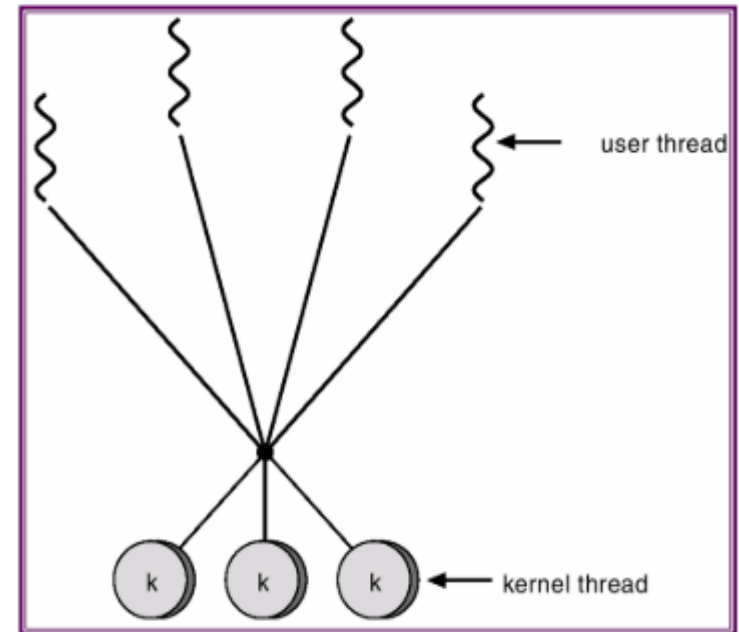
One-to-one Model



- Maps each user thread to a kernel thread
- Allow another thread to run when a thread makes a blocking system call
- Creating a user thread requires creating the corresponding kernel thread
 - the overhead of creating kernel threads can burden the performance of an application
 - $\Rightarrow$ restrict the number of threads supported by the system
- Implemented in Window NT/2000/XP

Many-to-Many Model

- Multiplexes many user-level threads to a smaller or equal number of kernel threads
- Number of kernel threads: specific to either a particular application or a particular machine
 - E.g.: on multiprocessor -> more kernel thread than on uniprocessor
- Combine advantages of previous models
 - Developers can create as many user threads as necessary
 - kernel threads can run in parallel on a multiprocessor
 - when a thread performs a blocking system call, the kernel can schedule another thread for execution
- Supported in: UNIX



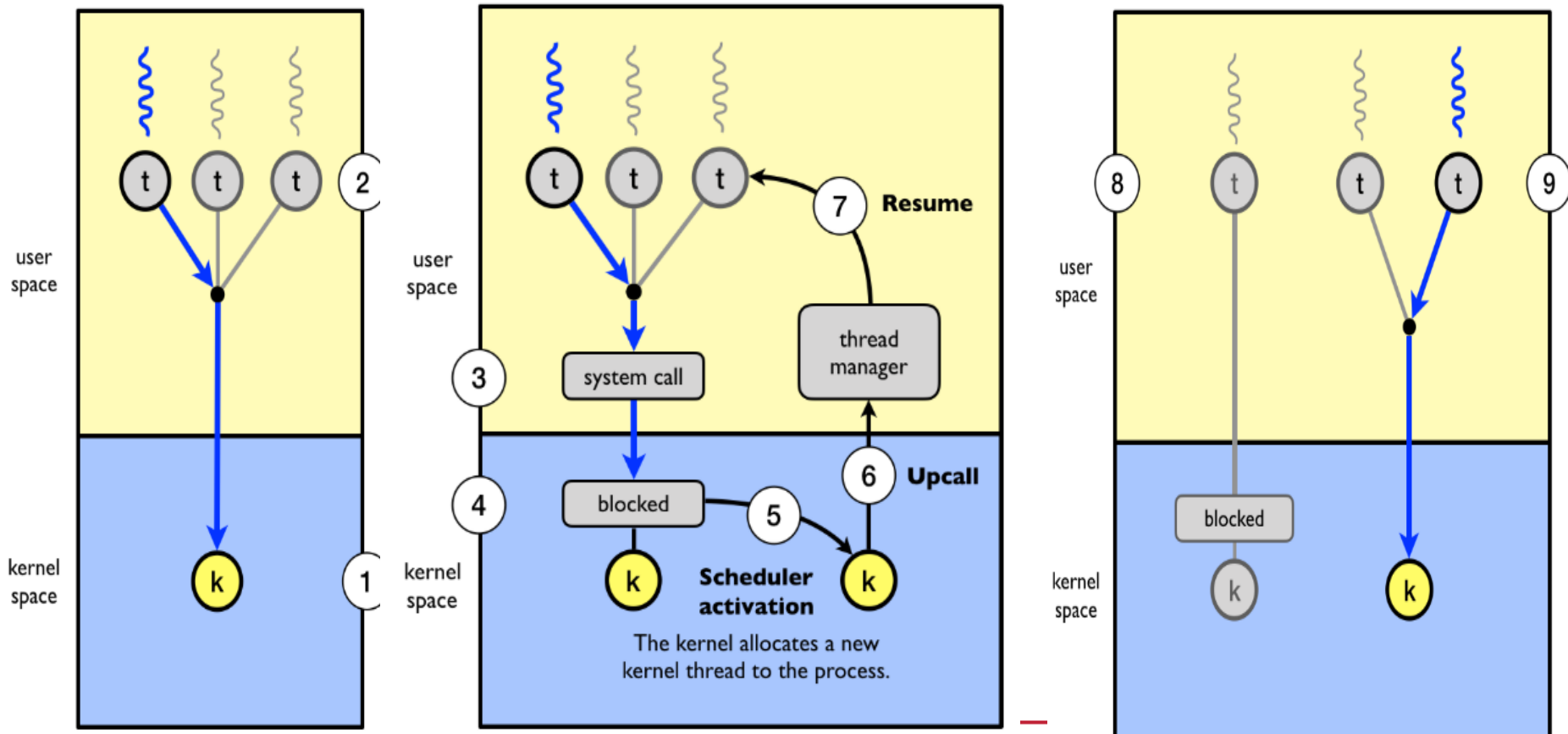
Chapter 2 Process Management

2. Thread

event-driven threading model

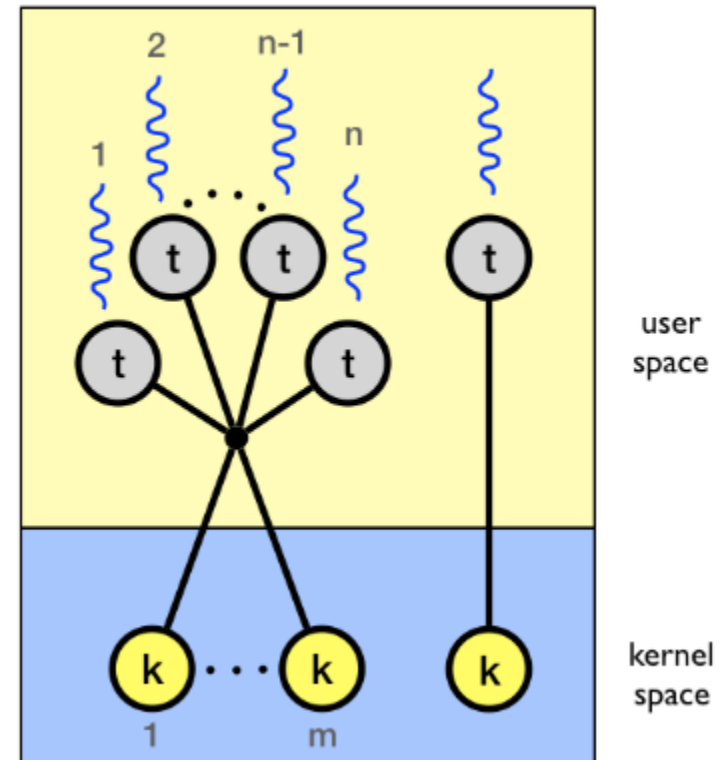
scheduler activation

- A way for **kernel** to **communicate** with **user level thread manager** to maintain an appropriate **number of kernel level thread** for process



Two-level model

- A variation of many to many
- Allow favor process with higher priority



- Introduction
- Multithreading model
- Thread implementation with Windows
- Multithreading problem

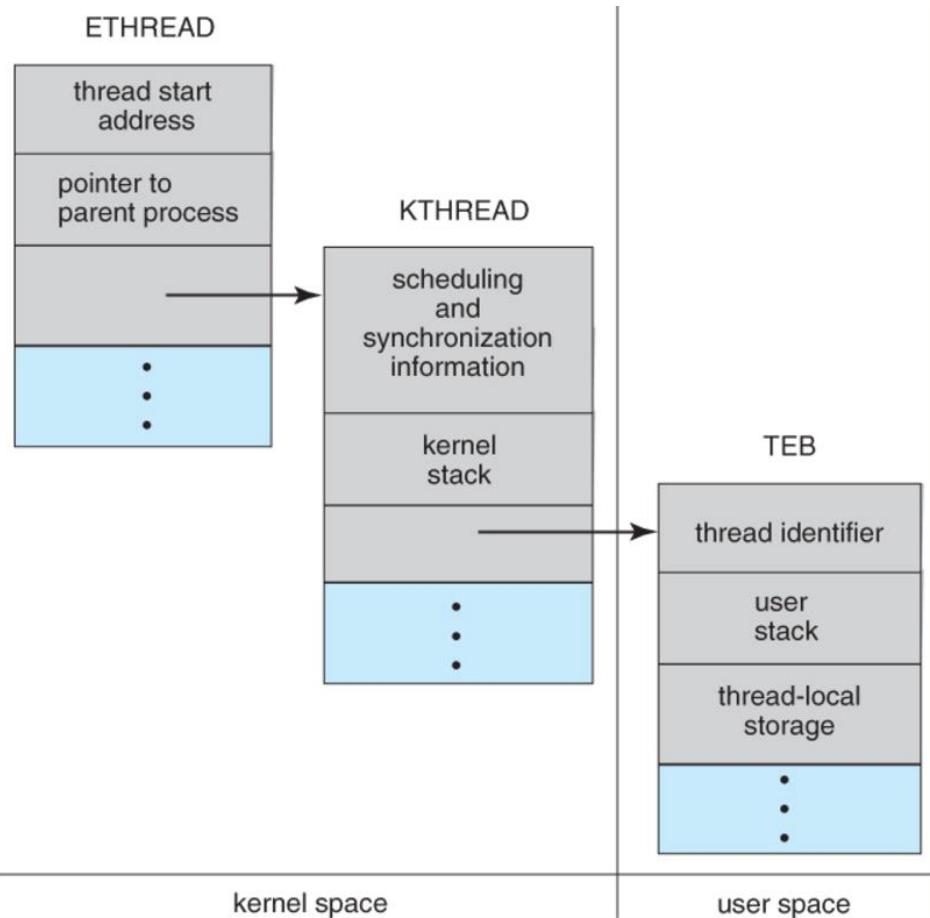
Functions with thread in WIN32 API

- HANDLE CreateThread(. . .);
 - LPSECURITY_ATTRIBUTES lpThreadAttributes,
⇒ A pointer to a SECURITY_ATTRIBUTES structure that determines whether the returned handle can be inherited by child processes
 - DWORD dwStackSize,
⇒ The initial size of the stack, in bytes
 - LPTHREAD_START_ROUTINE lpStartAddress,
⇒ pointer to the application-defined function to be executed by the thread
 - LPVOID lpParameter,
⇒ A pointer to a variable to be passed to the thread
 - DWORD dwCreationFlags,
⇒ The flags that control the creation of the thread
 - CREATE_SUSPENDED : thread is created in a suspended state
 - 0: thread runs immediately after creation
 - LPDWORD lpThreadId
⇒ pointer to a variable that receives the thread identifier
- Return value: handle to the new thread or NULL if the function fails

Example

```
#include <windows.h>
#include <stdio.h>
void Routine(int *n){
    printf("My argument is %d\n", &n);
}
int main(){
    int i, P[5]; DWORD Id;
    HANDLE hHandles[5];
    for (i=0;i < 5;i++) {
        P[i] = i;
        hHandles[i] = CreateThread(NULL,0,
                                   (LPTHREAD_START_ROUTINE)Routine,&P[i],0,&Id);
        printf("Thread %d was created\n",Id);
    }
    for (i=0;i < 5;i++) WaitForSingleObject(hHandles[i],INFINITE);
    return 0;
}
```


Thread in Windows XP



Thread includes

- Thread ID
- Registers
- user stack used in user mode, kernel stack used in kernel mode.
- Separated memory area used by runtime and dynamic linked library

Executive thread block

Kernel thread block

Thread environment block

- Introduction
- Multithreading model
- Thread implementation with Windows
- Multithreading problem

Example

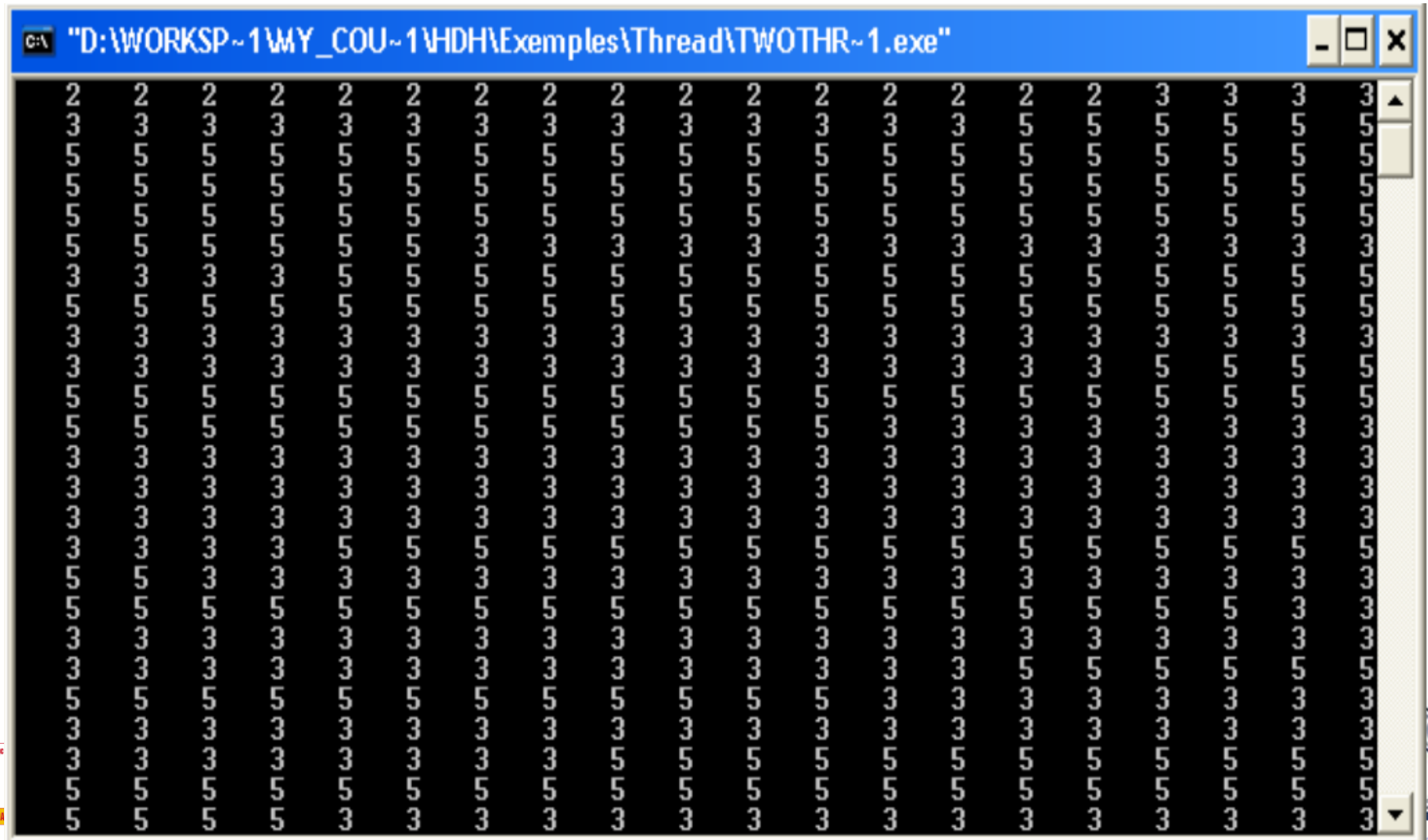
```
#include <windows.h>
#include <stdio.h>
int x = 0, y = 1;
void T1(){
    while(1){ x = y + 1; printf("%4d", x); }
}
void T2(){
    while(1){ y = 2; y = y * 2; }
}
int main(){
    HANDLE h1, h2; DWORD Id;
    h1=CreateThread(NULL,0,(LPTHREAD_START_ROUTINE)T1,NULL,0,&Id);
    h2=CreateThread(NULL,0,(LPTHREAD_START_ROUTINE)T2,NULL,0,&Id);
    WaitForSingleObject(h1,INFINITE);
    WaitForSingleObject(h2,INFINITE);
    return 0;
}
```

Chapter 2 Process Management

2. Thread

2.1.1 Running thread problem

Running result



```
C:\ "D:\WORKSP~1\MY_COU~1\HDH\Exemples\Thread\TWOTHR~1.exe"

2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 2555555555 3555555555 3555555555 3555555555 3555555555
```

Explanation

Shared int $y = 1$	
Thread T_1	Thread T_2
$x \leftarrow y + 1$	$y \leftarrow 2$
	$y \leftarrow y * 2$
$x = ?$	

Thread T1

$x = y + 1$ $\longleftrightarrow$

$x = y + 1$ $\longleftrightarrow$

$x = y + 1$ $\longleftrightarrow$

Thread T2

$\longleftrightarrow y = 2$

$\longleftrightarrow y = y * 2$

The result of parallel running threads depends on the order of accessing the sharing variable



- ① Process
- ② Thread
- ③ **CPU scheduling**
- ④ Critical resource and process synchronization
- ⑤ Deadlock and solutions

- Basic Concepts
- Scheduling Criteria
- Scheduling algorithms
- Multi-processor scheduling

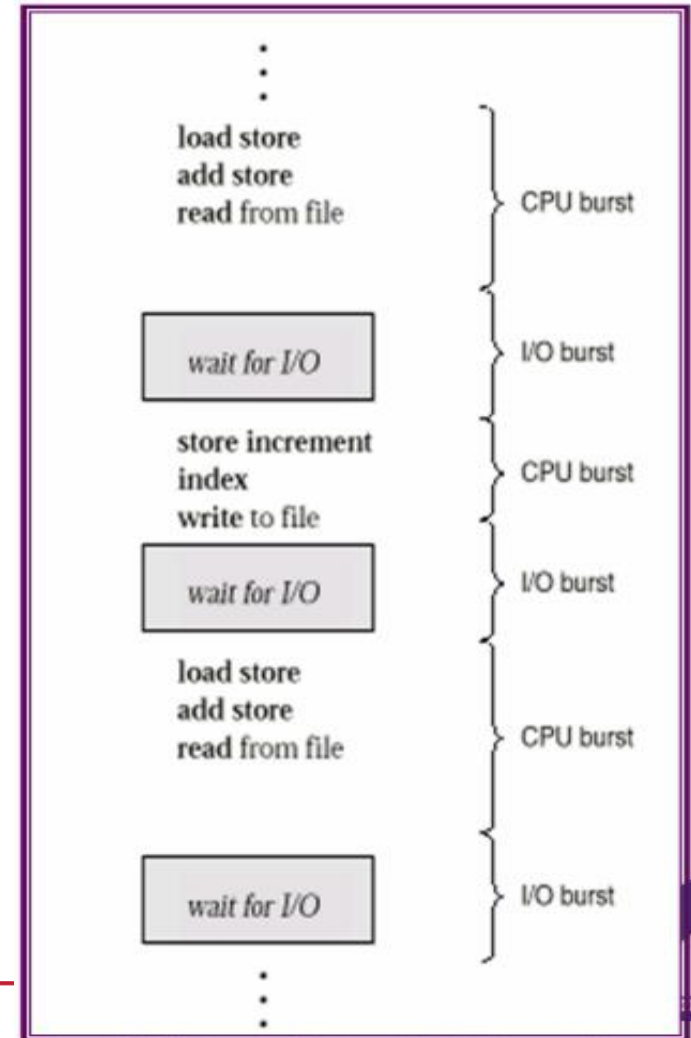
Introduction

- System has one *processor* → Only 1 process is running at a time
- Process is *executed* until it must *wait*, typically for the *completion* of some *I/O request*
 - *Simple system*: CPU is not used ⇒ Waste CPU time
 - *Multiprogramming system*: try to use this time productively, give CPU for another process
 - Several *ready processes* are kept *inside memory* at a single time
 - When a process has to *wait*, OS takes the *CPU* away and gives the *CPU* to *another process*
- *Scheduling* is a fundamental OS's function
 - Switch CPU between processes → exploit the system more efficiently

Introduction

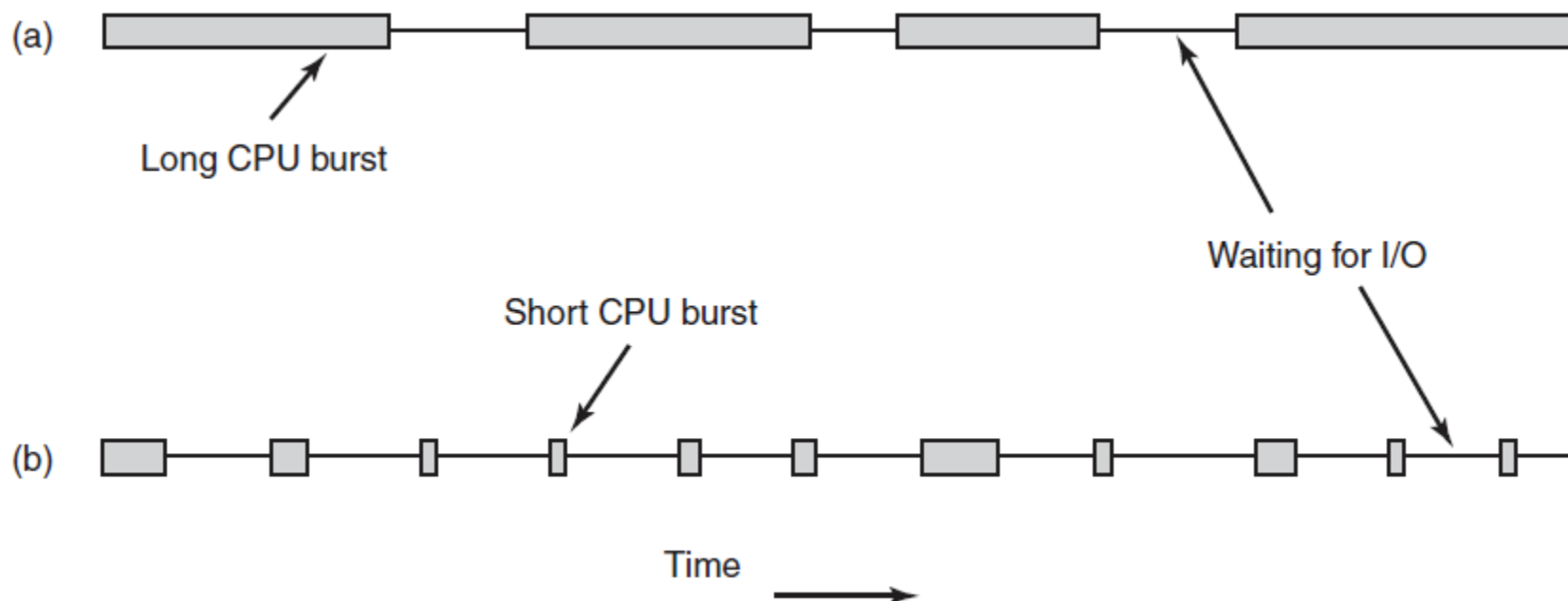
CPU-I/O Burst Cycle

- Process execution consists of a cycle of CPU execution and I/O wait
 - Begins with a **CPU burst**
 - followed by an **I/O burst**
 - CPU burst → I/O burst → CPU burst → I/O burst → . . .
 - End: the last CPU burst will end with a system request to terminate execution
- Process classification
 - Based on distribution of CPU & I/O burst
 - **CPU-bound** program might have a few very long
 - **I/O-bound** program would typically have many very short CPU bursts
 - help us select an appropriate CPU-scheduling algorithm



CPU-I/O Burst Cycle CPU - I/O

- Distinguish the types of processes
 - Based on time allocation for CPU & I / O cycles
 - CPU-bound process has several long CPU cycles
 - I/O-bound process has many short CPU cycles
 - To choose the appropriate scheduling algorithm



CPU Scheduler

- CPU idle -> select 1 process from ready queue to run it
 - Process in ready queue
 - FIFO queue, Priority queue, Simple linked list . . .
- CPU scheduling decisions may take place when process switches from
 - 1) running -> waiting state (I/O request)
 - 2) running -> ready state (out of CPU usage time → time interrupt)
 - 3) waiting -> ready state (e.g. completion of I/O)
 - 4) Or process terminates
- Note
 - Case 1&4 ⇒ non-preemptive scheduling scheme
 - Other cases ⇒ preemptive scheduling scheme

Preemptive and non-preemptive Scheduling

● Non-preemptive scheduling

- Process keeps the CPU until it releases the CPU either by
 - terminating
 - switching to the waiting state
 - does not require the special hardware (timer)
- Example: DOS, Win 3.1, Macintosh

Preemptive and non-preemptive Scheduling

● Preemptive scheduling

- Process only allowed to run in specified period
 - End of period, time interrupt appear, dispatcher is invoked to decide to resume process or select another process
- Protect CPU from "CPU hungry" processes
- Sharing data problem
 - Process 1 updating data and the CPU is taken
 - Process 2 executed and read data which is not completely update
- Example: Multiprogramming OS WinNT, UNIX

Preemptive and non-preemptive Scheduling

- Basic Concepts
- **Scheduling Criteria**
- Scheduling algorithms
- Multi-processor scheduling

Scheduling Criteria I

● CPU utilization

- keep the CPU as busy as possible
- should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system)

● Throughput

- number of processes completed per time unit
 - Long process: 1 process/hour
 - Short processes: 10 processes/second

● Turnaround time

- Interval from the time of submission of a process to the time of completion
- Can be the sum of
 - periods spent waiting to get into memory
 - waiting in the ready queue
 - executing on the CPU
 - doing I/O

Scheduling Criteria II

●Waiting time

- sum of the periods spent waiting in the ready queue
- CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/O; it affects only the amount of time that a process spends waiting in the ready queue

●Response time

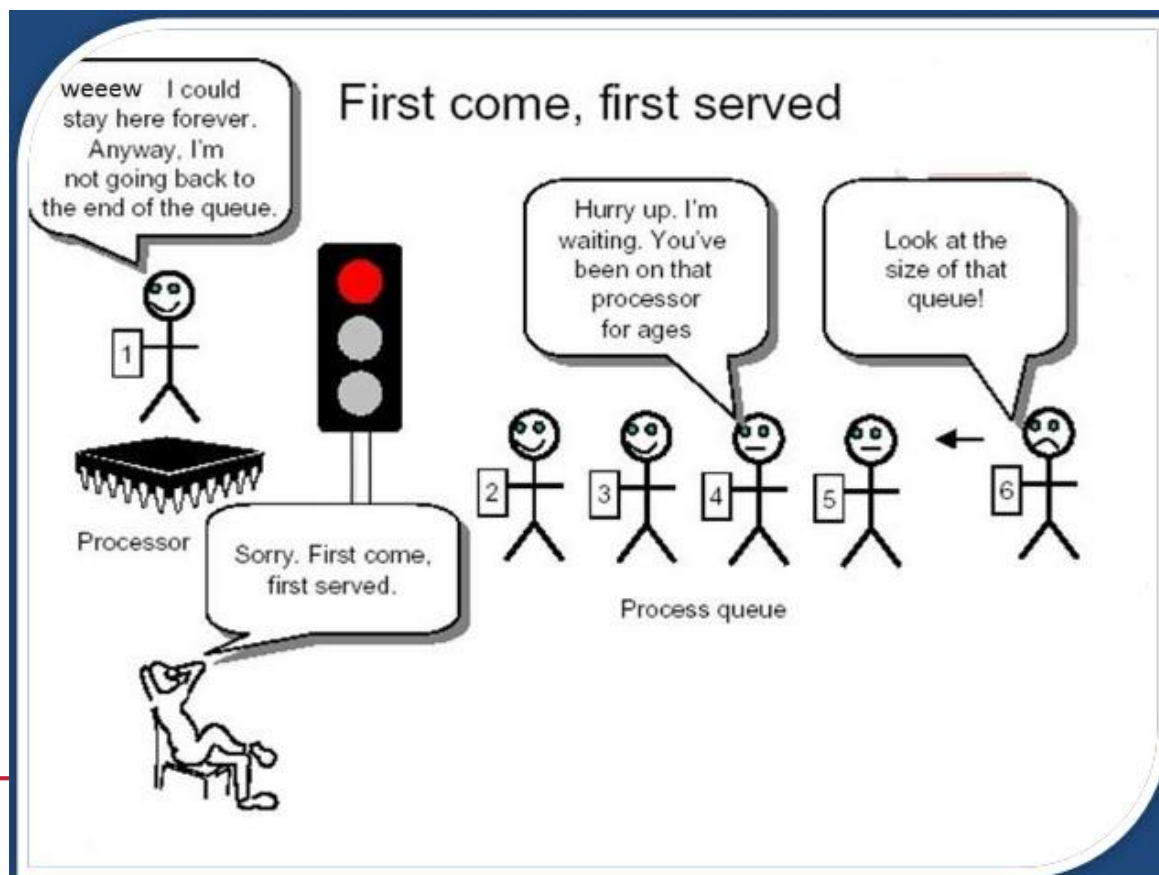
- time from the submission of a request until the first response is produced
 - process can produce some output fairly early
 - continue computing new results while previous results are being output to the user

- Assumption: Consider one CPU burst (ms) per process
- Measure: Average waiting time

- Basic Concepts
- Scheduling Criteria
- **Scheduling algorithms**
- Multi-processor scheduling

FCFS: First Come, First Served

- Rule: process that requests the CPU first is allocated the CPU first
- Process owns CPU until it terminates or blocks for I/O request



FCFS: First Come, First Served

- Rule: process that requests the CPU first is allocated the CPU first
Process owns CPU until it terminates or blocks for I/O request

- Example

Process	Burst Time
P_1	24
P_2	3
P_3	3



- Pros and cons
 - Simple, easy to implement
 - Short process must wait like long process
 - If P₁ executed last?

SJF: Shortest Job First

- Rule: associates with each process the length of the latter's next CPU burst
 - process that has the smallest next CPU burst
 - Two methods
 - Non-preemptive
 - preemptive (SRTF: Shortest Remaining Time First)
- Example

Process	Burst Time	Arrival Time
P_1	8	0.0
P_2	4	1.0
P_3	9	2.0
P_4	5	3.0
- Pros and Cons
 - SJF (SRTF) is optimal: Average waiting time is minimum
 - It's not possible to predict the length of next CPU burst
 - Predict based on previous one

Priority Scheduling

- Each process is attached with a **priority value** (a number)
 - CPU is allocated to the process with the highest priority
 - SJF: priority (p) is the inverse of the (predicted) next CPU burst
 - 2 methods
 - Non-preemptive
 - Preemptive

- **Example**

Process	Burst Time	Priority	Arrival Time
P_1	10	3	0
P_2	1	1	1
P_3	2	4	2
P_4	1	5	3
P_5	5	2	4

- **“Starvation” problem:** low-priority process has to wait indefinitely (even forever)
- **Solution:** increase priority by the time process wait in the system

Round Robin Scheduling

●Rule

- Each process is given a time quantum (time slice) τ to be executed
- When time's up processor is preemptive, and process is placed in the last position of ready queue
- If there are n process, longest waiting time is $(n - 1)\tau$

●Example

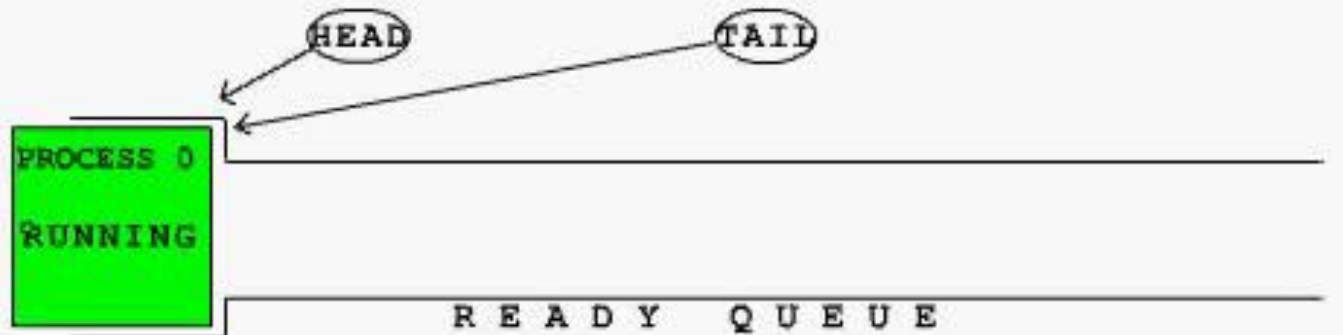
Process	Burst Time
P_1	24
P_2	3
P_3	3

quantum $\tau = 4$

●Problem: select τ

- τ large: FCFS
- τ small: CPU is switched frequently
- Commonly $\tau = 10\text{-}100\text{ms}$

Round Robin Scheduling



Max Ticks Allowed = 15

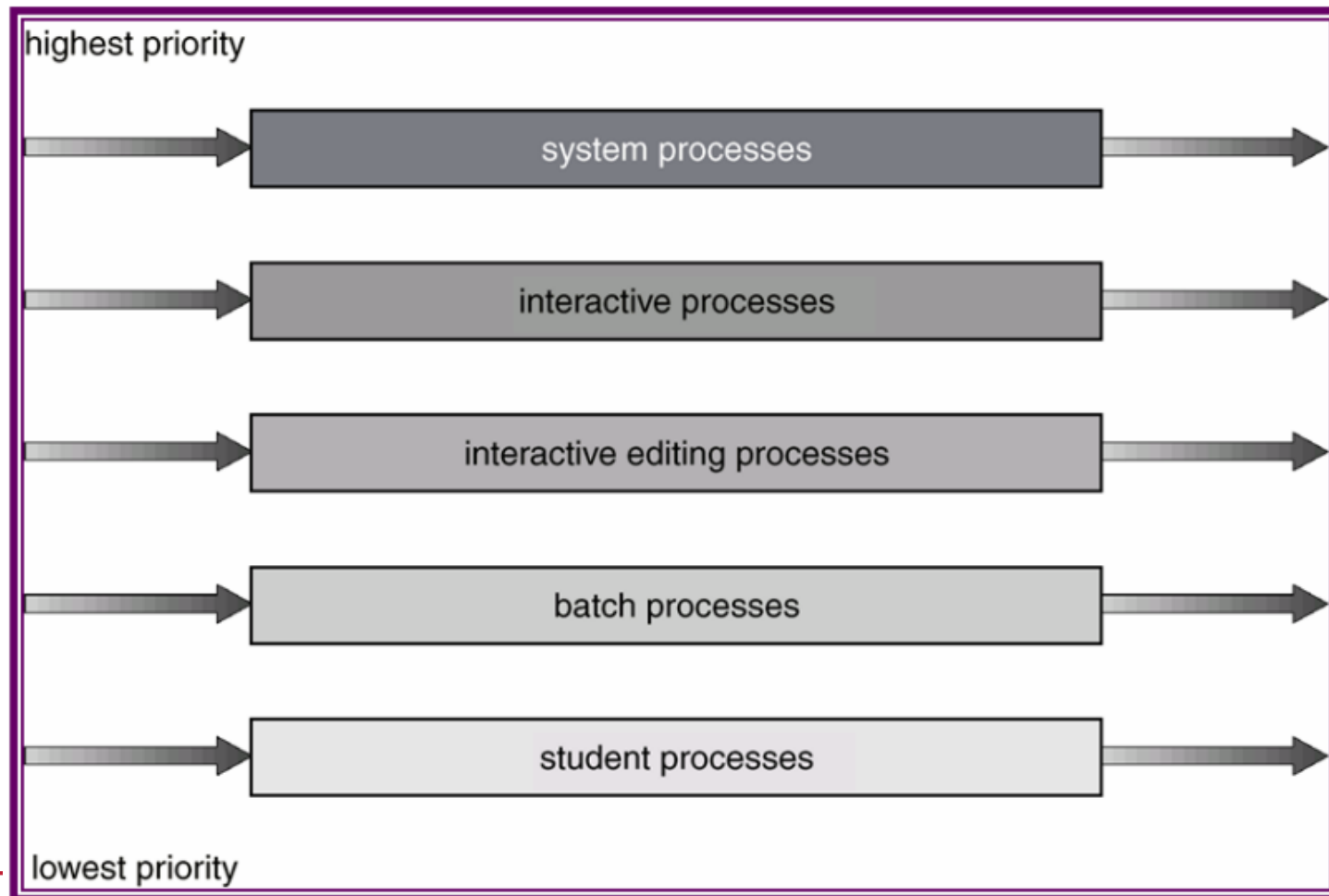
Multilevel Queue Scheduling

- Ready queue is divided into several separate queues
- Processes are permanently assigned to 1 queue
 - Based on some properties of the process, such as memory size, priority, or type..
- Each queue has its own scheduling algorithm

Multilevel Queue Scheduling

- among the queues, the scheduler must consider
 - Commonly implemented as fixed-priority preemptive scheduling
 - Processes in **lower priority** queue only executed if **higher priority** queues are **empty**
 - **High priority** process **preemptive** CPU from **lower** priority process
 - *Starvation* is possible
 - Time slice between the queues
 - **foreground** process queue, 80% CPU time for RR
 - **background** process queue, 20% CPU time for FCFS

Multilevel Queue Scheduling (Example)



Multilevel Feedback Queue

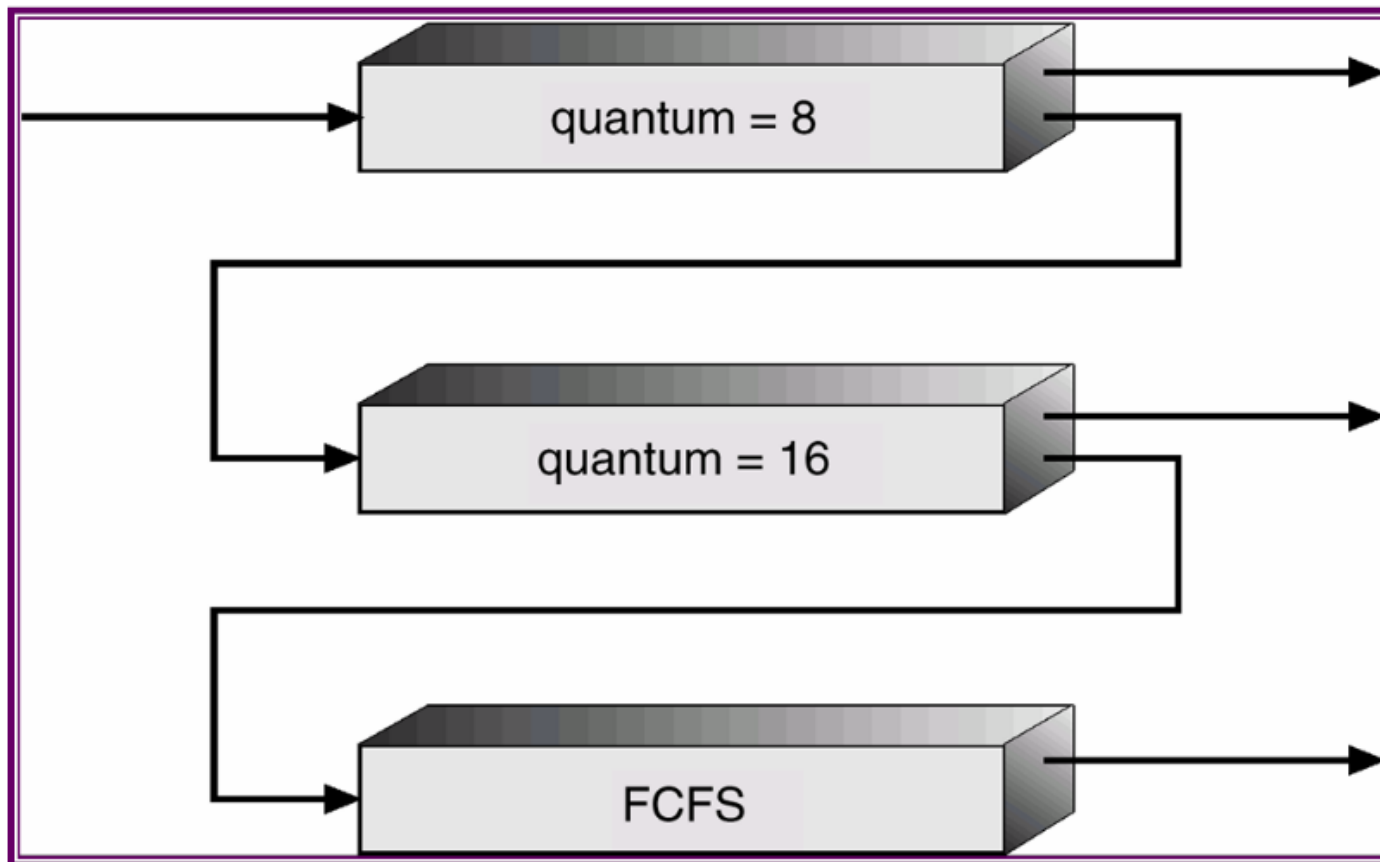
- Allows a process to move between queues
- separate processes with different CPU-burst characteristics
 - If a process uses too much CPU time -> moved to a lower-priority queue
 - I/O-bound and interactive processes in the higher-priority queues
 - process that waited too long in a lower- priority queue may be moved to a higher-priority queue
 - Prevent “starvation”

Multilevel Feedback Queue

- Queue's **scheduler** is defined by the following parameters:
 - number of queues
 - scheduling algorithm for each queue
 - method used to determine when to upgrade/demote a process to a higher/lower priority queue
 - method used to determine which queue a process will enter when that process needs service

Multilevel Feedback Queue

● Example



- Basic Concepts
- Scheduling Criteria
- Scheduling algorithms
- Multi-processor scheduling

Problem

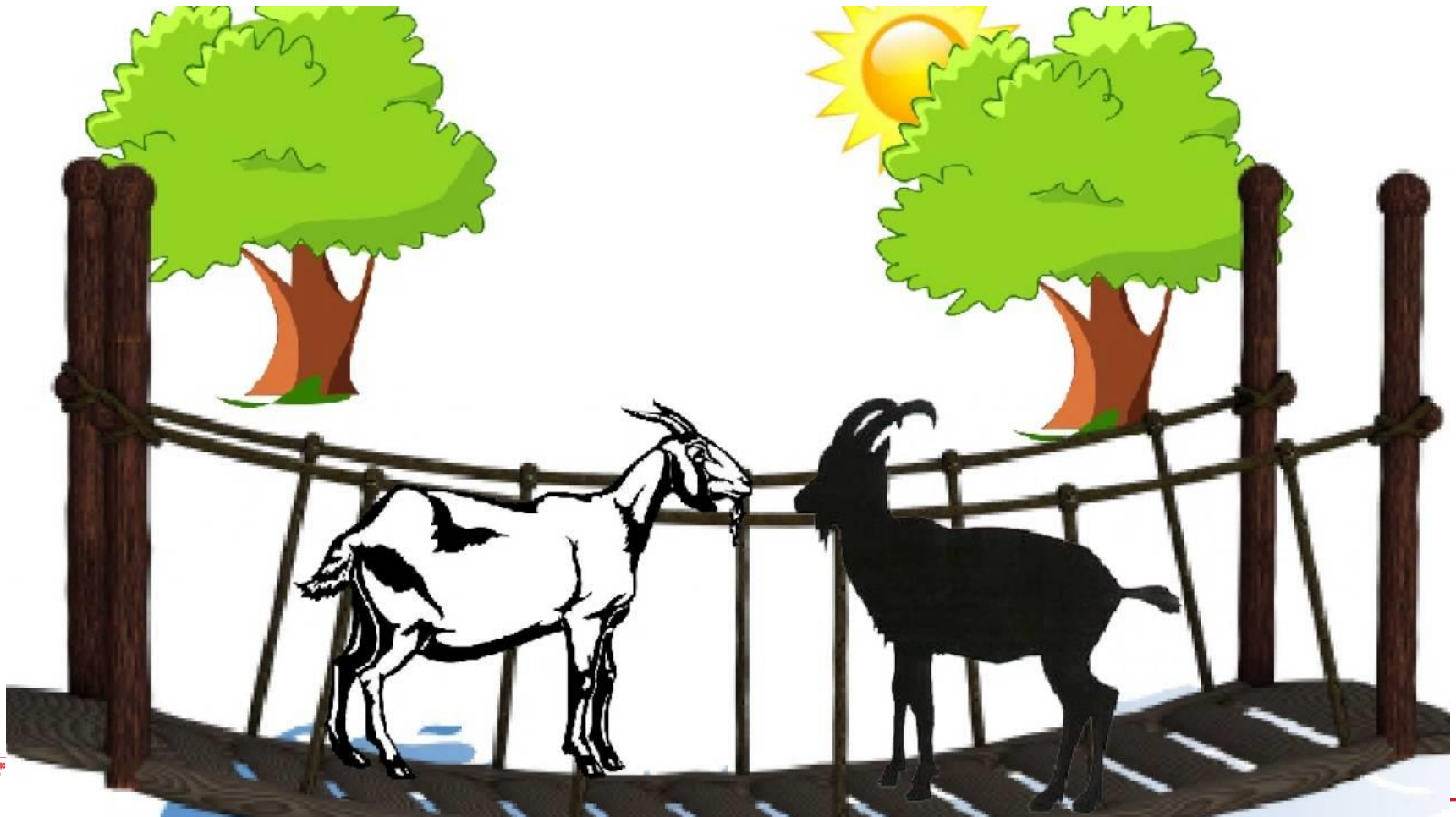
- the scheduling problem is correspondingly more complex
- Load sharing
 - separate queue for each processor
 - one processor could be idle, with an empty queue, while another processor was very busy
 - use a common ready queue
 - problems :
 - two processors choose the same process or
 - processes are lost from the queue
- asymmetric multiprocessing
 - only one processor accesses the system's queue -> no sharing problem
 - I/O-bound processes may bottleneck on the CPU that is performing all of the operations

Homework

- Write program to simulate multilevel feedback queue

- ① Process
- ② Thread
- ③ CPU scheduling
- ④ Critical resource and process synchronization
- ⑤ Deadlock and solutions

- Critical resource
- Internal lock method
- Test and Set method
- Semaphore mechanism
- Process synchronization example
- Monitor



Chapter 2 Process Management

4. Critical resource and process synchronization

4.1 Critical Resource





Example

```
#include <windows.h>
#include <stdio.h>
int x = 0, y = 1;
void T1(){
    while(1){ x = y + 1; printf("%4d", x); }
}
void T2(){
    while(1){ y = 2; y = y * 2; }
}
int main(){
    HANDLE h1, h2; DWORD Id;
    h1=CreateThread(NULL,0,(LPTHREAD_START_ROUTINE)T1,NULL,0,&Id);
    h2=CreateThread(NULL,0,(LPTHREAD_START_ROUTINE)T2,NULL,0,&Id);
    WaitForSingleObject(h1,INFINITE);
    WaitForSingleObject(h2,INFINITE);
    return 0;
```



Chapter 2 Process Management

4. Critical resource and process synchronization

Results

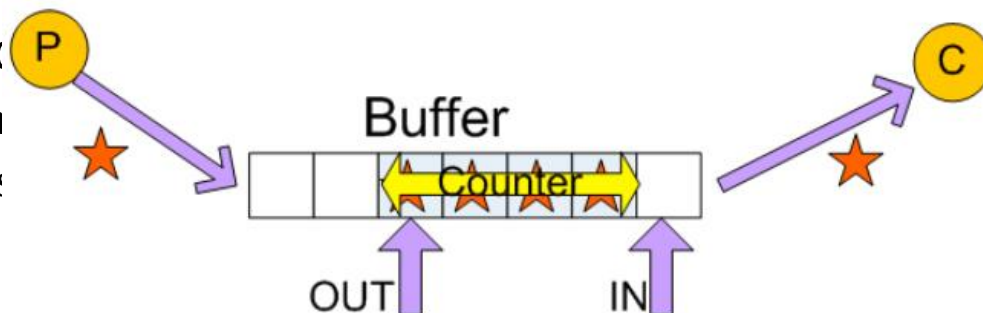


- 2 processes in the system

producer-consumer I

- **Producer** and

- Do not crea
- Do not con:



Producer-Consumer Problem II

Producer

```
while(1) {  
    /*produce an item in  
nextProduced*/  
    while (Counter == BUFFER_SIZE) ;  
        /*do nothing*/  
    Buffer[IN] = nextProduced;  
    IN = (IN + 1) % BUFFER_SIZE;  
    Counter++;  
}
```

Consumer

```
while(1){  
    while(Counter == 0) ;  
        /*do nothing*/  
    nextConsumed = Buffer[OUT];  
    OUT = (OUT + 1) % BUFFER_SIZE;  
    Counter--; /*consume the item in  
nextConsumed*/  
}
```

Remark

- Producer creates 1 product
 - Consumer consumes 1 product
- ⇒ Number of product inside Buffer does not change

Producer-Consumer Problem II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

R1 = ?

R2 = ?



counter = 5



Chapter 2 Process Management

4. Critical resource and process synchronization

4.1 Critical resource

Producer-Consumer Problem II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

Load R1, counter ↔

R1 = 5

t

R2 = ?

counter = 5



Chapter 2 Process Management

4. Critical resource and process synchronization

4.1 Critical resource

Producer-Consumer Problem II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

Load R1, counter



Load R2, counter

R1 = 5

t

R2 = 5

counter = 5



Chapter 2 Process Management

4. Critical resource and process synchronization

4.1 Critical resource

Producer-Consumer Problem II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

Load R1, counter

Inc R1

Load R2, counter

R1 = 6

R2 = 5

counter = 5

t



Chapter 2 Process Management

4. Critical resource and process synchronization

Bài toán người sản xuất (producer)-người tiêu thụ(consumer) II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

Load R1, counter

Inc R1

Load R2, counter

Dec R2

R1 = 6

R2 = 4

counter = 5



Chapter 2 Process Management

4. Critical resource and process synchronization

4.1 Critical resource

Producer-Consumer Problem II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

Load R1, counter

Inc R1

Store counter, R1

Load R2, counter

Dec R2

R1 = 6

R2 = 4

counter = 6

t



Bài toán người sản xuất (producer)-người tiêu thụ(consumer) II

counter++

Load R1, **counter**
Inc R1
Store **counter**, R1

counter--

Load R2, **counter**
Dec R2
Store **counter**, R2

Counter++

counter--

Load R1, counter

Inc R1

Store counter, R1

Load R2, counter

Dec R2

Store counter, R2

R1 = 6

R2 = 4

counter = 4



t

Definition

Resource

Everything that is required for process's execution

Critical resource

- Resource that is **limited** of **sharing capability**
- **Required concurrently** by processes
- Can be either **physical** devices or **sharing data**

Problem

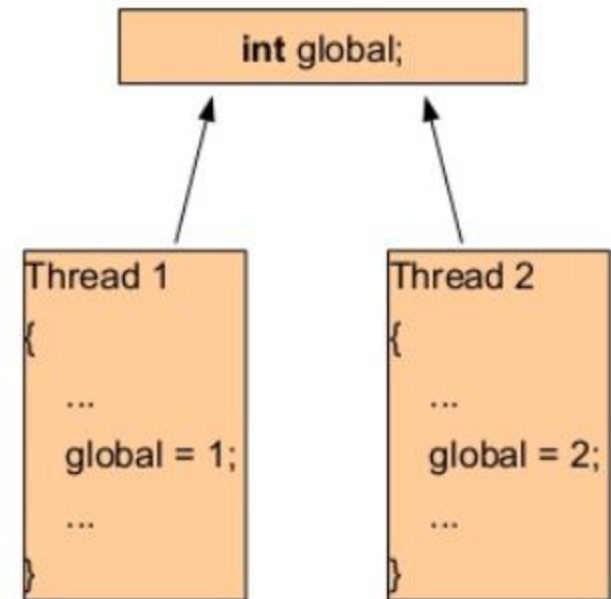
Sharing critical resource may not guarantee **data completeness**
⇒ Require **process synchronization** mechanism

Race condition



Race condition

- Situation where the results of many processes access the sharing data depend of the order of these actions
 - Make program's result undefinable



Race condition

- Prevent race condition by **synchronize concurrent running processes**
 - **Only 1** process can access sharing data **at a time**
 - Variable **counter** in Producer-Consumer problem
 - The **code segments** that **access sharing data** in the processes must be **executed** in a **defined order**
 - E.g.: $x \leftarrow y + 1$ instruction in Thread T_1 only both two instruction of Thread T_2 are done

Critical section

- The part of the program where the **shared memory** is accessed is called the **critical region** or **critical section**
- If there are **more than 1** process use **critical resource**, then we must **synchronize** them
 - Object: guarantee that **no more than 1** process can stay **inside** critical section

Conditions to have a good solution

- **Mutual Exclusion:** Critical resource does not have to serve the number of process more than its capability at any time
 - If process ***P_i*** is executing in its critical section, then no other processes could execute in their critical sections

Conditions to have a good solution

- Mutual Exclusion:
- Progressive: If critical resource still able to serve and there are process want to be executed in critical section then this process can use critical resource

Conditions to have a good solution

- Mutual Exclusion:
- Progressive:
- Bounded Waiting: There exists a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

Rule

- There are 2 processes $P1$ & $P2$ concurrently running
- Sharing the same critical resource
- Each process put the critical section at begin and the remainder section is next
 - Process must **ask before enter** the critical section $\{entry\ section\}$
 - Process perform $\{exit\ section\}$ **after exiting** from critical section
- Program structure

```
do {
```

```
    entry section
```

```
    critical section
```

```
    exit section
```

```
    remainder section
```

```
} while (1);
```

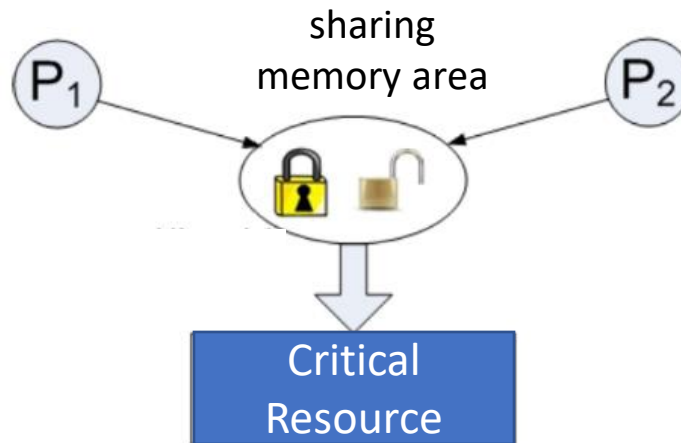
Methods' classification

- Low level method
 - Variable lock
 - Test and set
 - Semaphore
- High level method
 - Monitor

- Critical resource
- **Variable lock method**
- Test and Set method
- Semaphore mechanism
- Process synchronization example
- Monitor

Principle

Principle



- Each process uses 1 byte in the sharing memory area as a **lock**
 - enters critical section, lock (byte lock = true)
 - exits from critical section, unlock (byte lock = false)
- Process want to enter critical section: check other process's **lock** 's status
 - Locking $\Rightarrow$ Wait
 - Not lock $\Rightarrow$ Has the right to enter critical section

Algorithm

- Share var C_1, C_2 Boolean // sharing variable used as lock
- Initialization $C_1 = C_2 = \text{false}$ // Critical resource is free

Process P1

```
do{  
    while( $C_2 == \text{true}$ );  
     $C_1 \leftarrow \text{true}$ ;  
    { Process P1's critical section }  
     $C_1 \leftarrow \text{false}$ ;  
    { Process P1's remaining section }  
}while(1);
```

Process P2

```
do{  
    while( $C_1 == \text{true}$ );  
     $C_2 \leftarrow \text{true}$ ;  
    { Process P2's critical section }  
     $C_2 \leftarrow \text{false}$ ;  
    { Process P2's remaining section }  
}while(1);
```


Algorithm

- Share var C1,C2 Boolean // sharing variable used as lock
- Initialization C1 = C2 = false // Critical resource is free

Process P1

```
do{  
    C1 ← true;  
    while(C2 == true);  
    { Process P1's critical section }  
    C1 ← false;  
    { Process P1's remaining section }  
}while(1);
```

Process P2

```
do{  
    C2 ← true;  
    while(C1 == true);  
    { Process P2's critical section }  
    C2 ← false;  
    { Process P2's remaining section }  
}while(1);
```

- Not properly synchronize

Remark

- Mutual exclusion problem (Case 1)
- Progressive problem (Case 2)
- Reason: The following actions are done separately
 - **Test** the right to enter critical section
 - **Set** the right to enter critical section

- Utilize **turn** variable to show process with priority

Dekker's algorithm

Process P1

do{

```

C1 ← true;
while( C2==true){
    if(turn == 2){
        C1 ← false;
        while(turn ==2);
        C1 ← true;
    }
}

```

{ Process P1's critical section }

```

turn = 2;
C1 ← false;

```

{ P1's remaining section }

}while(1);

Process P2

do{

```

C2 ← true;
while( C1==true){
    if(turn == 1){
        C2 ← false;
        while(turn ==1);
        C2 ← true;
    }
}

```

{ P2's critical section }

```

turn = 1;
C2 ← false;

```

{ P2s remaining section }

}while(1);

- Synchronize properly for all cases

Remark

languages

- Complex when the number of processes and resources increase
- “busy waiting” before enter critical section
 - When waiting, process must check **the right to enter** the critical section => Waste processor's time

- Critical resource
- Variable lock method
- **Test and Set method**
- Semaphore mechanism
- Process synchronization example
- Monitor

P

Utilize hardware support

Principle

- Test and change the content of 1 word
- Swap the content of 2 different words
- Instruction is executed atomically
- Code block is uninterruptible when executing
 - When called at the same time, done in any order

```
boolean TestAndSet(VAR boolean target) {  
    boolean rv = target;  
    target = true;  
    return rv;  
}
```

```
void Swap(VAR boolean a, VAR boolean b) {  
    boolean temp = a;  
    a = b;  
    b = temp;  
}
```

no TestAndSet

- Sharing variable **Boolean: Lock**; resource's status;

Algorithm with TestAndSet instruction

- Free ($Lock = false$)
- Initialization: $Lock = false \Rightarrow$ Resource is free
- Algorithm for process P_i

```
do{  
    while(TestAndSet(Lock));  
    { Process P's critical section }  
    Lock = false;  
    { P's remaining section }  
}while(1);
```


- Sharing variable **Lock** shows the resource's status

Algorithm with Swap instruction

- Initialization: $Lock = false \Rightarrow$ Resource is free
- Algorithm for process P_i

do{

```
key = true;
while(key == true)
    swap(Lock, Key);
```

{ Process P's critical section }

```
Lock = false;
```

{ P's remaining section }

}while(1);

Remark

- Simple, complexity is not increase when number of processes and critical resource increase
- “busy waiting” before enter critical section
 - When waiting, process has to check if sharing resource is free or not
=> Waste processor's time
- No bounded waiting guarantee
 - The next process enters critical section is dependent on the resource release time of current resource-using process
=> Need to be solved

- Critical resource
- Variable lock method
- Test and Set method
- Semaphore mechanism
- Process synchronization example
- Monitor

Semaphore

- An **integer variable**, **initialized** by **resource sharing capability**
 - Number of **available resources** (e.g. 3 printers)
 - Number of **resource's unit** (10 empty slots in buffer)
- Can **only** be **changed** by 2 operation P and V
- Operation **P(S)** (or **wait(S)**)

```
wait(S) {  
    while(S ≤ 0) no-op;  
    S --;  
}
```

- Operation **V(S)** (or **signal(S)**)

```
signal(S) {  
    S ++;  
}
```

- P and V are **uninterruptible instructions**

Semaphore usage I

- n -process critical-section problem
 - processes share a semaphore, mutex
 - initialized to 1.
 - Each process P_i is organized as

```
do {
```

```
    (wait(mutex);
```

```
        critical section
```

```
    signal(mutex);
```

```
        remainder section
```

```
} while (1);
```

Semaphore usage II

- The order of execution inside processes:
 - P1 with a statement S1 , P2 with a statement S2.
 - require that S2 be executed only after S1 has completed

P1 and P2 share a common semaphore **synch**, initialized to 0,
Code for each process

Process 1
S1; signal (synch) ; { Remainder code }

Process 2
wait (synch) ; S2; { Remainder code }

To overcome the need for busy waiting

Use 2 operations

- `Block()` Temporarily **suspend** running process
- `Wakeup(P)` Resume **process P** **suspended** by `block()` operation

To overcome the need for busy waiting

(cont. 1)

- **Block()** Temporarily suspend running process
- When a process executes the wait operation and semaphore value is not positive
 - it must wait. (block itself -> not busy waiting)
 - **block operation** places a process into a waiting queue associated with the semaphore,
 - **Process's state** is switched to the waiting
 - Control is transferred to
 - the CPU scheduler,

To overcome the need for busy waiting

(cont.2)

- **Wakeup(P)** Resume **process P** **suspended** by block() operation
 - process that is blocked,
 - waiting on a semaphore S,
 - restarted when some other process executes a signal operation.
- The process is **restarted** by a **wakeup** operation
 - changes the process from the **waiting** state to the ready **state**.
 - process is then placed in the **ready queue**.

Semaphore implementation

Semaphore S

```
typedef struct{  
    int value;  
    struct process * Ptr;  
}Semaphore;
```

wait(S)/P(S)

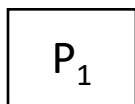
```
void wait(Semaphore S) {  
    S.value--;  
    if(S.value < 0) {  
        Insert process to S.Ptr  
        block();  
    }  
}
```

signal(S)/V(S)

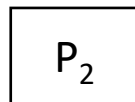
```
void signal(Semaphore S) {  
    S.value++;  
    if(S.value ≤ 0) {  
        Get process from S.Ptr  
        wakeup(P);  
    }  
}
```

Synchronization example

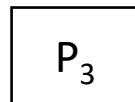
running



running

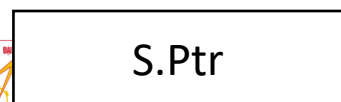
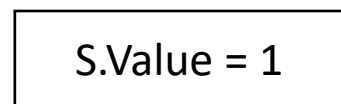


running



t

Semaphore S

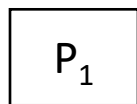


→ NULL



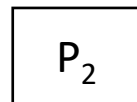
Synchronization example

running

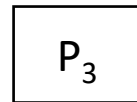


$P_1 \rightarrow P(S)$

running

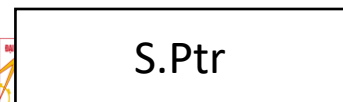
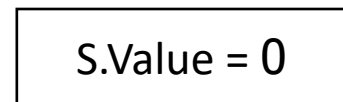


running



t

Semaphore S

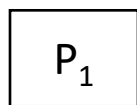


→ NULL



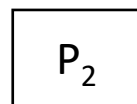
Synchronization example

running



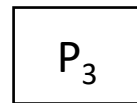
$P_1 \rightarrow P(S)$

block



$P_2 \rightarrow P(S)$

running



t

Semaphore S

S.Value = -1

S.Ptr

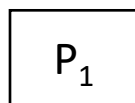


PCB2



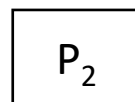
Synchronization example

running



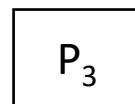
$P_1 \rightarrow P(S)$

block



$P_2 \rightarrow P(S)$

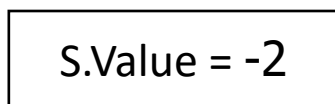
block



$P_3 \rightarrow P(S)$

t

Semaphore S



S.Ptr

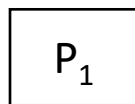
PCB2

PCB3



Synchronization example

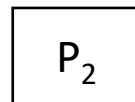
running



$P_1 \rightarrow P(S)$

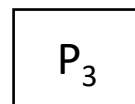
$P_1 \rightarrow V(S)$

running



$P_2 \rightarrow P(S)$

block



$P_3 \rightarrow P(S)$

t

Semaphore S

S.Value = -1

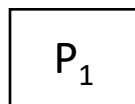
S.Ptr

PCB3



Synchronization example

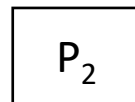
running



$P_1 \rightarrow P(S)$

$P_1 \rightarrow V(S)$

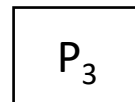
running



$P_2 \rightarrow P(S)$

$P_2 \rightarrow V(S)$

running



$P_3 \rightarrow P(S)$

t

Semaphore S

S.Value = 0

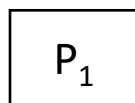
S.Ptr

NULL



Synchronization example

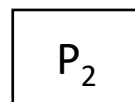
running



$P_1 \rightarrow P(S)$

$P_1 \rightarrow V(S)$

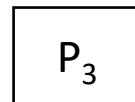
running



$P_2 \rightarrow P(S)$

$P_2 \rightarrow V(S)$

running

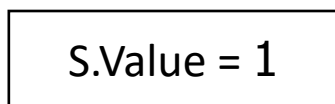


$P_3 \rightarrow P(S)$

$P_3 \rightarrow V(S)$

t

Semaphore S



→ NULL



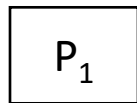
Chapter 2 Process Management

4. Critical resource and process synchronization

in the semaphore mechanism

Synchronization example

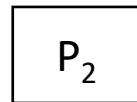
running



$P_1 \rightarrow P(S)$

$P_1 \rightarrow V(S)$

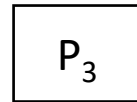
running



$P_2 \rightarrow P(S)$

$P_2 \rightarrow V(S)$

running

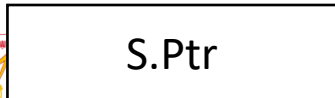
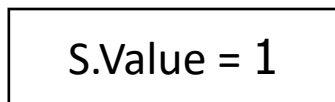


$P_3 \rightarrow P(S)$

$P_3 \rightarrow V(S)$

t

Semaphore S



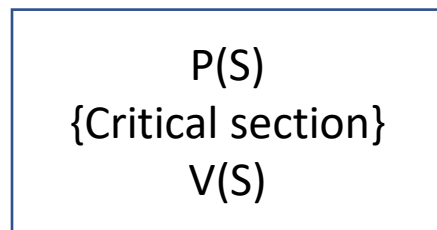
→ NULL



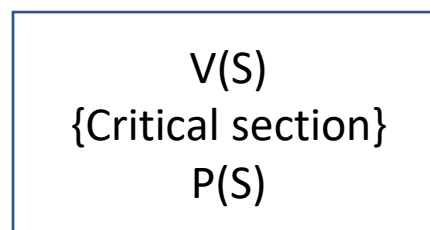
- Easy to apply for complex system

Remark

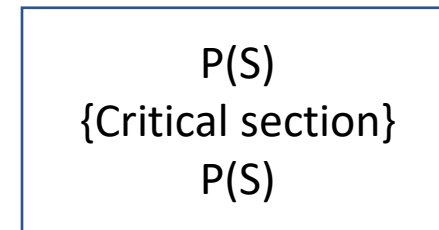
- The effectiveness is dependent on user



Correct
synchronize



Wrong order



Wrong command

- $P(S)$ and $V(S)$ is **nonshareable**

Remark

→ Need synchronization

- **Uniprocessor system**: Forbid interrupt when perform `wait()`, `signal()`
- **Multiprocessor system**
 - Not possible to forbid interrupt on other processors
 - Use variable lock method $\Rightarrow$ busy waiting, however waiting time is short (10 commands)

● `CreateSemaphore( . . . )` : Create a Semaphore

Semaphore object in WIN32 API

⇒ pointer to a `SECURITY_ATTRIBUTES` structure, handle can be inherited?

- `LONG InitialCount`, ⇒ initial count for Semaphore object
- `LONG MaximumCount`, ⇒ maximum count for Semaphore object
- `LPCTSTR lpName` ⇒ Name of Semaphore object

Example: `CreateSemaphore(NULL,0,1,NULL);`

- Return `HANDLE` of Semaphore object or `NULL`

● `WaitForSingleObject(HANDLE h, DWORD time)`

● `ReleaseSemaphore ( . . . )`

- `HANDLE hSemaphore`, ⇐ handle for a Semaphore object
- `LONG lReleaseCount`, ⇐ increase semaphore object's current count
- `LPLONG lpPreviousCount` ⇐ pointer to a variable to receive the previous count

● Example: `ReleaseSemaphore(S, 1, NULL);`

Example

```
#include <windows.h>
#include <stdio.h>
int x = 0, y = 1;
HANDLE S1, S2;
void T1();
void T2();
int main(){
    HANDLE h1, h2;
    DWORD ThreadId;
    S1 = CreateSemaphore( NULL,0, 1,NULL);
    S2 = CreateSemaphore( NULL,0, 1,NULL);
    h1 = CreateThread(NULL,0,T1, NULL,0,&ThreadId);
    h2 = CreateThread(NULL,0,T2, NULL,0,&ThreadId);
    getch();
    return 0;
}
```

Example

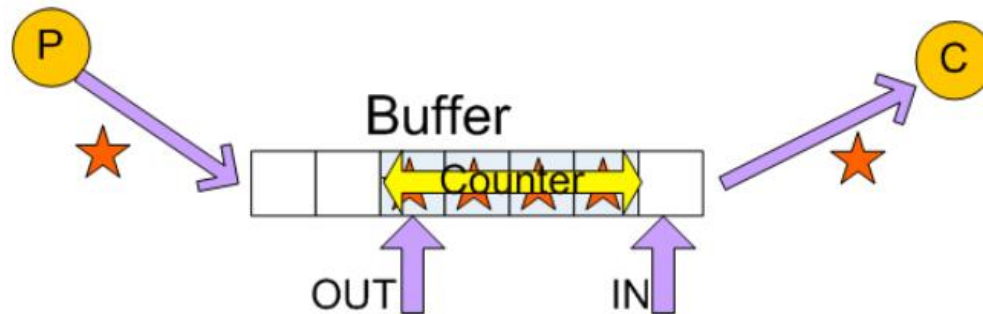
```
void T1(){
    while(1){
        WaitForSingleObject(S1,INFINITE);
        x = y + 1;
        ReleaseSemaphore(S2,1,NULL);
        printf("%4d",x);
    }
}

void T2(){
    while(1){
        y = 2;
        ReleaseSemaphore(S1,1,NULL);
        WaitForSingleObject(S2,INFINITE);
        y = 2 * y;
    }
}
```

- Critical resource
- Variable lock method
- Test and Set method
- Semaphore mechanism
- **Process synchronization examples**
- Monitor

- Producer-Consumer problem
- Dining Philosophers problem
- Readers-Writers
- Sleeping Barber
- Bathroom Problem

Producer-consumer problem



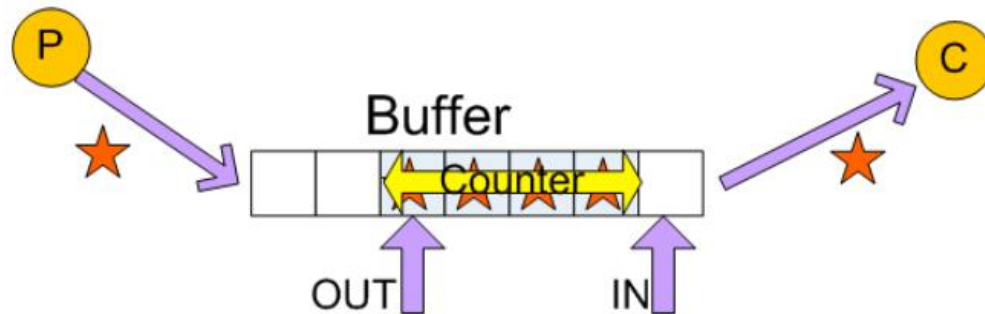
```
while(1) {  
    /*produce an item in Buffer*/  
    while (Counter == BUFFER_SIZE) ;  
        /*do nothing*/  
    Buffer[IN] = nextProduced;  
    IN = (IN + 1) % BUFFER_SIZE;  
    Counter++;  
}
```

Producer

```
while(1){  
    while(Counter == 0) ;  
        /*do nothing*/  
    nextConsumed = Buffer[OUT];  
    OUT = (OUT + 1) % BUFFER_SIZE;  
    Counter--; /*consume the item in  
        nextConsumed*/  
}
```

Consumer

Producer-Consumer problem



```

while(1) {
    /*produce an in Buffer */
    if(Counter==SIZE) block();
    /*do nothing*/
    Buffer[IN] = nextProduced;
    IN = (IN + 1) % BUFFER_SIZE;
    Counter++;
    if(Counter==1) wakeup(Consumer);
}

```

Producer

```

while(1){
    if(Counter == 0) block();
    /*do nothing*/
    nextConsumed = Buffer[OUT];
    OUT =(OUT + 1) % BUFFER_SIZE;
    Counter--;
    if(Counter==SIZE-1) wakeup(Producer);
    /*consume the item in Buffer*/
}

```

Consumer

Producer-Consumer problem

Solution: Utilize 1 semaphore Mutex to synchronize variable Counter

Initialization: Mutex = 1

```
while(1) {  
    /*produce a product in Buffer */  
    if(Counter==SIZE) block();  
    /*do nothing*/  
    Buffer[IN] = nextProduced;  
    IN = (IN + 1) % BUFFER_SIZE;  
    Counter++;  
    if(Counter==1) wakeup(Consumer);  
}
```

Producer

```
while(1){  
    if(Counter == 0) block();  
    /*do nothing*/  
    nextConsumed = Buffer[OUT];  
    OUT =(OUT + 1) % BUFFER_SIZE;  
    Counter--;  
    if(Counter==SIZE-1) wakeup(Producer);  
    /*consume the item in Buffer*/  
}
```

Consumer

Problem: Assume Counter=0

- **Consumer** check counter => call block()
- **Producer** increase counter by 1 and call wakeup(Consumer)
- **Consumer** not blocked yet => wakeup() is skipped



Chapter 2 Process Management

4. Critical resource and process synchronization

Producer-Consumer problem

Solution 2: Utilize 2 semaphore **full**, **empty** .

Initialization: **full** $\leftarrow$ 0 : Number of item in buffer

empty $\leftarrow$ BUFFER_SIZE: Number of empty slot in buffer

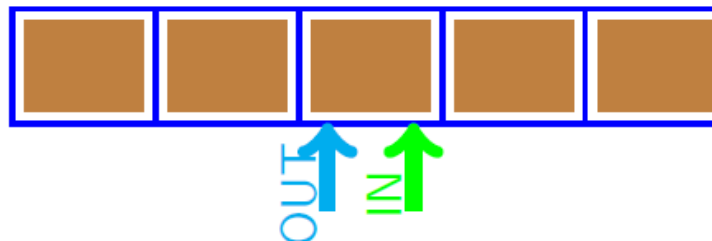
```
do{
  {Create new product}
  wait(empty);
  {Put new product into Buffer}
  signal(full);
} while (1);
```

Producer

```
do{
  wait(full);
  {Take out 1 product from Buffer}
  signal(empty);
  {Consume product}
} while (1);
```

Consumer

Consumer
Running
Producer
Running



empty = 0

full = 5

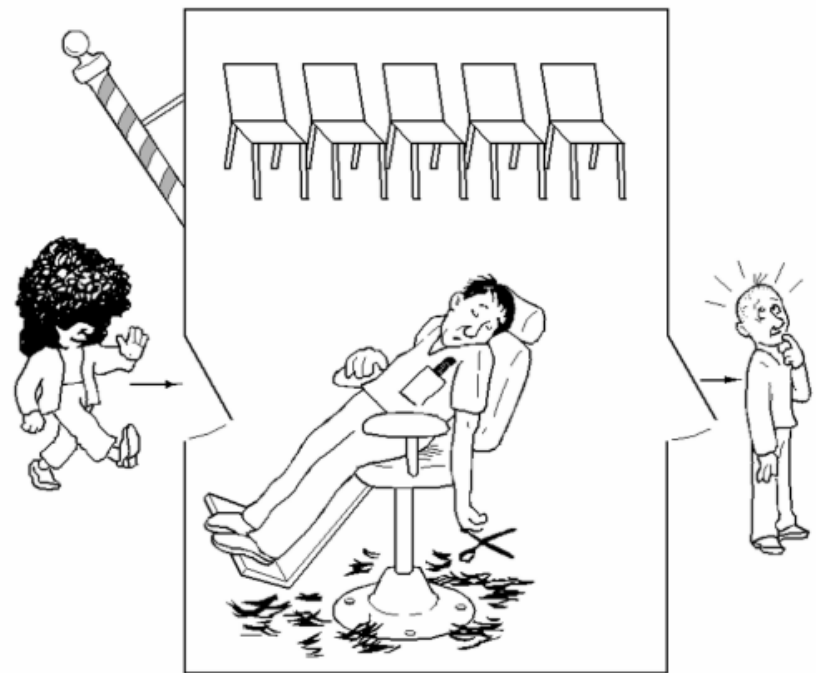


Readers and Writers problem

- Many **Reader** processes access the database at the same time
- Several **Writer** processes update the database
- Allow unlimited **Readers** to access the database
 - 1 **Reader** process is accessing the database, new **Reader** process can access the database
 - (**Writers** have to stay in waiting queue)
- Allow only 1 **Writer** process to update the database at a time
- Non-preemptive problem. Process stays inside critical section without being interrupted

Sleeping barber

- N waiting chair for client
- Barber can cut for one client at a time
- No client, barber go to sleep
- When client comes
 - If barber is sleeping $\Rightarrow$ wake him up
 - If barber is working
 - Empty chair exists $\Rightarrow$ sit and wait
 - No empty chair left $\Rightarrow$ Go away



Bathroom Problem

- A bathroom is to be used by both men and women, but not at the same time
 - If the bathroom is empty, then anyone can enter
 - If the bathroom is occupied, then only a person of the same sex as the occupant(s) may enter
 - The number of people that may be in the bathroom at the same time is limited
-
- Problem implementation require to satisfy constraints
 - 2 types of process: male() và female()
 - Each process enter the Bathroom in a random period of time

Dining philosopher problem

Classical synchronization problem, show the situation where many processes share resources

- 5 philosophers having dinner at a round table
- In front each person is a disk of spaghetti
- Between two disk is a fork
- Philosopher do 2 things :Eat and Think
- Each person need two forks to eat
- Take only one fork at a time
- Take the left fork then the right fork
- Finish eating, return the fork to original place



Dining philosopher problem: Simple method

- Each fork is a critical resource, synchronized by a semaphore `fork[i]`
- Semaphore `fork[5] = {1, 1, 1, 1, 1};`
- Algorithm for philosopher P_i

```
do{  
    wait(fork[i])  
    wait(fork[(i+1)% 5]);  
    {Eat}  
    signal(fork[(i+1)% 5]);  
    signal(fork[i]);  
    {Thinks}  
} while (1);
```

- If all the philosophers want to eat
 - Take the left fork (call to: `wait(fork[i])`)
 - Wait for the right fork (call to: `wait(fork[(i+1)%5])`)

⇒ deadlock



Dining philosopher problem – Solution 1

- Allow only one philosopher to take the fork at a time
- Semaphore mutex $\leftarrow 1$;
- Algorithm for philosopher P_i

```
do{
    wait(mutex)
    wait(fork[i])
    wait(fork[(i+1)% 5]);
    signal(mutex)
    {Eat}
    signal(fork[(i+1)% 5]);
    signal(i);
    {Thinks}
} while (1);
```

- It's possible to allow 2 non-close philosopher to eat at a time (P1: eats, P2: owns mutex $\Rightarrow$ P3 waits)

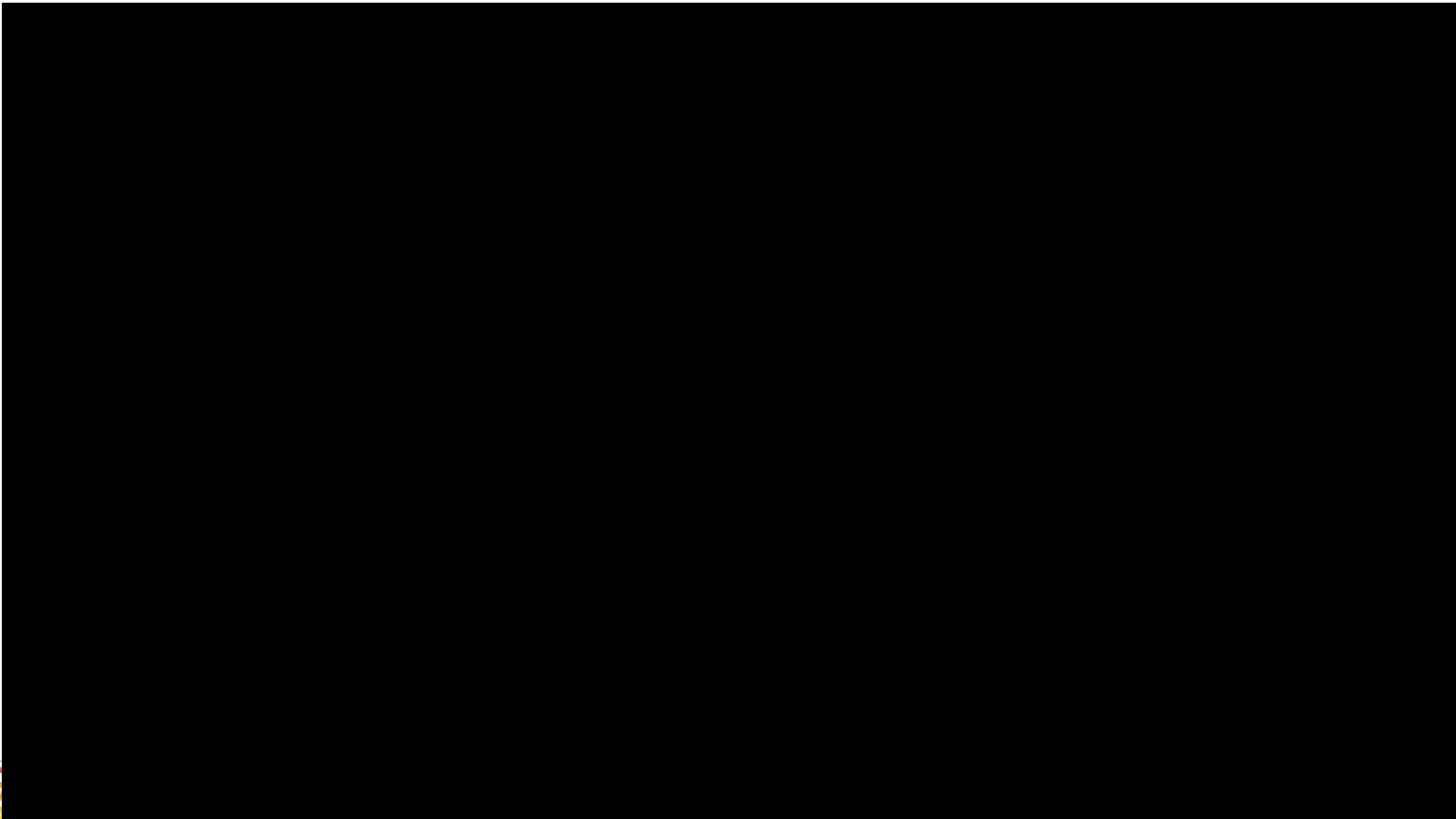
Dining philosopher problem – Solution 1

- Philosopher take the forks with different order
- Even id philosopher take the even id fork first
- Odd id philosopher take the odd id fork first

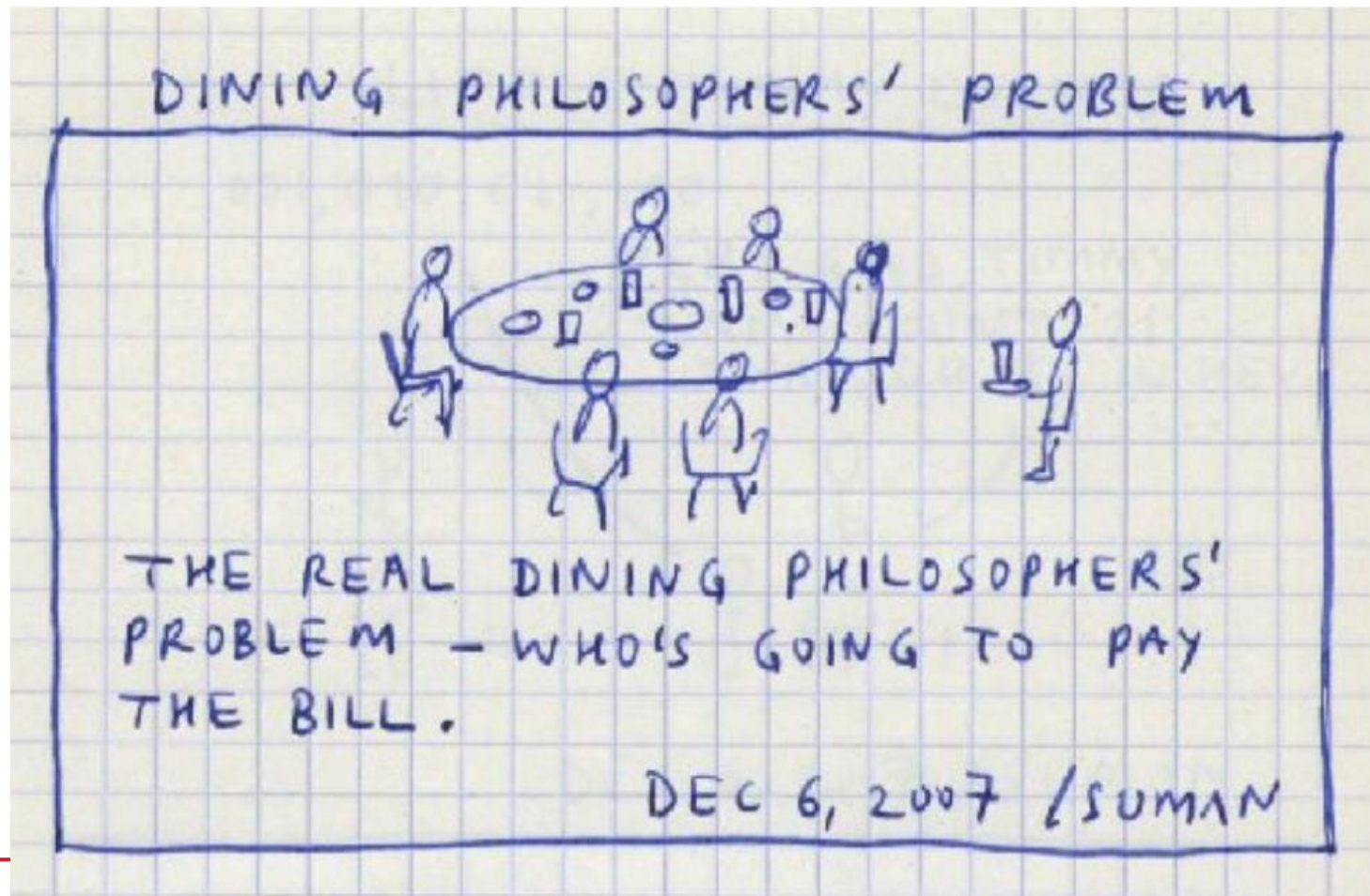
```
do{
    j = i%2
    wait(fork[(i + j)%5])
    wait(fork[(i+1 - j)% 5]);
    {Eat}
    signal(fork[(i+1 - j)% 5]);
    signal((i + i)%5);
    {Thinks}
} while (1);
```

- Solve the deadlock problem

Demonstration



True problem ?



- Critical resource
- Variable lock method
- Test and Set method
- Semaphore mechanism
- Process synchronization examples
- **Monitor**

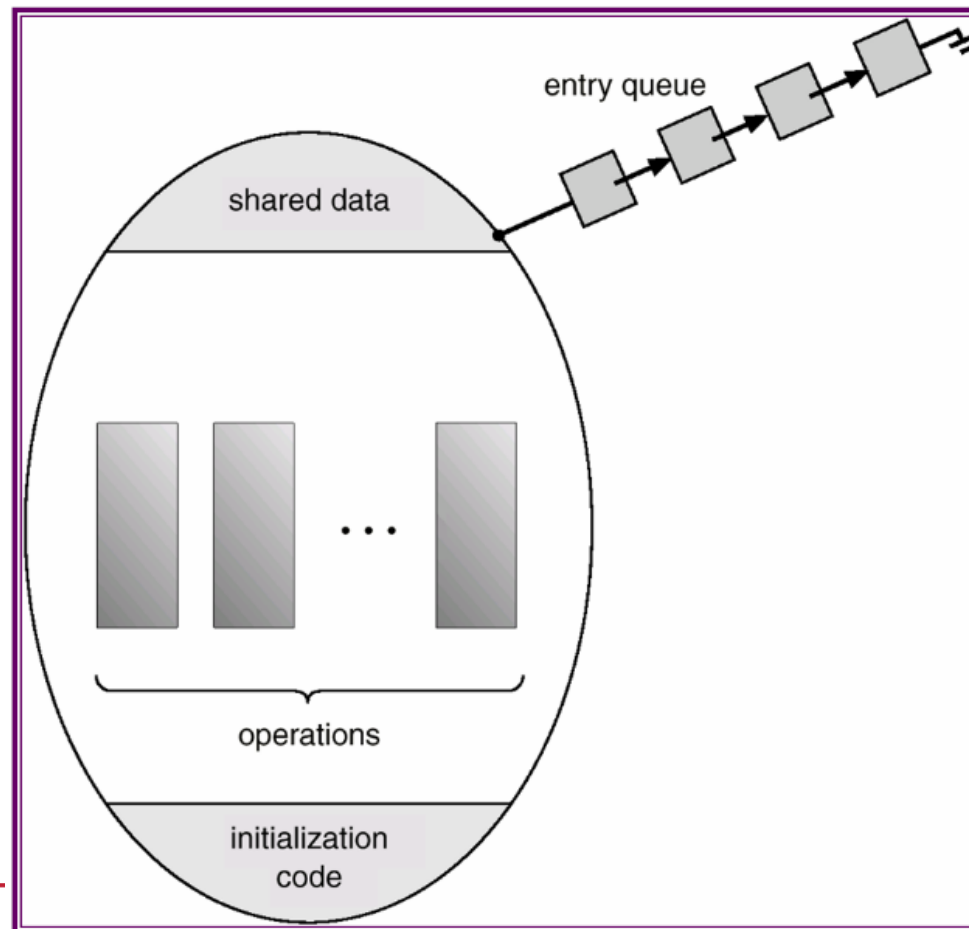
Introduction

- Special data type, proposed by HOARE 1974
- Combines of procedures, local data, initialization code
- Process can only access variables via procedures of Monitor
- Only one process can work with Monitor at a time
 - Other processes have to wait
- Allow process to wait inside Monitor
 - Utilize **condition variable**

```
monitor monitorName{  
    Sharing variables declarations ;  
    procedure P1(...){  
        ...  
    }  
    ...  
    procedure Pn(...){  
        ...  
    }  
  
    {  
        Initialization code  
    }  
};
```

Monitor's syntax

Model



Condition Variable

Actually, **name** of a **queue**

Declare: **condition x,y;**

Only used by 2 operations

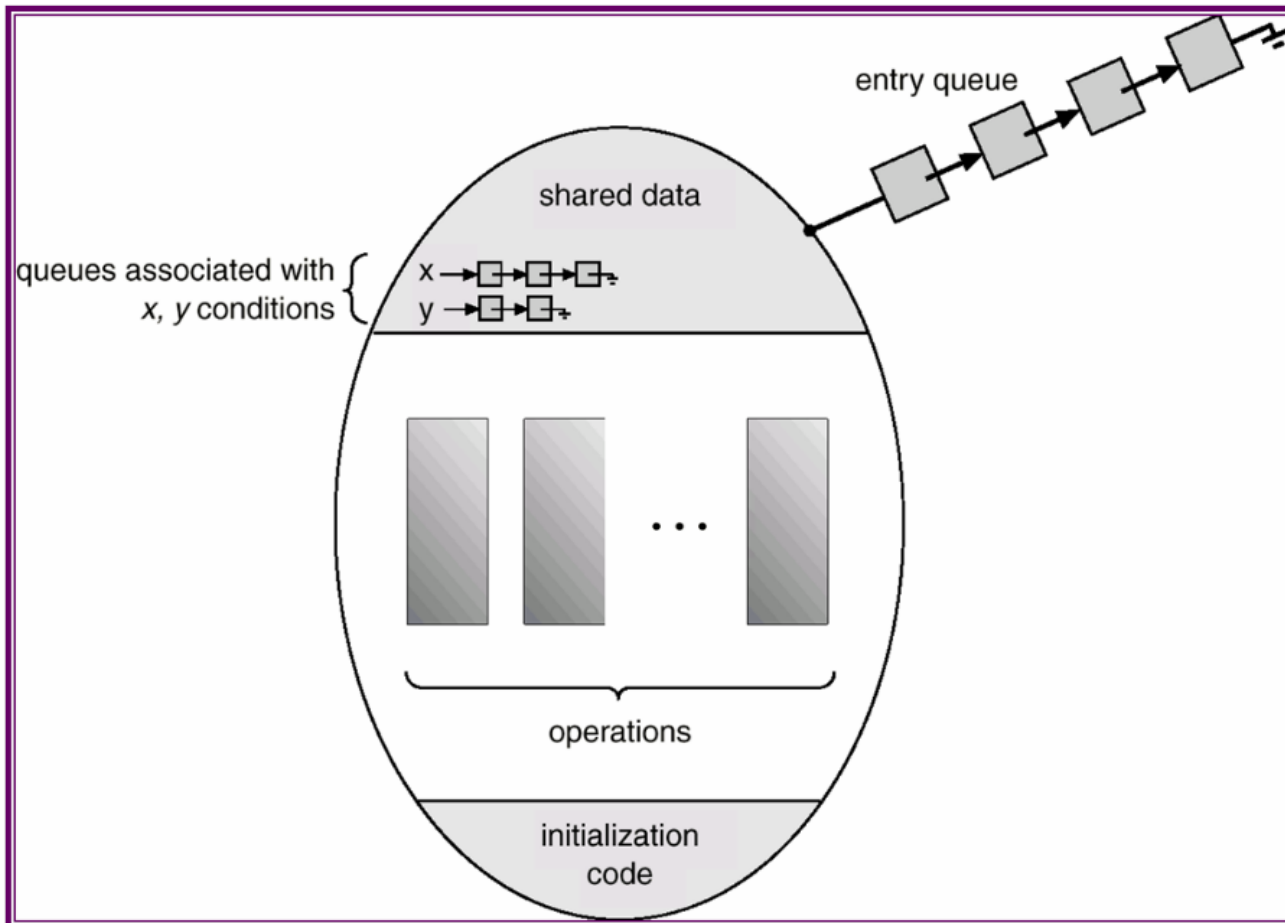
wait() Called by Monitor's procedures (**syntax x.wait() or wait(x)**).

Allow process to be **blocked until activated by other process**
via **signal()** procedure

signal() Called by Monitor's procedures (**syntax x.signal() or signal(x)**). **Activate** a process waiting at x variable queue.

If no waiting process then the operation is skipped

Model



Monitor's usage: sharing a resource

```
Monitor Resource{
    Condition Nonbusy;
    Boolean Busy
//-- 1      User's part      ; --
    void Acquire(){
        if(busy) Nonbusy.wait();
        busy=true;
    }
    void Release(){
        busy=false
        signal(Nonbusy)
    }
//--- Initialization part  ----
    busy= false;
    Nonbusy = Empty;
}
```

Process's structure

```
while(1){
    ...
    Resource.Acquire()
    {      Using resource      }
    Resource.Release()
    ...
}
```

Producer – Consumer problem

```
Monitor ProducerConsumer{
    Condition Full, Empty;
    int Counter ;
    void Put(Item){
        if(Counter=N) Full.wait();
        { Put Item into Buffer };
        Counter++;
        if(Counter=1)Empty.signal()
    }
    void Get(Item){
        if(Counter=0) Empty.wait()
        {Take Item from Buffer}
        Counter--;
        if(Counter=N-1)Full.signal()
    }
    Counter=0;
    Full, Empty = Empty;
}
```

ProducerConsumer M;

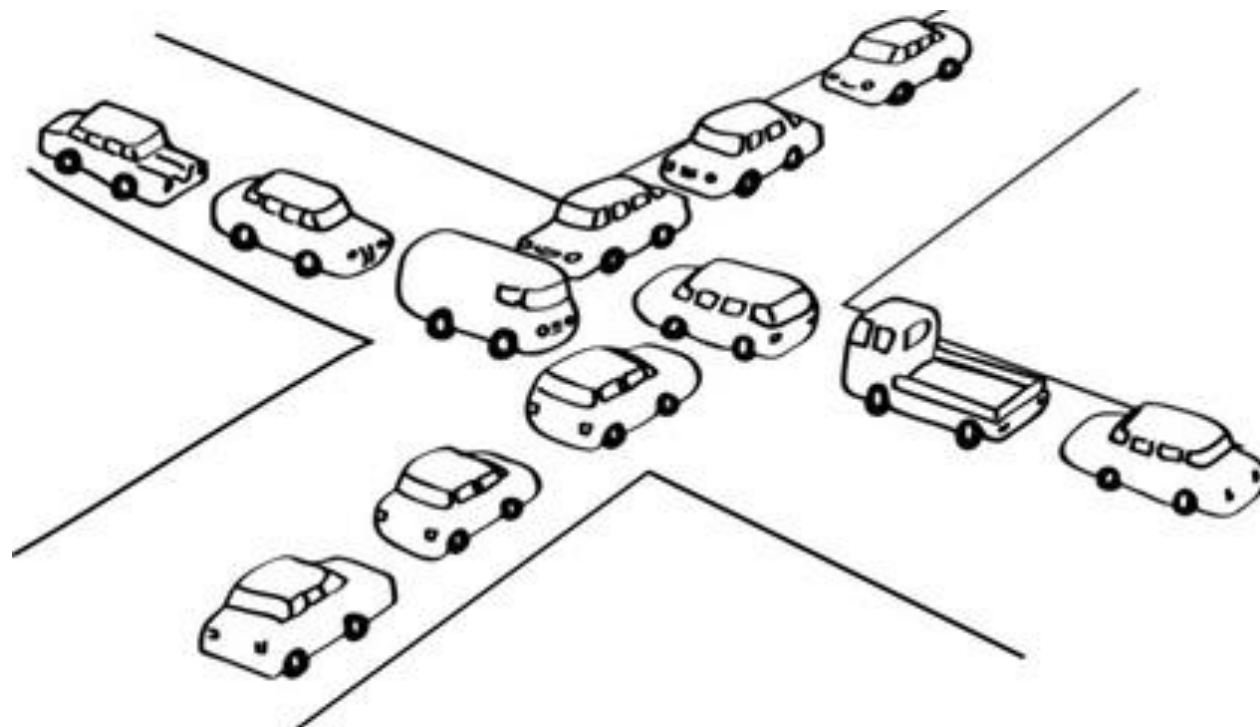
Producer

```
while(1){
    Item = New product
    M.Put(Item)
    ...
}
```

Consumer

```
while(1){
    M.Get(&Item)
    {Use Item}
    ...
}
```

- ① Process
- ② Thread
- ③ CPU scheduling
- ④ Critical resource and process synchronization
- ⑤ **Deadlock and solutions**





- Deadlock conception
- Conditions for resource deadlocks
- Solutions for deadlock
- Deadlock prevention
- Deadlock avoidance
- Deadlock detection and recovery

Deadlock conception

- System combines of **concurrently running** processes, **sharing resources**
 - **Resources** have **different types** (e.g.: CPU, memory,...).
 - Each type of resource may has **many unit** (e.g.: 2 CPUs, 5 printers..)
- Each process is combines of sequences of continuous operations
 - **Require resource**: if resource is not available (being used by other processes) $\Rightarrow$ process has to wait
 - **Utilize resource** as required (printing, input data...)
 - **Release** allocated resources
- When processes share **at least 2 resources**, system may “in danger”

Chapter 2 Process Management

5. Dead lock and solutions

Deadlock conception

- Example: Two processes in the system P_1 & P_2
 - P_1 & P_2 share 2 resources R_1 & R_2
 - R_1 is synchronized by semaphore S_1 ($S_1 \leftarrow 1$)
 - R_2 is synchronized by semaphore S_2 ($S_2 \leftarrow 1$)
 - Code for P_1 and P_2

```
P(S1)  
P(S2)  
{ Use R1&R2 }  
V(S1)  
V(S2)
```

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2



$S1 = 1$



$S2 = 1$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2

$P(S_1)$ $\longleftrightarrow$



$S1 = 0$



$S2 = 1$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2

$P(S_1)$

$P(S_1)$

P_2 block()
Enter queue for R_1



$S1 = -1$



$S2 = 1$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2

$P(S_1)$

$P(S_2)$

$P(S_1)$

P_2 block()
Enter queue for R_1



$S1 = -1$



$S2 = 0$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P1

$P(S_1) \longleftrightarrow$

$P(S_2) \longleftrightarrow$

Use $R_1 \& R_2 \longleftrightarrow$

$V(S_1) \longleftrightarrow$

wakeup(P_2)

Process P2

$\longleftrightarrow P(S_1)$



$S1 = 0$



$S2 = 0$

t

Chapter 2 Process Management

5. Dead lock and solutions

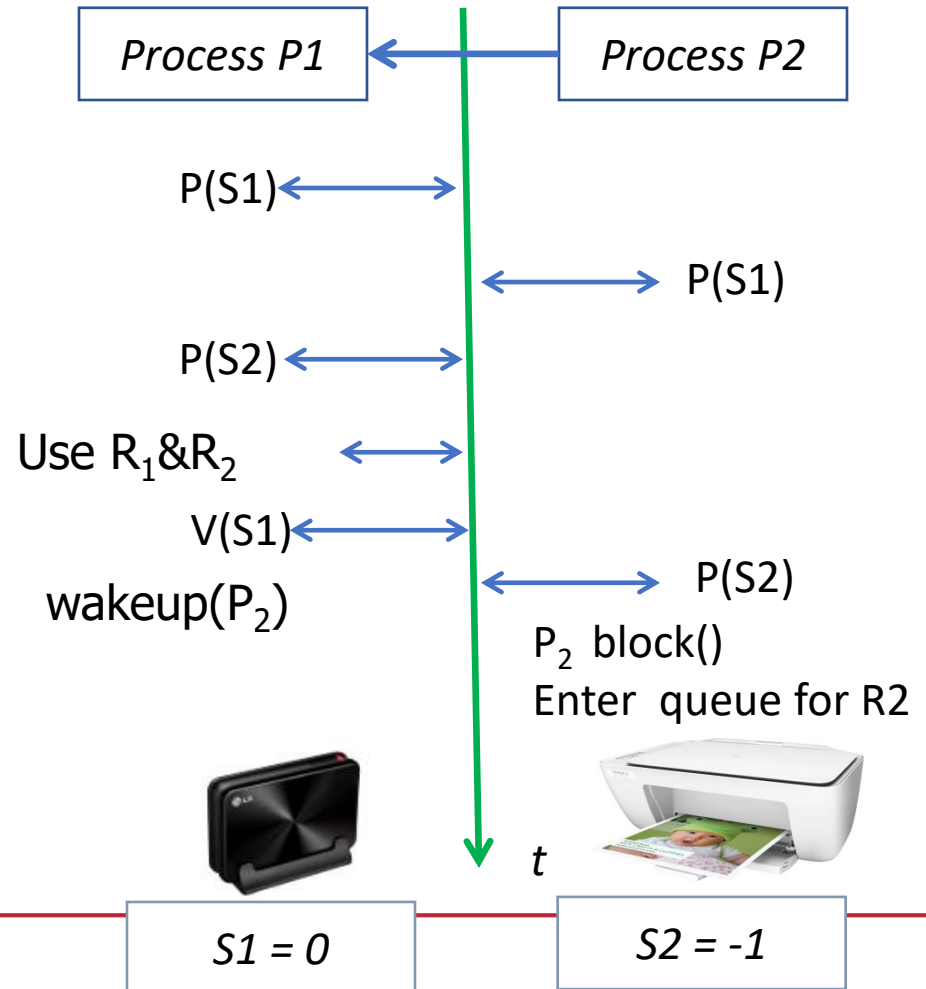
Example

Process P1

P(S₁)
P(S₂)
{ Use R₁&R₂ }
V(S₁)
V(S₂)

Process P2

P(S₁)
P(S₂)
{ Use R₁&R₂ }
V(S₁)
V(S₂)



Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P1

$P(S_1)$

$P(S_2)$

Use $R_1 \& R_2$

$V(S_1)$

$V(S_1)$
wakeup(P_2)

Process P2

$P(S_1)$

$P(S_2)$



$S_1 = 0$



$S_2 = 0$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P1

$P(S_1)$

$P(S_2)$

Use $R_1 \& R_2$

$V(S_1)$

$V(S_2)$
wakeup(P_2)



$S_1 = 0$

Process P2

$P(S_1)$

$P(S_2)$

Use $R_1 \& R_2$



$S_2 = 0$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_2)$
 $P(S_1)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2



$S1 = 1$



$S2 = 1$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_2)$
 $P(S_1)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2

$P(S_1)$ $\longleftrightarrow$



$S1 = 0$



$S2 = 1$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_2)$
 $P(S_1)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

$P(S_1)$ $\longleftrightarrow$

Process P2

$\longleftrightarrow$ $P(S_2)$



$S1 = 0$



$S2 = 0$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_2)$
 $P(S_1)$
{ Use $R_1 \& R_2$ }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2

$P(S_1) \longleftrightarrow$

$\longleftrightarrow P(S_2)$

$P(S_2) \longleftrightarrow$

P_1 block()
Enter queue for R_2



$S1 = 0$



$S2 = -1$

t

Chapter 2 Process Management

5. Dead lock and solutions

Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_2)$
 $P(S_1)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P1

Process P2

$P(S_1)$

$P(S_2)$

$P(S_2)$

P_1 block()
Enter queue for R_2

$P(S_1)$

P_2 block()
Enter queue for R_1



$S1 = -1$



$S2 = -1$

t

Chapter 2 Process Management

5. Dead lock and solutions

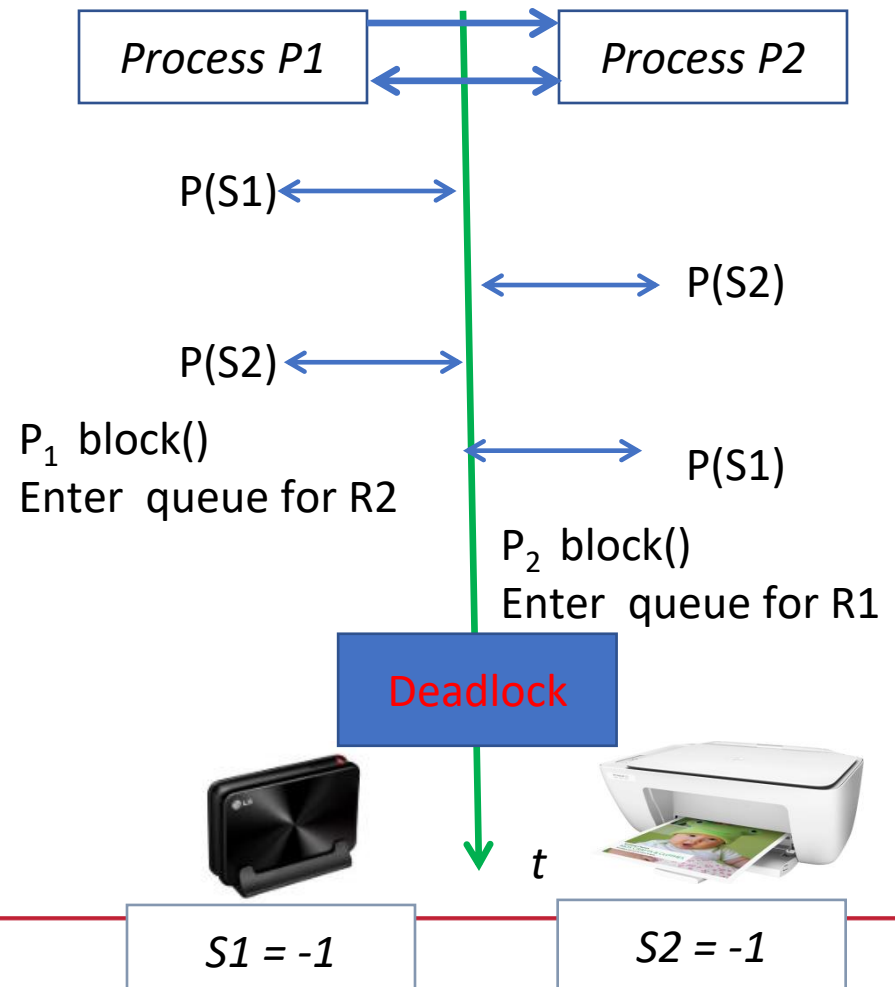
Example

Process P1

$P(S_1)$
 $P(S_2)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$

Process P2

$P(S_2)$
 $P(S_1)$
{ Use R_1 & R_2 }
 $V(S_1)$
 $V(S_2)$



Definition

A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause

- Deadlock conception
- **Conditions for resource deadlocks**
- Solutions for deadlock
- Deadlock prevention
- Deadlock avoidance
- Deadlock detection and recovery

Conditions

4 conditions, must occur at the same time

- **Critical resource**
 - Resource is used in a non-shareable model
 - Only 1 process can use resource at a time
 - Other process request to use resource $\Rightarrow$ request must be postponed until resource is released
- **Wait before enter the critical section**
 - Process can not enter critical section has to wait in queue
 - Still own resources while waiting
- **No resource reallocation system**
 - Resource is non-preemptive
 - Resource is released only by currently using process after this process finished its task
- **Circular waiting**
 - Set of processes $\{P_0, P_2, \dots, P_n\}$ waiting in a order: $P_0 \rightarrow R_1 \rightarrow P_1; P_1 \rightarrow R_2 \rightarrow P_2; \dots P_{n-1} \rightarrow R_n \rightarrow P_n; P_n \rightarrow R_0 \rightarrow P_0$
 - Circular waiting create nonstop loop

Example: Dining philosopher problem

- Critical resource
- Wait before enter critical section
- Non-preemptive resource
- Circular waiting



- Deadlock conception
- Conditions for resource deadlocks
- **Solutions for deadlock**
- Deadlock prevention
- Deadlock avoidance
- Deadlock detection and recovery

Methods

- Prevention
- Avoidance
- Detection and recovery

Methods

- Prevention

- Apply methods to guarantee that the system **never** has deadlock
- Expensive
- Apply for system that deadlock happens **frequently** and once it happen the **cost is high**

- Avoidance

- Check each process's resource request and reject request if this request may lead to deadlock
- Require extra information
- Apply for system that deadlock happens **least frequently** and once it happen the **cost is high**



Methods

- Deadlock detection and recovery
 - Allow the system work normally $\Rightarrow$ deadlock may happen
 - Periodically check if deadlock is happening
 - If deadlock apply methods to remove deadlock
 - Apply for system that deadlock happens least frequently and once it happen the cost is low

- Deadlock conception
- Conditions for resource deadlocks
- Solutions for deadlock
- **Deadlock prevention**
- Deadlock avoidance
- Deadlock detection and recovery

Rule

Attack 1 of 4 required conditions for deadlock to appear

Critical resource

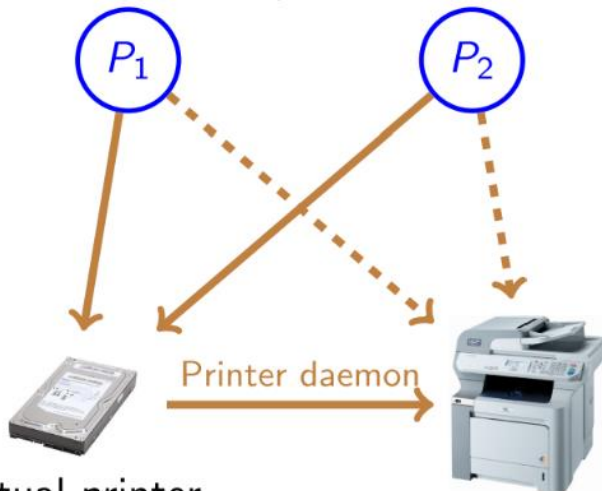
Wait before entering critical section

Non-preemptive resource

Circular wait

Critical resource condition

- Reduce the system's critical degree
 - Shareable resource(read-only file): accessed simultaneously
 - Non-shareable resource: **Cannot** be accessed simultaneously
- SPOOL mechanism(*Simultaneous peripheral operation on-line*)
 - Do not allocate resource when it's not necessary
 - A limited number of processes can request resource



- Only process *printer daemon* works with printer $\Rightarrow$ Deadlock for resource printer is canceled
- Not every resource can be used with SPOOL

Wait before entering critical section condition

Rule: Make sure 1 process **request** resource **only** when it **doesn't own any other resources**

- **Prior allocate**
 - processes request **all** their resources **before starting execution** and only **run** when required resources are **allocated**
 - **Effectiveness** of resource utilization is **low**
 - Process only use resource at the last cycle?
 - Total requested resource higher than the system's capability?
- **Resource release**
 - Process **releases all** resource **before apply(re-apply)** new resource
 - Process **execution's speed** is **low**
 - Must guarantee that **data** kept in temporary release resource **won't be lost**



Wait before entering critical section condition - Example

- Process combines of 2 **phases**
 - Copy data from **tape** to a file in the **disk**
 - Arrange the data in **file** and bring to **printer**



- **Prior allocation method**
 - Request both tape, file and printer
 - **Waste** printer in first phase, tape in the second phase

Wait before entering critical section condition - Example

- Process combines of 2 **phases**
 - Copy data from **tape** to a file in the **disk**
 - Arrange the data in **file** and bring to **printer**



- **Prior allocation method**
 - Request both tape, file and printer
 - **Waste** printer in first phase, tape in the second phase
- **Resource release method**
 - Request tape and file for phase 1
 - Release tape and file
 - Request file and printer for phase 2

No preemption resource condition

Rule: allow process to preempt resource when it's necessary

- Process P_i apply for resource R_j
 - R_j is available: allocate R_j to P_i
 - R_j not available: (R_j is being used by process P_k)
 - P_k is waiting for another resource
 - Preempt R_j from P_k and allocate to P_i as requested
 - Add R_j into the list of needing resource of P_k
 - P_k is execution again when
 - Receive the needing resource
 - Take back resource R_j
 - P_k is running
 - P_i must wait (*no resource release*)
 - Allow resource preempt only when it's necessary

No preemption resource condition

Rule: allow preemptive when it's necessary

- Only applied for resources that can be store and recover easily
 - (*CPU, memory space*)
 - Difficult to apply for resource like *printer*
- 1 process is preempted many time ?

Circular wait condition

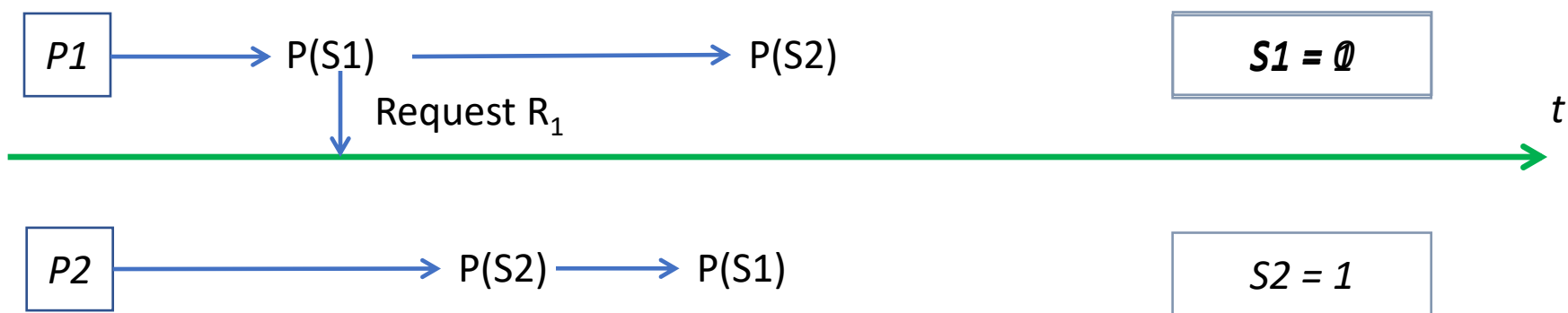
- Provide a global numbering of all type of resources
 - $R = \{R_1, R_2, \dots, R_n\}$ Set of resources
 - Construct an ordering function $f: R \rightarrow N$
 - Function f is constructed based on the order of resource utilization
 - $f(\text{Tape}) = 1$
 - $f(\text{Disk}) = 5$
 - $f(\text{Printer}) = 12$
- Process can only request resource in an increasing order
 - holding resource type R_k can only request resource type R_j satisfy $f(R_j) > f(R_k)$
 - Process requests resource R_k has to release all resource R_i satisfy condition $f(R_i) \geq f(R_k)$

Circular wait condition

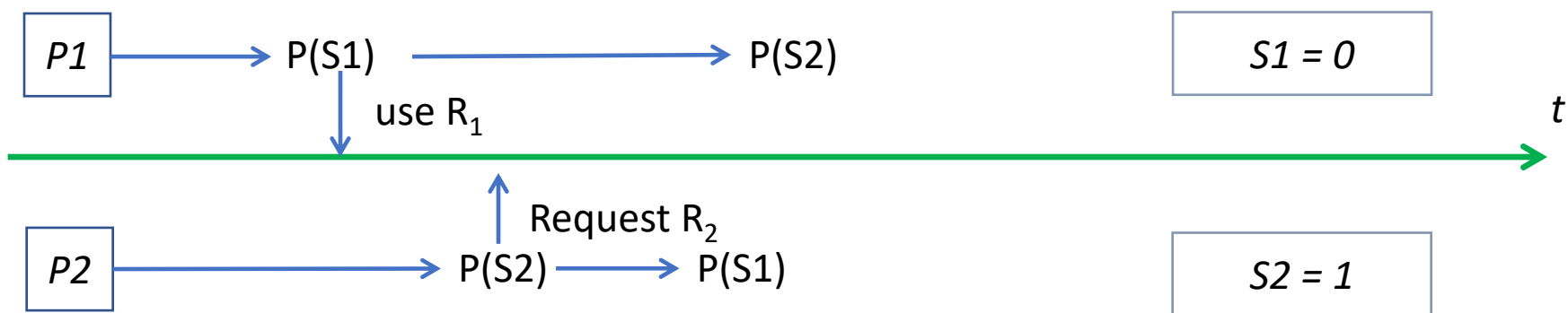
- Process can only request resource in an **increasing order**
 - Process holding resource type R_k can only request resource type R_j satisfy $f(R_j) > f(R_k)$
 - Process requests resource R_k has to release all resource R_i satisfy condition $f(R_i) \geq f(R_k)$
- Prove
 - Suppose deadlock happen between processes $\{P_1, P_2, \dots, P_m\}$
 - $R_1 \rightarrow P_1 \rightarrow R_2 \rightarrow P_2 \Rightarrow f(R_1) < f(R_2)$
 - $R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \Rightarrow f(R_2) < f(R_3) \dots$
 - $R_m \rightarrow P_m \rightarrow R_1 \rightarrow P_1 \Rightarrow f(R_m) < f(R_1)$
 - $f(R_1) < f(R_2) < \dots < f(R_m) < f(R_1) \Rightarrow$ **invalid**

- Deadlock conception
- Conditions for resource deadlocks
- Solutions for deadlock
- Deadlock prevention
- **Deadlock avoidance**
- Deadlock detection and recovery

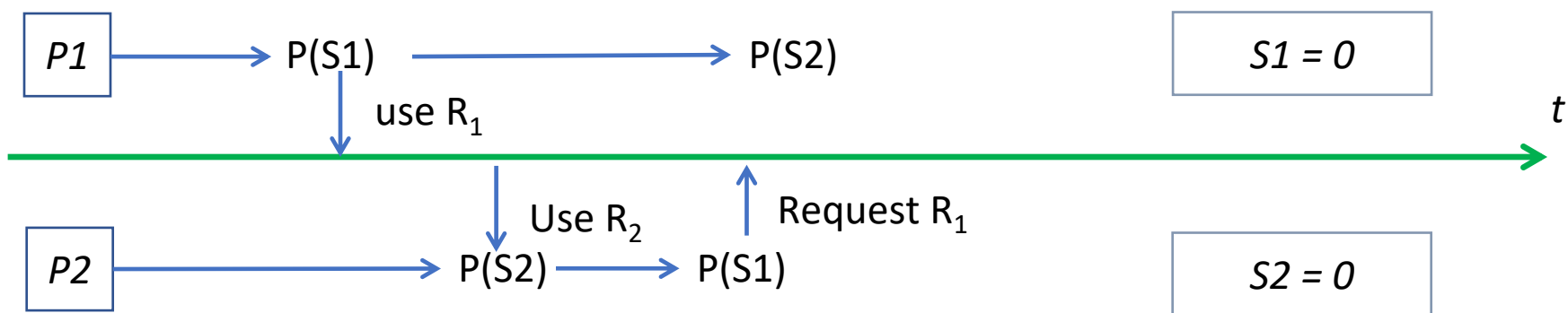
Example



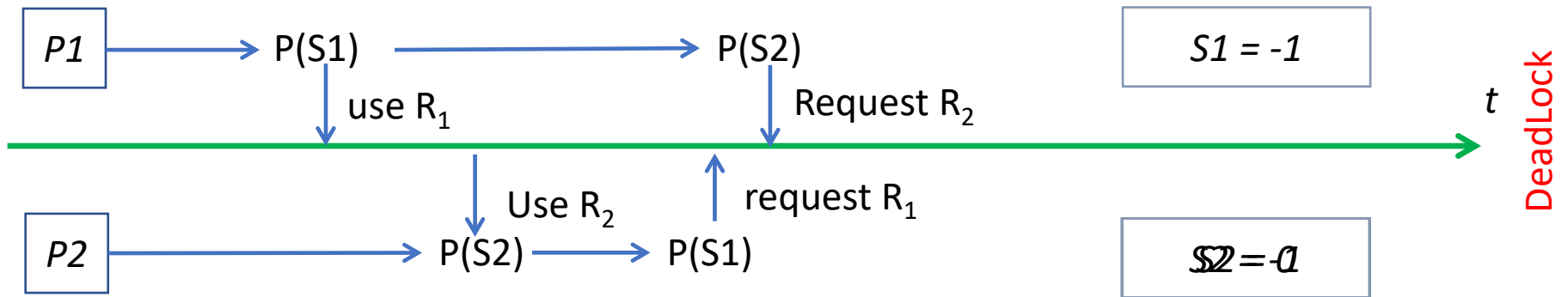
Example



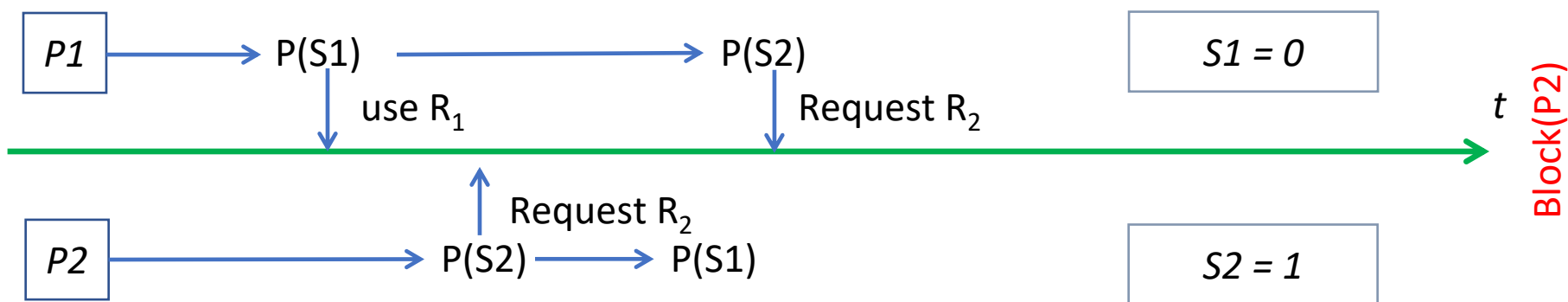
Example



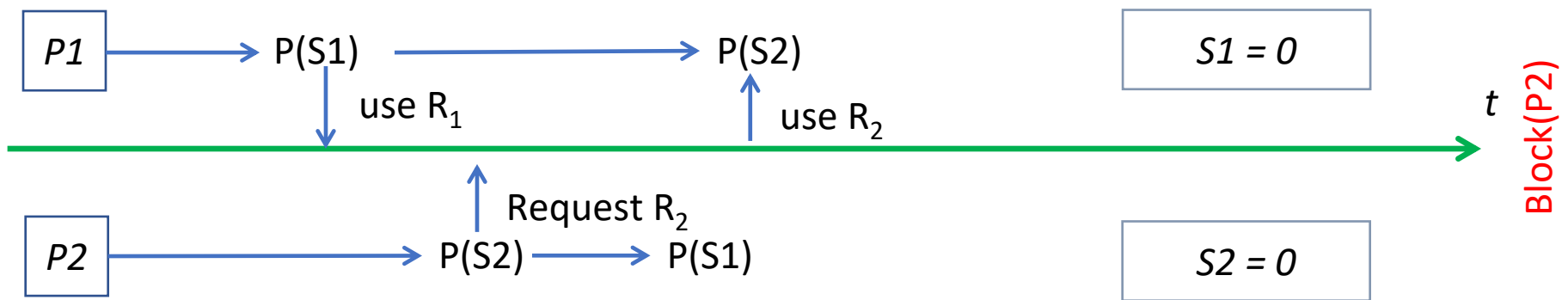
Example



Example



Example



Conclude:

If processes' orders of request/release resources are known in advance, the system could make a resource allocation decision (accept or block) for all request to let deadlock not occur.

Rule

- Must **know in advance** information of processes and resources
 - Process has to declare the **maximum amount** of **each resources type** that is **required for execution**
- **Decision** is made based on the result of **Resource-Allocation State check**
 - Resource allocation state is defined by following parameters
 - Number of resource unit **available** in the system
 - Number of resource unit **allocated** for each process
 - Number of **maximum** resource unit each process may **request**
 - If system is **safe**, request is **accepted**
- Perform checking every time a resource request is received
 - Objective: Guarantee the system's always in safe state
 - At the beginning (resource is not allocated), system is safe
 - Only allocate resource when safety is guaranteed
 - ⇒ System changes from current safety state to another safety state

Safe state

The system's state is safe when

- It's possible to provide resource for each process (to maximum requirement) follow an order such that deadlock will not occur
- A **safe sequence** of all processes is existed

Safe sequence

Set of process $P = \{P_1, P_2, \dots, P_n\}$ is safe if

- For each process P_i , each resource request in future is accepted due to
 - The amount of available resource in the system
 - Resource is currently occupied by process $P_j (j < i)$

In safe sequence, when P_i request for resource

- If not accept immediately, P_i wait until P_j terminates ($j < i$)
- When P_j terminate and release resource, P_i receives required resource, executes, releases allocated resource and terminate
- When P_j terminate and release resource $\Rightarrow P_{i+1}$ will receive resource and able to finish. . .
- All the processes in the safe sequence is able to finish

Example

Consider a system includes

- 3 processes P_1, P_2, P_3 and one resource R has 12 units
- (P_1, P_2, P_3) may request maximum (10, 4, 9) unit of resource R
- At time t_0 , (P_1, P_2, P_3) allocated (5, 2, 2) unit of resource R
- At current time (t_0), is the system safe?
 - System allocate (5 + 2 + 2) units $\Rightarrow$ 3 units remain
 - (P_1, P_2, P_3) may request (5, 2, 7) units
 - With current 3 units, all request of P_2 is acceptable $\Rightarrow P_2$ guaranteed to finish and will release 2 unit of R after finished
 - With 3 + 2 units, P_1 guaranteed to finish and will release 5 units
 - With 3 + 2 + 5 unit P_3 guaranteed to finish
- At time t_0 , P_1, P_2, P_3 are guaranteed to finish $\Rightarrow$ system is safe with safe sequence (P_2, P_1, P_3)

Example

Consider a system includes

- 3 processes P_1, P_2, P_3 and one resource R has 12 units
- (P_1, P_2, P_3) may request maximum (10, 4, 9) unit of resource R
- At time t_0 , (P_1, P_2, P_3) allocated (5, 2, 2) unit of resource R
- A time t_1 , P_3 request and allocated 1 resource unit. Is the system safe?
 - With current 2 units, all request of P_2 is acceptable $\Rightarrow P_2$ guaranteed to finish and will release 2 unit of R after finished
 - When P_2 finished, the amount of available resource in the system is 4
 - With 4 resource unit, P_1 and P_3 both have to wait when apply for 5 more resource unit
 - Hence, system is not safe with (P_1, P_3)
- Remark: At time t_1 , if P_3 has to wait when request for 1 more resource unit, deadlock will be removed

Example

- Conclude
 - System is safe $\Rightarrow$ Processes are able to finish
 $\Rightarrow$ no deadlock
 - System is not safe $\Rightarrow$ Deadlock may occur
- Method
 - Verify all resource request
 - If the system is still safe after allocate resource $\Rightarrow$ allocate
 - If not $\Rightarrow$ process has to wait
 - The banker algorithm

The banker algorithm: Introduction

- Good for systems have resources with many units
- New appearing process has to declare the maximum unit of each resource type
 - Not larger than the total amount of the system
- When one process request for resource, system verify if it's safe to accept this requirement
 - If system still safe $\Rightarrow$ Allocate resource for process
 - Not safe $\Rightarrow$ wait

Algorithm's data I

System

n number of process in the system

m number of resource type in the system

Data structures

Available Vector with length m represents the number of available resource in the system. (**Available[3] = 8 $\Rightarrow$?**)

Max Matrix $n * m$ represents each process maximums request for each type of resource. (**Max[2,3] = 5 $\Rightarrow$?**)

Allocation Matrix $n * m$ represents amount of resource allocated for processes (**Allocation[2,3] = 2 $\Rightarrow$?**)

Need Matrix $n * m$ represents amount of resource is needed for each process **Need[2,3] = 3 $\Rightarrow$?**)

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$



Algorithm's data I

Rule

- X, Y are vectors with length n
 - $X \leq Y \Leftrightarrow X[i] \leq Y[i] \forall i = 1, 2, \dots, n$
- Each row of *Max, Need, Allocation* is processed like vector
- Algorithm calculated on vectors
- Local structures
 - Work** vector with length m represents how much each resource still available
 - Finish** vector with Boolean type, length n represents if a process is guaranteed to finish or not

Algorithm for safety checking

```
BOOL Safe(Current Resource-Allocation State){  
    Work ← Available  
    for (i : 1 → n) Finish[i] ← false  
    flag ← true  
    While(flag){  
        flag ← false  
        for (i : 1 → n)  
            if(Finish[i]=false AND Need[i] ≤ Work){  
                Finish[i] ← true  
                Work ← Work + Allocation[i]  
                flag ← true  
            }//endif  
    }//endwhile  
    for (i : 1 → n) if (Finish[i]=false) return false  
    return true;  
}//End function
```

Example

- Consider system with 5 processes P_0, P_1, P_2, P_3, P_4 and 3 resources R_0, R_1, R_2
 - R_0 has 10 units, R_1 has 5 units, R_2 has 7 units
- Maximum resource requirement and allocated resource for each process

	R0	R1	R2
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3
Max			

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

- Is the system safe?
- P_1 requests 1 unit of R_0 and 2 unit of R_2 ?
- P_4 requests 3 unit of R_0 and 3 unit of R_1 ?
- P_0 requests 2 unit of R_1 . allocate?

Example: Check for safety

- Number of available resource in the system $Available(R_0, R_1, R_2) = (3, 3, 2)$
- Remaining request of each process ($Need = Max - Allocation$)

	R ₀	R ₁	R ₂
P ₀	7	5	3
P ₁	3	2	2
P ₂	9	0	2
P ₃	2	2	2
P ₄	4	3	3
Max			

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Example: Check for safety

- Number of available resource in the system $Available(R_0, R_1, R_2) = (3, 3, 2)$

	R ₀	R ₁	R ₂
P ₀	7	5	3
P ₁	3	2	2
P ₂	9	0	2
P ₃	2	2	2
P ₄	4	3	3
Max			

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	F	T	T	F	F
Work	(3, 3, 2)				

Example: Check for safety

- Number of available resource in the system $\text{Available}(R_0, R_1, R_2) = (3, 3, 2)$

	R ₀	R ₁	R ₂
P ₀	7	5	3
P ₁	3	2	2
P ₂	9	0	2
P ₃	2	2	2
P ₄	4	3	3
Max			

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	T	T	T	T
Work	(1, 4, 5, 7)				

System is safe
(P₁, P₃, P₄, P₀, P₂)

Algorithm for resource request

- $\text{Request}[i]$ Resource requesting vector of process P_i
 - $\text{Request}[3,2] = 2$: P_3 requests 2 units of resource R_2
- When P_i make a resource request, the system checks
 - ① $\text{if}(\text{Request}[i] > \text{Need}[i])$
Error(Request higher than declared number)
 - ② $\text{if}(\text{Request}[i] > \text{Available})$
Block(Not enough resource, process has to wait)
 - ③ Set the new resource allocation for the system
 - $\text{Available} = \text{Available} - \text{Request}[i]$
 - $\text{Allocation}[i] = \text{Allocation}[i] + \text{Request}[i]$
 - $\text{Need}[i] = \text{Need}[i] - \text{Request}[i]$
 - ④ Allocate resource based on the result of new system safety check
if(Safe(New Resource Allocation State))
 Allocate resource for P_i as requested
else
 P_i has to wait
 Recover former state (Available, Allocation, Need)

Example: P_1 request (1, 0, 2)

- $\text{Request}[1] \leq \text{Available}$ $((1, 0, 2) \leq (3, 3, 2)) \Rightarrow$ It's possible to allocate
- If allocate resource: **Available = (2, 3, 0)**

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	F	F	F	F	F
Work	(2,3,0)				

Example: P_1 request (1, 0, 2)

- $\text{Request}[1] \leq \text{Available}$ $((1, 0, 2) \leq (3, 3, 2)) \Rightarrow$ It's possible to allocate
- If allocate resource: **Available = (2, 3, 0)**

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	F	F	F	F	F
Work	(2,3,0)				



Example: P_1 request (1, 0, 2)

- Request[1] ≤ Available ((1, 0, 2) ≤ (3, 3, 2)) ⇒ It's possible to allocate
- If allocate resource: Available = (2, 3, 0)

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	F	T	F	F	F
Work	(5,3,2)				

Example: P_1 request (1, 0, 2)

- $\text{Request}[1] \leq \text{Available}$ $((1, 0, 2) \leq (3, 3, 2)) \Rightarrow$ It's possible to allocate
- If allocate resource: $\text{Available} = (2, 3, 0)$

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	F	T	F	F	F
Work	(5,3,2)				

Example: P_1 request (1, 0, 2)

- $\text{Request}[1] \leq \text{Available}$ $((1, 0, 2) \leq (3, 3, 2)) \Rightarrow$ It's possible to allocate
- If allocate resource: **Available = (2 , 3, 0)**

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	F	T	F	T	F
Work	(7,4,3)				



Example: P_1 request (1, 0, 2)

- Request[1] ≤ Available ((1, 0, 2) ≤ (3, 3, 2)) ⇒ It's possible to allocate
- If allocate resource: Available = (2, 3, 0)

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	F	T	F	T	T
Work	(7,4,5)				

Example: P_1 request (1, 0, 2)

- $\text{Request}[1] \leq \text{Available}$ $((1, 0, 2) \leq (3, 3, 2)) \Rightarrow$ It's possible to allocate
- If allocate resource: **Available = (2, 3, 0)**

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	T	T	F	T	T
Work	(7,5,5)				

Example: P_1 request (1, 0, 2)

- $\text{Request}[1] \leq \text{Available}$ $((1, 0, 2) \leq (3, 3, 2)) \Rightarrow$ It's possible to allocate
- If allocate resource: $\text{Available} = (2, 3, 0)$

	R0	R1	R2
P0	0	1	0
P1	3	0	2
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	7	4	3
P1	0	2	0
P2	6	0	0
P3	0	1	1
P4	4	3	1
Need			

Process	P_0	P_1	P_2	P_3	P_4
Finish	T	T	T	T	T
Work	(10, 5, 7)				

Request is accepted



Example: (continue)

- P_4 request 3 units of R_0 and 3 units of R_2
 - $\text{Request}[4] = (3, 0, 3)$
 - $\text{Available} = (2, 3, 0)$ $\Rightarrow$ Resource is not enough, P_4 has to wait
- P_0 request 2 units of R_1
 - $\text{Request}[0] \leq \text{Available} ((0, 2, 0) \leq (2, 3, 0)) \Rightarrow$ It's possible to allocate
 - If allocate: $\text{Available} = (2, 1, 0)$
 - Perform Safe algorithm $\Rightarrow$ All processes may not finish
 $\Rightarrow$ if accepted, system may be unsafe
 $\Rightarrow$ Resource is sufficient but do not allocate, P_0 has to wait

- Deadlock conception
- Conditions for resource deadlocks
- Solutions for deadlock
- Deadlock prevention
- Deadlock avoidance
- **Deadlock detection and recovery**

Introduction

- Rule
 - Do not apply deadlock prevention or avoidance method, allow deadlock to occur
 - Timely check if the system has deadlock or not if yes then find a solution
 - To function properly, system has to provide
 - Algorithm to check if the system is deadlock or not
 - Algorithm to recover from deadlock
- Deadlock detection
 - Algorithm for showing the deadlock
- Deadlock recovery
 - Terminate process
 - Resource preemptive



Algorithm to point out deadlock: Introduction

- Apply for system that has resource types with many units
- Similar to **banker algorithm**
- Data structures
 - **Available** Vector with length m : Available resource in the system
 - **Allocation** Matrix $n * m$: Resources allocated to processes
 - **Request** Matrix $n * m$: Resources requested by processes
- Local structures
 - **Work** Vector with length m : available resource
 - **Finish** Vector with length n : process is able to finish or not
- Rule
 - $\leq$ relations between Vectors
 - Rows in matrix $n * m$ are processed similar to vectors



Algorithm to point out deadlock

```
BOOL Deadlock(Current Resource-Allocation State){
    Work ← Available
    for (i : 1 → n)
        if (Allocation[i] ≠ 0) Finish[i] ← false
        else Finish[i] ← true;           //Allocation = 0 not in waiting circular
    flag ← true
    While(flag){
        flag ← false
        for (i : 1 → n)
            if (Finish[i] = false AND Request[i] ≤ Work){
                Finish[i] ← true
                Work ← Work + Allocation[i]
                flag ← true
            } //endif
        } //endwhile
    for (i : 1 → n) if (Finish[i] = false) return true;
    return false;                       //Finish[i] = false, process Pi is in deadlock
} //End function
```

Example

- 5 processes P_0, P_1, P_2, P_3, P_4 ; 3 resources R_0, R_1, R_2
 - Resource R_0 has 7 units, R_1 has 2 units, R_2 has 6 units
- Resource allocation status at time t_0

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	F	F	F	F	F
Work	(0,0,0)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	F	F	F	F
Work	(0,1,0)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	F	T	F	F
Work	(3,1,3)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	F	T	T	F
Work	(5,2,4)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	F	T	T	F
Work	(5,2,4)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	T	T	T	F
Work	(7,2,4)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	0
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	T	T	T	T
Work	(7,2,6)				

System has no deadlock
(P₀, P₂, P₃, P₁, P₄)



Example

- At time t_1 : P_2 request 1 more resource unit of R_2
- Resource allocation status

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	6	0	2
Request			

- Available resource $(R_0, R_1, R_2) = (0, 0, 0)$

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	6	0	2
Request			

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	F	F	F	F	F
Work	(0,0,0)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	3
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	6	0	2
Request			

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	F	F	F	F
Work	(0,1,0)				

Example

	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	6	0	2
Request			

Process	P ₀	P ₁	P ₂	P ₃	P ₄
Finish	T	F	F	F	F
Work	(0,1,0)				

- P₀ may finish but the system is deadlock.
Processes are waiting for each other(P₁, P₂, P₃, P₄)



Deadlock recovery: Process termination method

Rule: Terminate processes in deadlock and take back allocated resource

- Terminate all processes
 - Quick to eliminate deadlock
 - Too expensive
 - Killed processes may be almost finished
- Terminate processes consequently until deadlock is removed
 - After process is terminated, check if deadlock is still exist or not
 - Deadlock checking algorithm complexity is $m * n^2$
 - Need to point out the order of process to be terminated
 - Process's priority
 - Process's turn around time, how long until process finish
 - Resources that process is holding, need to finish
 - ...
- Process termination's problem
 - Process is updating file $\Rightarrow$ File is not complete
 - Process is using printer $\Rightarrow$ Reset printer's status

Deadlock recovery: Resource preemption method

Preempt continuously several resources from a deadlocked processes and give to other processes until deadlock is removed

Need to consider:

① Victim's selection

- Which resource and which process is selected?
- Preemption's order for smallest cost
- Amount of holding resource, usage time. . .

② Rollback

- Rollback to a safe state before and restart
- Require to store state of running process

③ Starvation

- One process is preempted many times $\Rightarrow$ infinite waiting
- Solution: record the number of times that process is preempted

Another solution ?



Chapter Summary

- Deadlock is the situation when 2 or more processes are independently waiting for an event that can only be caused by these processes
- Deadlock occurs when the 4 following conditions exist
 - Critical resource
 - Wait before entering the critical region
 - No resource-reallocation system
 - Circular waiting
- 3 Approaches to handle Deadlock
 - Prevention
 - Attack the deadlock's happening conditions
 - Avoidance
 - Make the system avoid situations that may cause the deadlock to happen
 - Detection and handling
 - Allow deadlock to happen, indicate deadlock and handle later

Ví dụ minh họa

- 5 tiến trình P_0, P_1, P_2, P_3, P_4 ; 3 tài nguyên R_0, R_1, R_2
 - Tài nguyên R_0 có 6 đơn vị, R_1 có 4 đơn vị, R_2 có 7 đơn vị
- Trạng thái cung cấp tài nguyên tại thời điểm t_0

	R0	R1	R2
P0	0	1	1
P1	1	0	0
P2	3	0	2
P3	2	1	1
P4	0	2	2
Allocation			

	R0	R1	R2
P0	0	0	0
P1	2	1	2
P2	0	0	2
P3	1	0	0
P4	6	0	2
Request			